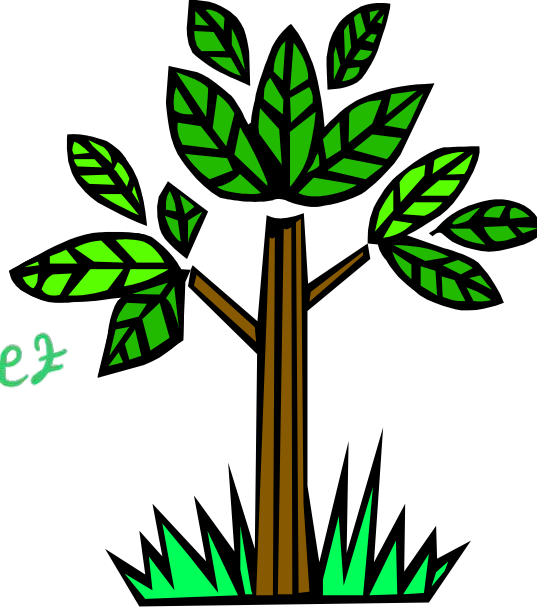


BÖLÜM 4

TREES (AĞAÇLAR)

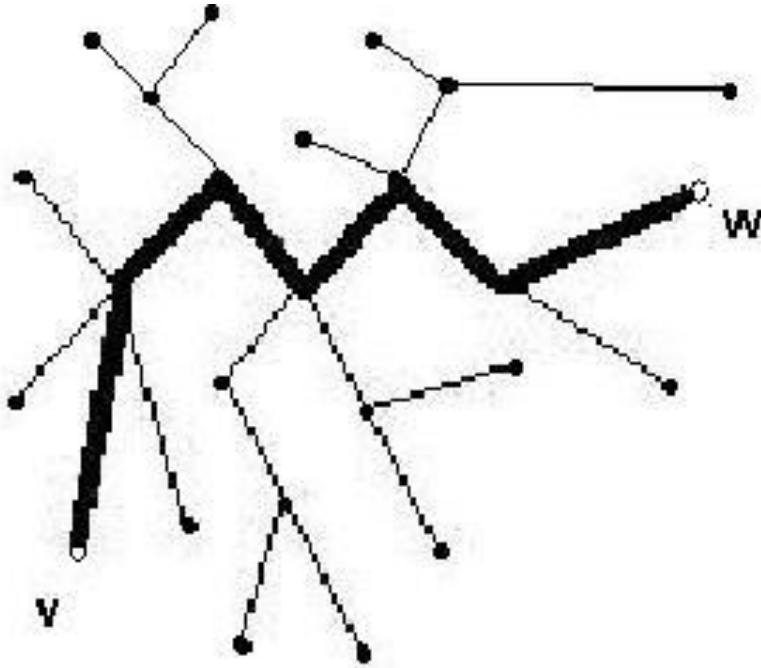


↳ ağaçlar
dairesel yapı içermez

1. ağaçlarda bir

↳ grafitir ve
daireseel yapı
içermez

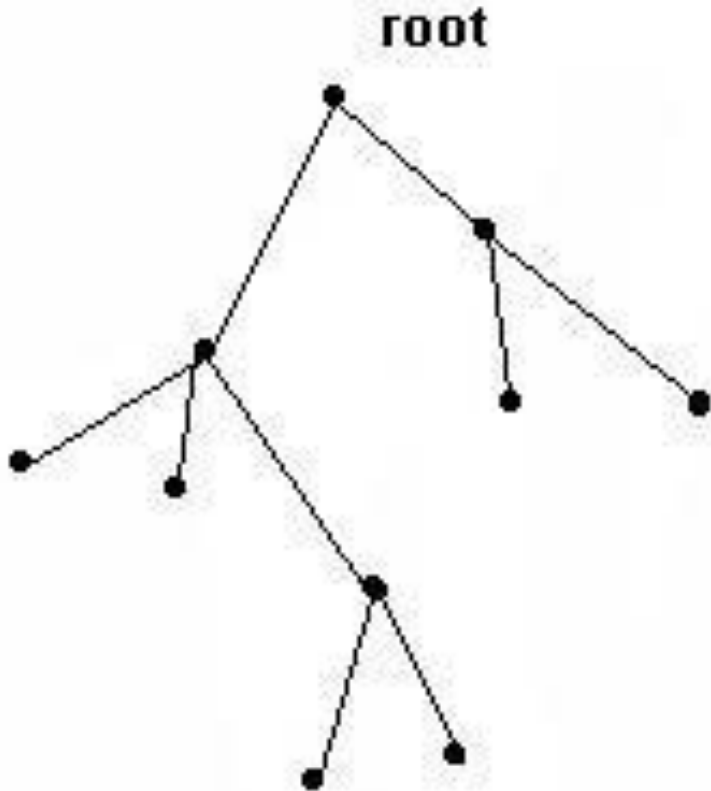
GİRİŞ



Bir ağaç

- v ve w düğüm çiftinden oluşan basit bir grafitir
- v 'den w 'ya tek bir yol vardır
- daireseel bir yapı içermez

Köklü Ağaç (Rooted tree)



Düğümlerinden biri kök olarak (root) belirlenmiş bir ağaçtır

Düğüm seviyesi ve ağacın yüksekliği

T köklü bir ağaç olsun:

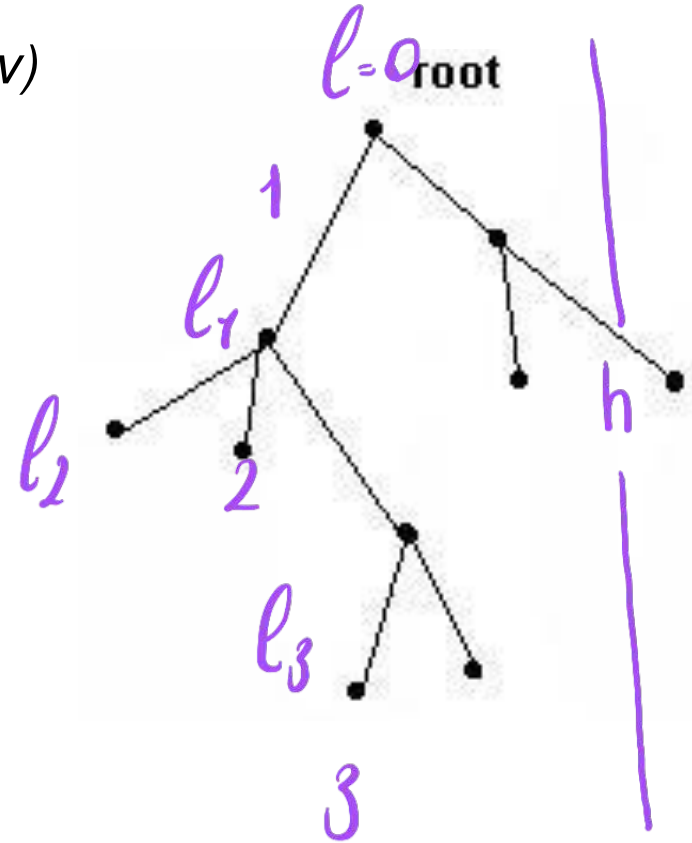
- Bir düğümün seviyesi, o düğümün $l(v)$ bulunduğu yerden ağacın köküne olan yolun uzunluğudur
- Bir ağacın yüksekliği, o ağacın maksimum seviyeye sahip olan düğümünün değerine eşittir

$$h = \max \{ l(v) \}$$

$$v \in V(T)$$

■ Örnek:

- sağdaki ağacın yüksekliği 3

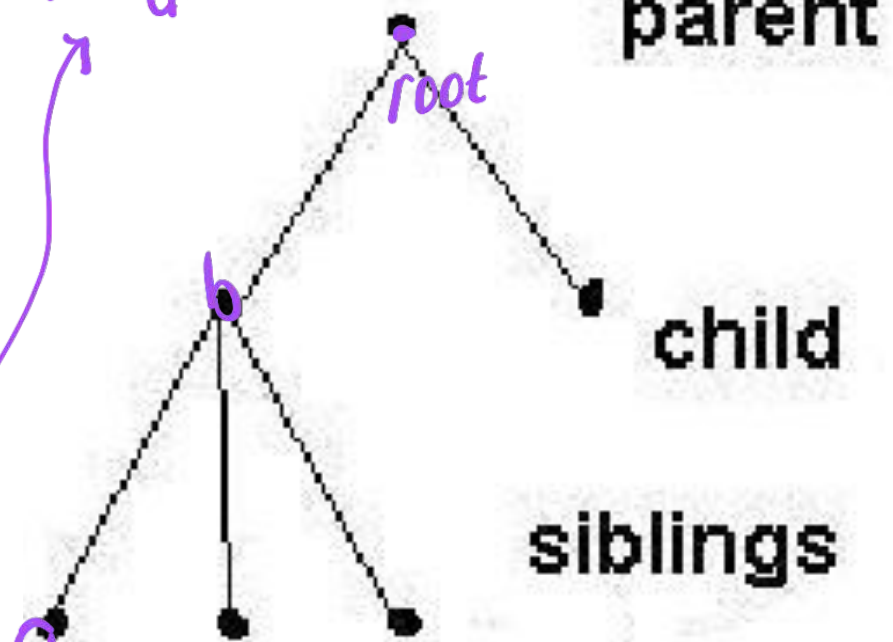


Terminol → Kendisinden / internal

Terminoloji

- Parent (ana/baba)
- Ancestor (ata)
- Child (çocuk)
- Descendant (torun)
- Siblings(kardeş düğüm)
- Terminal vertices
(uç düğüm)
- Internal vertices
(ara düğüm)
- Subtrees (alt ağaçlar)

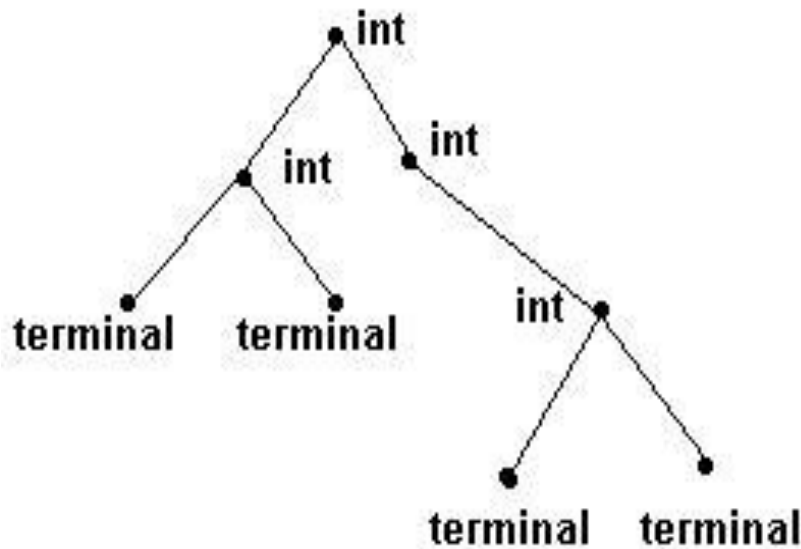
kendisinden dal
çıkma?



b a'nın
atasıdır

kendisinden
dal çıkar

Ara (internal) ve Uç (terminal) Düğümmler



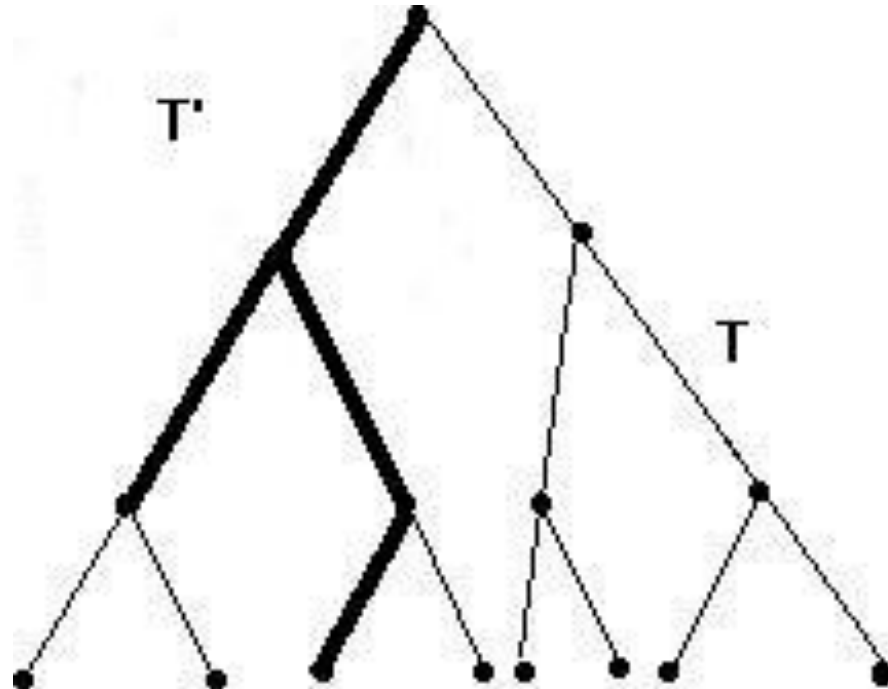
- En az bir çocuğa sahip olan düğümler ara (internal) düğümlerdir
- Hiç çocuğu olmayan düğümler uç (terminal) düğümlerdir
- Örnekteki ağaçta 4 adet internal ve 4 adet terminal düğüm mevcuttur

Altağaçlar (Subtrees)

Bir T ağacının altağacı (subtree) T' olup

- $V(T') \subseteq V(T)$ and
- $E(T') \subseteq E(T)$

↳ rootun size verilmesi lazım



Ağaçların Karakteristiği

Theorem

Eğer T , n düğümlü bir graf ise aşağıdakiler doğrudur:

a) T bir ağaçtır

b) T bağlantılı (connected) ve acyclic

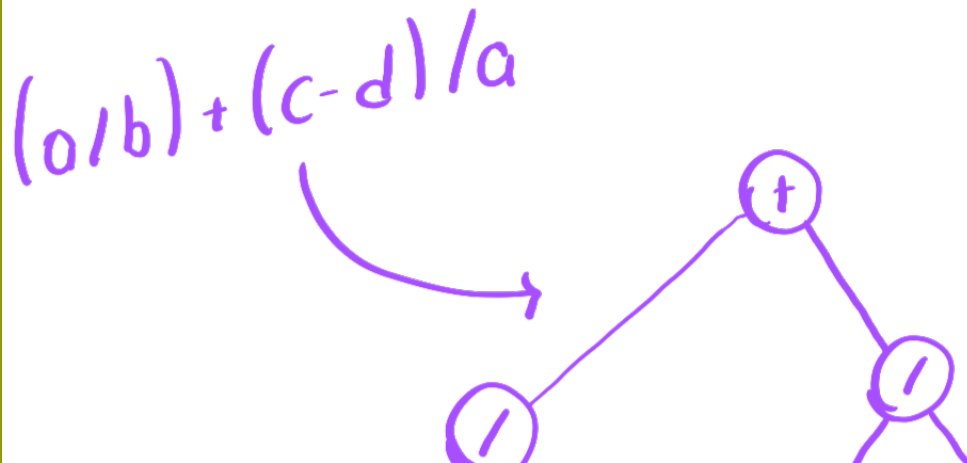
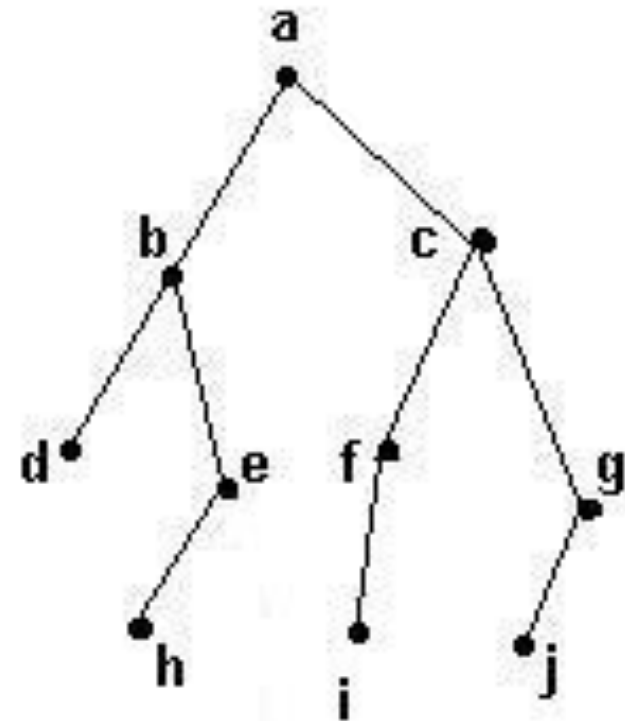
□ (“acyclic” = döngü mevcut değil)

c) T bağlantılı ve $n-1$ kenara sahiptir

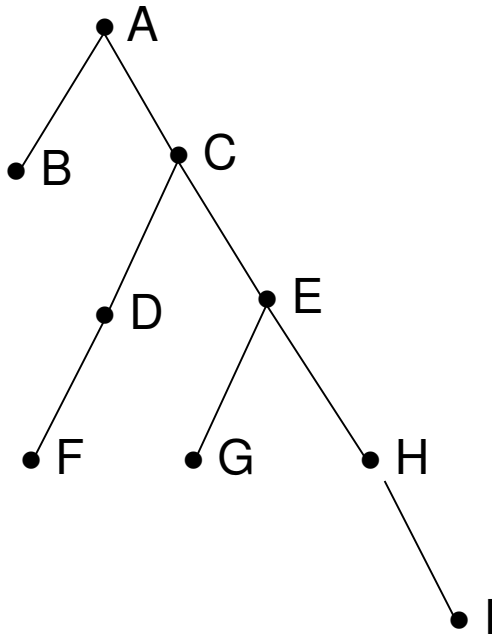
d) T acyclic ve $n-1$ kenara sahiptir

İkili Ağaçlar (Binary trees)

Bir ikili ağaçta, her bir düğüm en fazla iki çocuğa sahiptir. Bir düğüm 0, 1 veya 2 çocuğa sahip olabilir



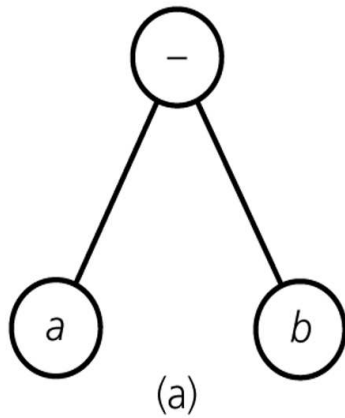
Binary Tree -- Örnek



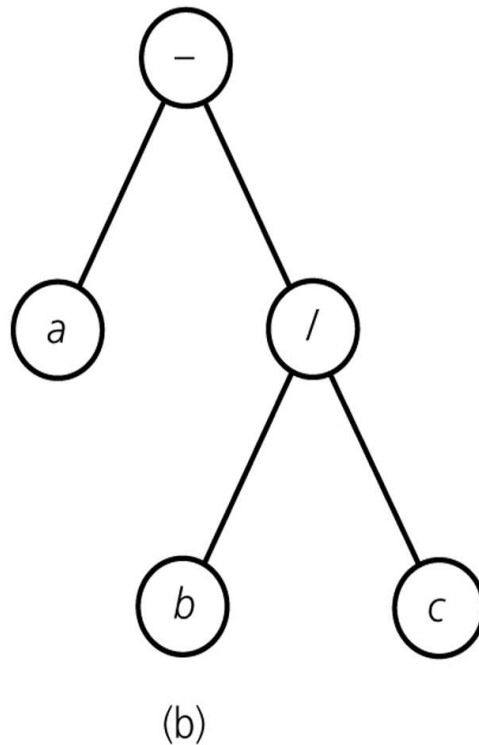
- A is the root.
- B is the left child of A, and C is the right child of A.
- D doesn't have a right child.
- H doesn't have a left child.
- B, F, G and I are leaves.

Binary Tree – Matematiksel İfadelerin Gösterilimi

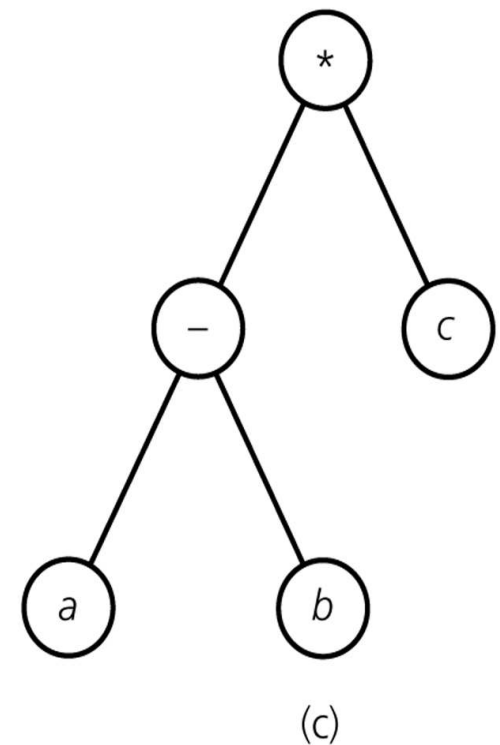
$a - b$

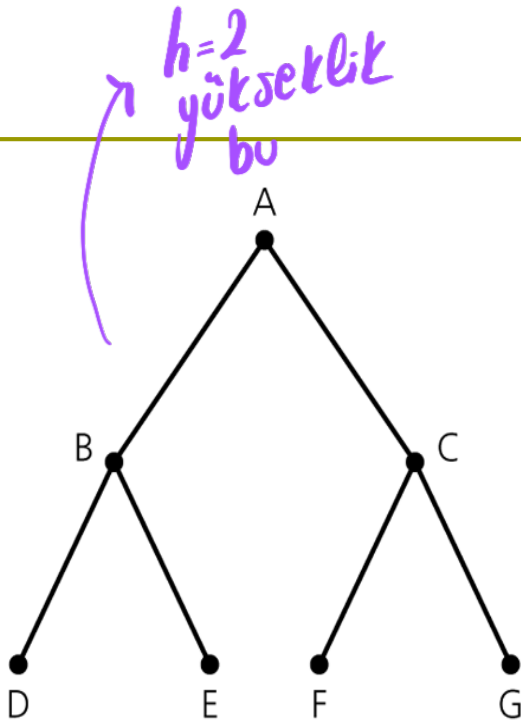


$a - b / c$

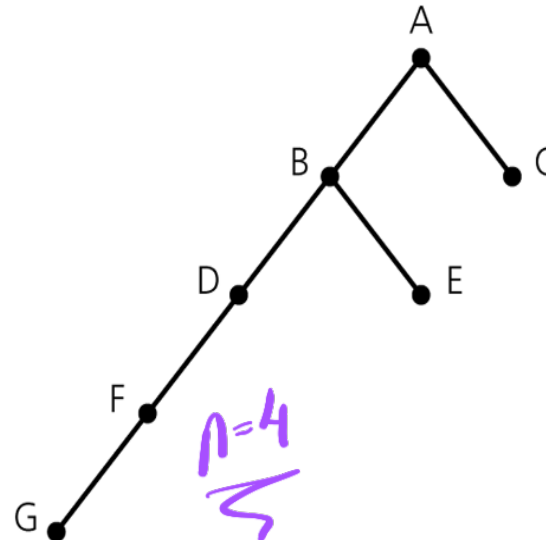


$(a - b) * c$

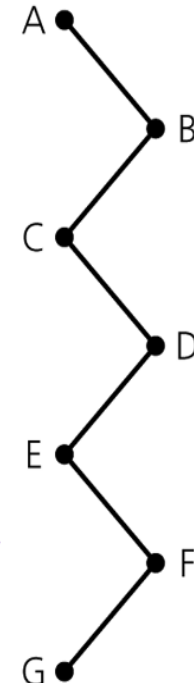




(a)



(b)



(c)

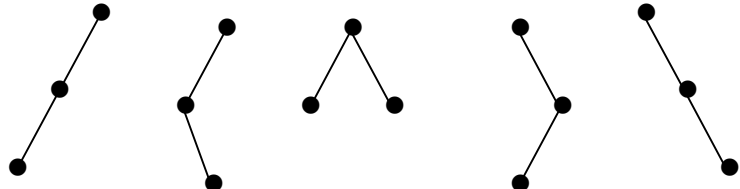
Aynı düğüm sayısına sahip, farklı yüksekliklerdeki İkili Ağaçlar

Aynı düğüm sayısına sahip farklı İkili Ağaçlar

$n=0 \rightarrow$ empty tree

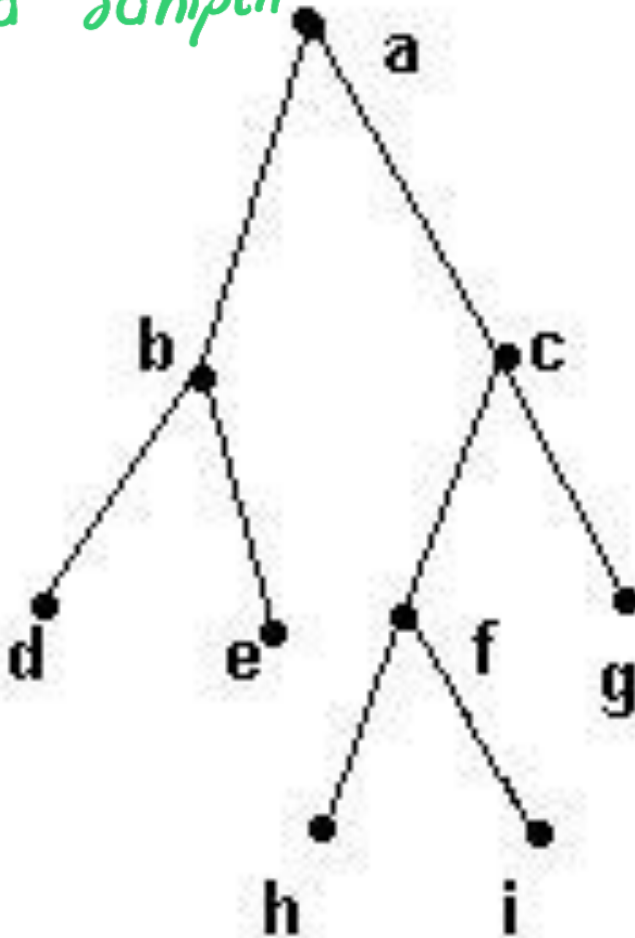
$n=1 \rightarrow$  (1 tree)

$n=2 \rightarrow$  (2 trees)

$n=3 \rightarrow$  (5 trees)

Tam İkili Ağaç (Full binary tree)

full binary tree'lerde uç düğümler hariç her düğüm 2 çocuğa sahiptir



Bir *full* binary tree 'de uç düğümler hariç her düğüm iki çocuğa sahiptir.

Burada internal düğümlerin içinde var kök düğüm de kök düğümde bir internal düğümdür.

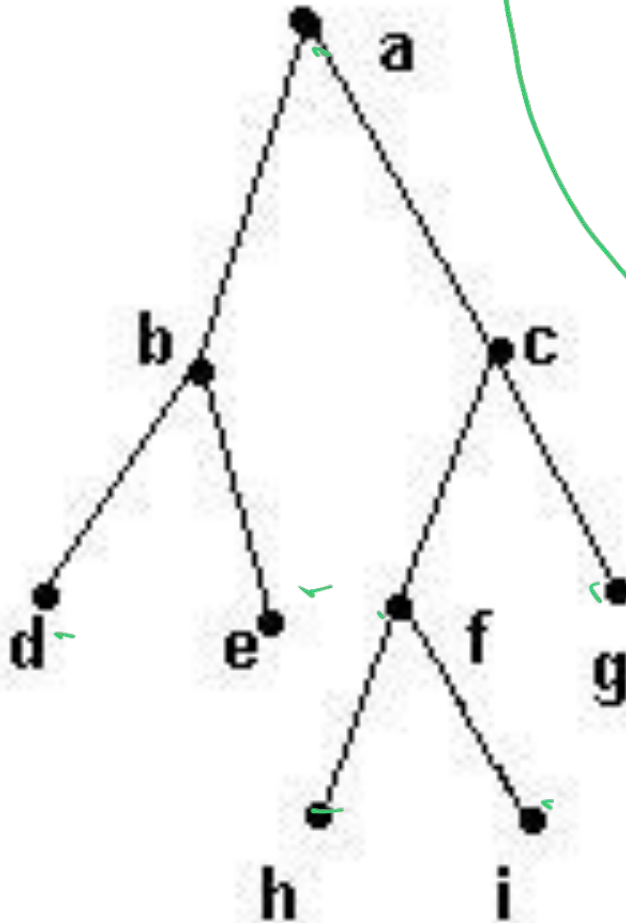
Tam ikili ağaç (full binary tree)'da :

* t adet uç düğüm varsa ağacın yüksekliği $\lg(t) \leq h$

* i adet ara düğümü varsa $(i+1)$ adet uç düğümü vardır

* i adet ara düğümü varsa toplam düğüm sayısı $(2i+1)$

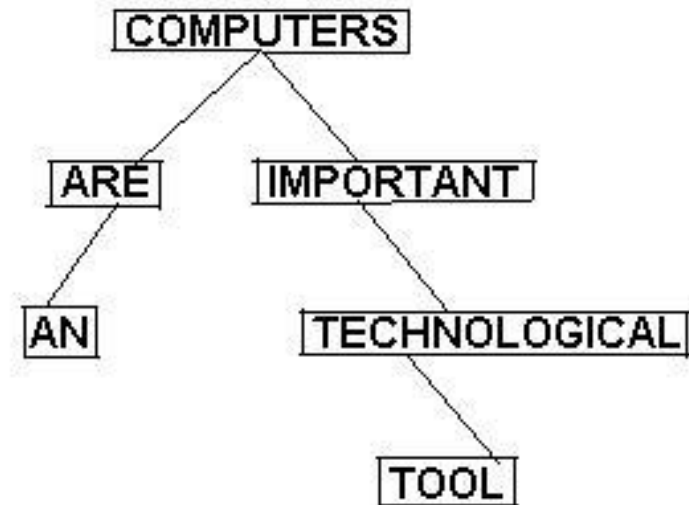
■ Örnek: $k = 4$ internal vertices (a, b, c ve f) ve 5 terminal vertices (d, e, g, h and i) toplamda 9 düğüm



İkili Arama Ağaçları (Binary Search Trees)

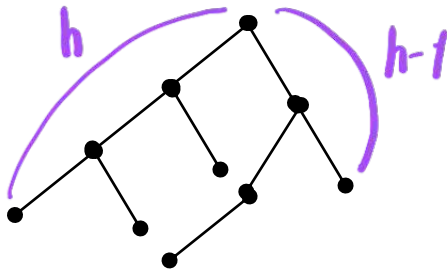
- Veri düğümlere yerleştirilir
- Veri alfabetik sırada ağaca yerleştirilir. Düğümün sol tarafındaki veri, o düğümdeki veriden daha küçüktür
- Ve düğümün sağ tarafındaki veri de o düğümdeki veriden daha büyüktür

Örnek: "Computers are an important technological tool"

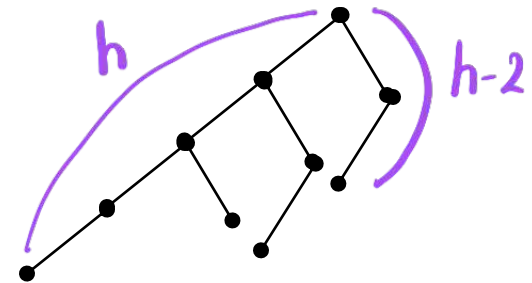


Dengeli Ağaç (Balanced Tree)

Yüksekliği h olan bir ağacın yapraklarının seviyesi h veya $h-1$ ise dengeli ağaç adını alır



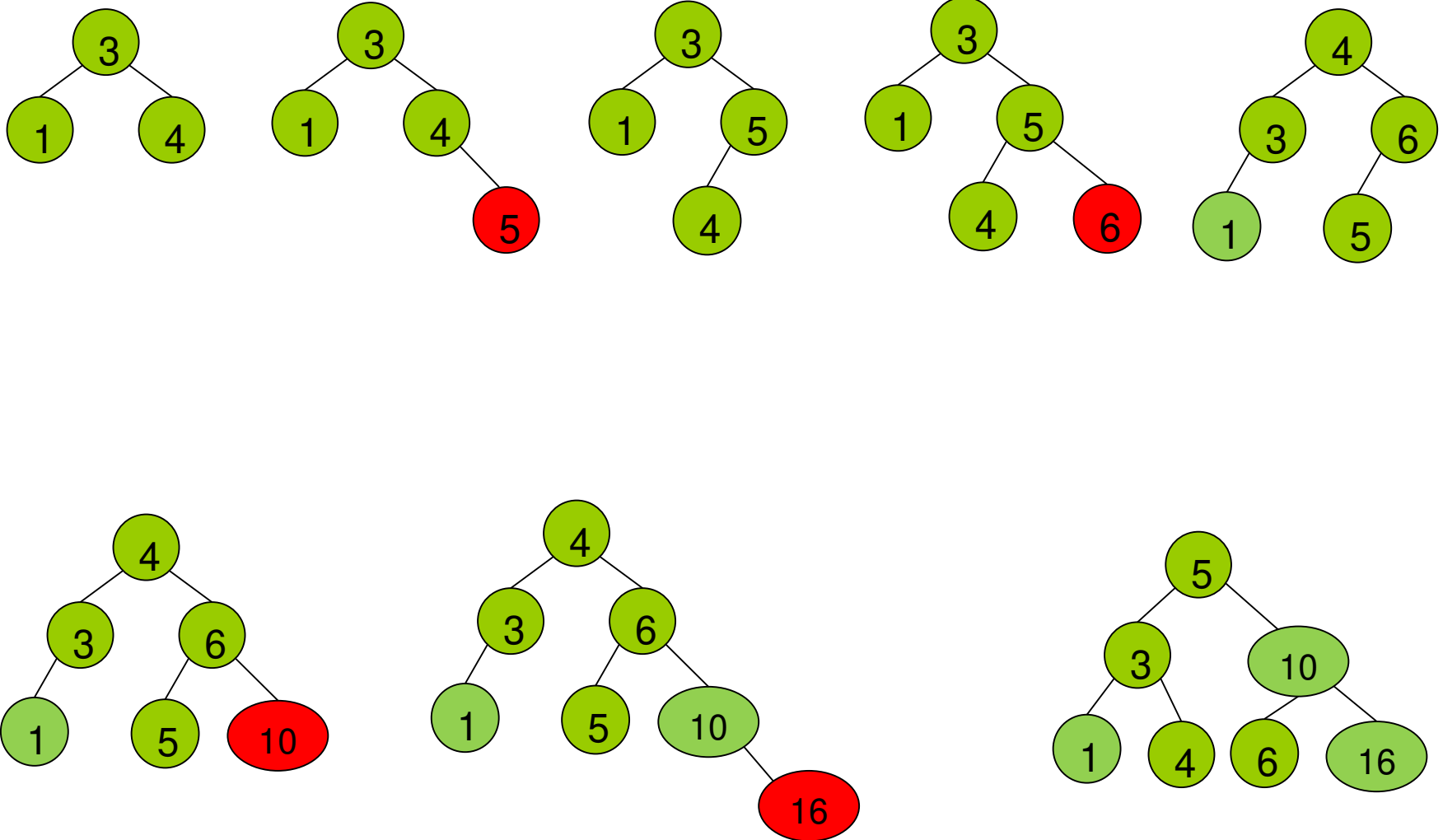
Dengeli ağaç



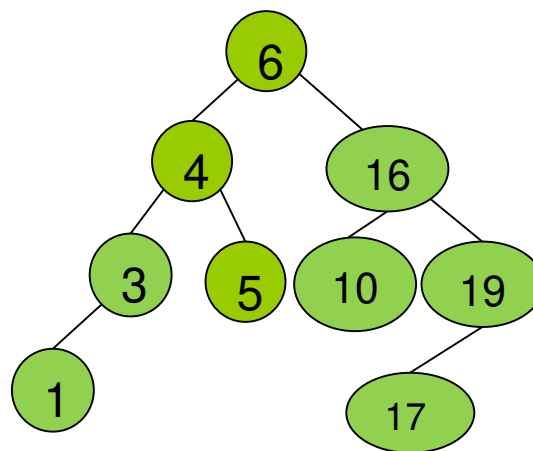
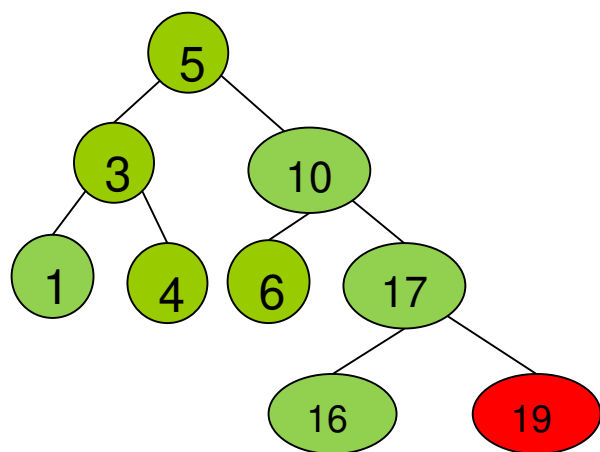
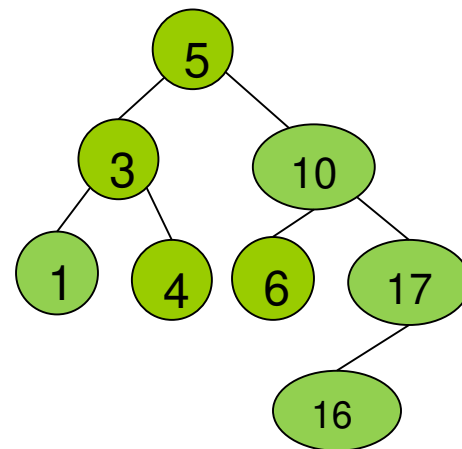
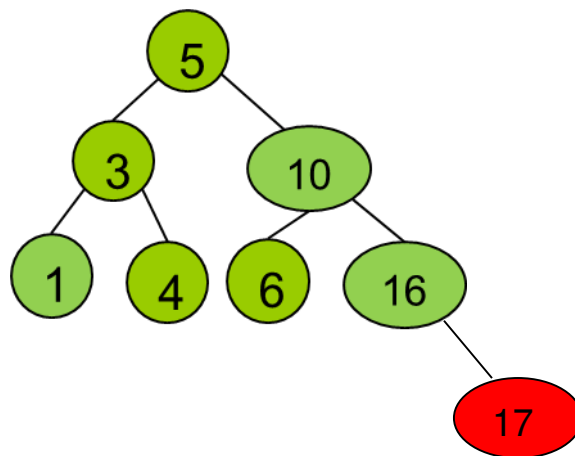
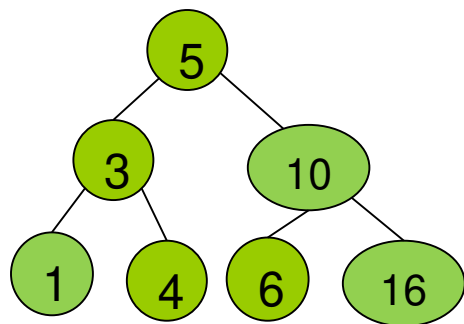
Dengesiz ağaç

Örnek : Binary Search Tree Oluşturma

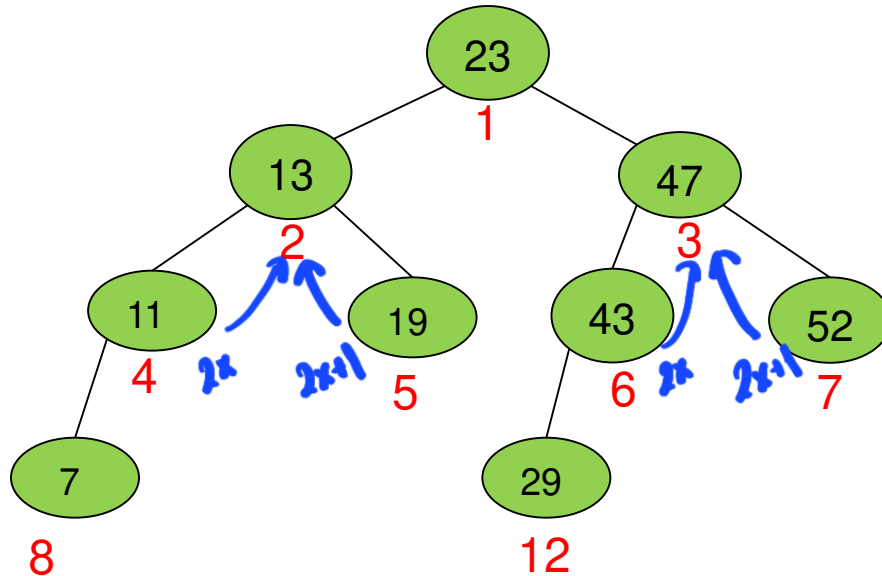
1 3 4 5 6 10 16 17 19



1 3 4 5 6 10 16 17 19



Binary Search uygun şekilde verilmiş olan **23 13 19 47 11 43 7 29 52** diziyi belirli bir kurala göre bir ağaca yerleştirelim.



29
7

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
23	13	47	11	19	43	52	7	0	0	0	29	0	0	0
0	0		0				0							

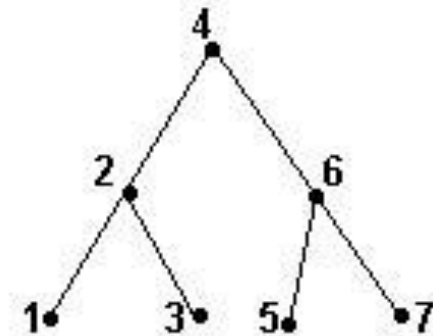
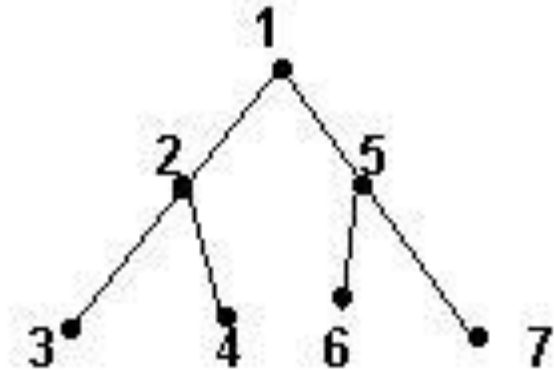
Ağacın Düğümlerinin Listesi (Tree Traversals)

- 1: Pre-order traversal
(önden sıralı)

↳ OLR

- 2: In-order traversal
(içten sıralı)

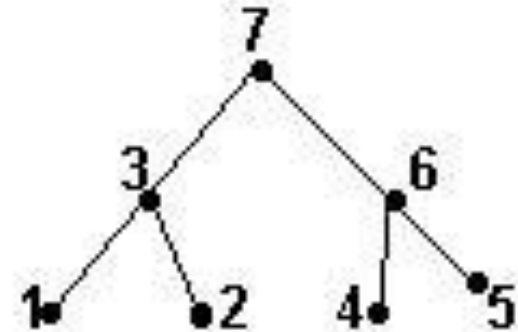
↳ LOR



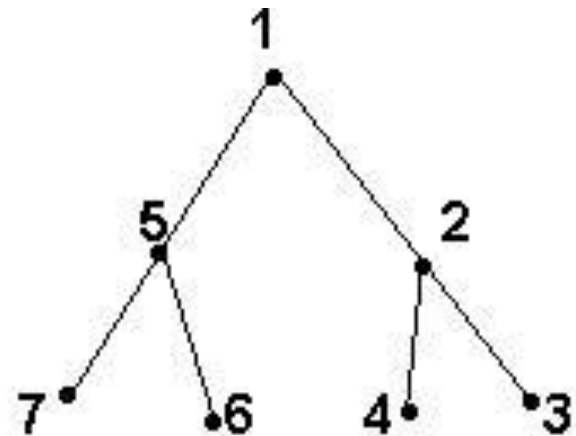
devam....

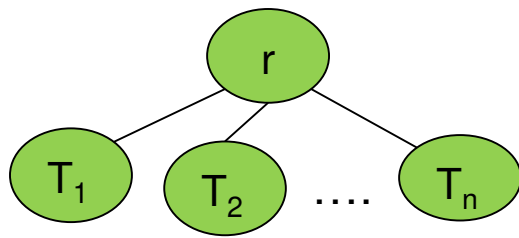
- 3: Post-order traversal
(sondan sıralı)

↳ LRO

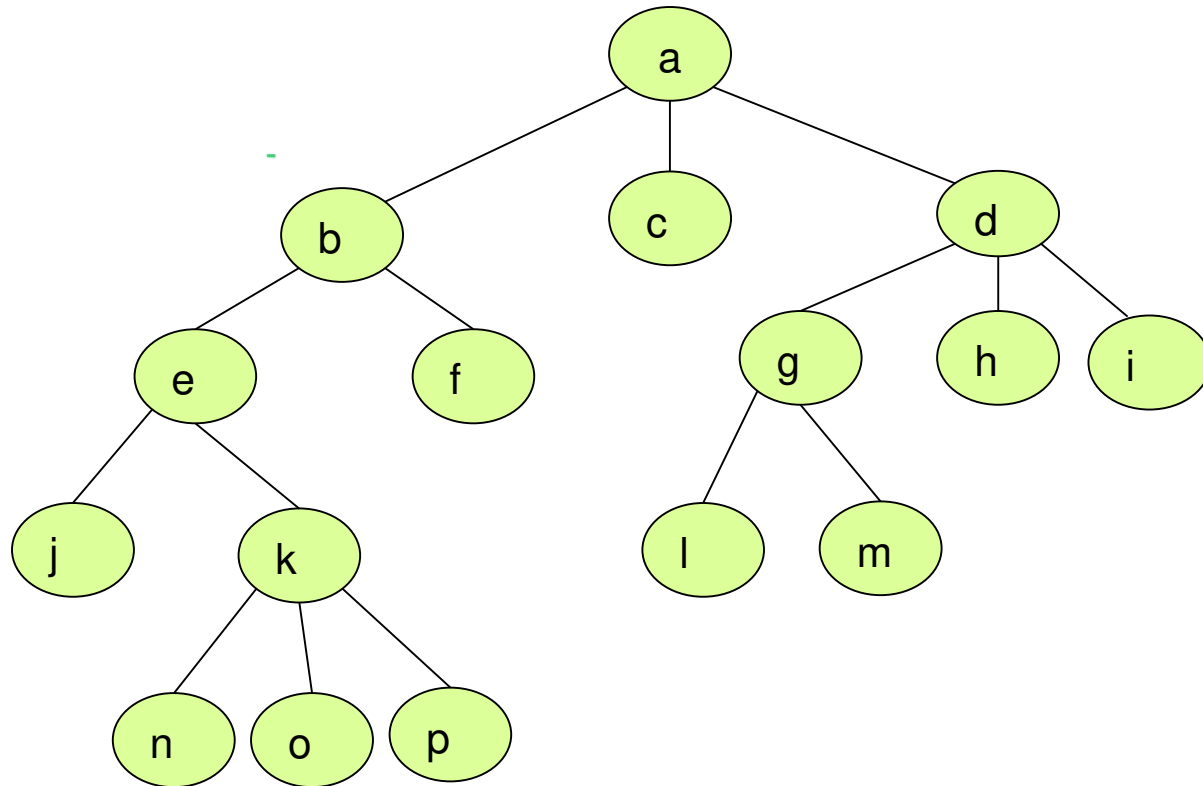


- 4: Reverse post-order traversal (ters önden sıralı)

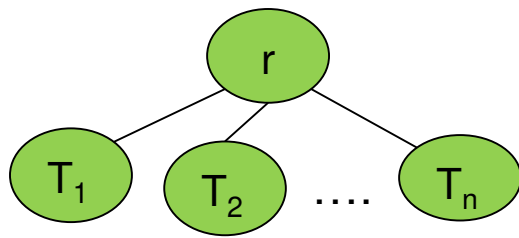




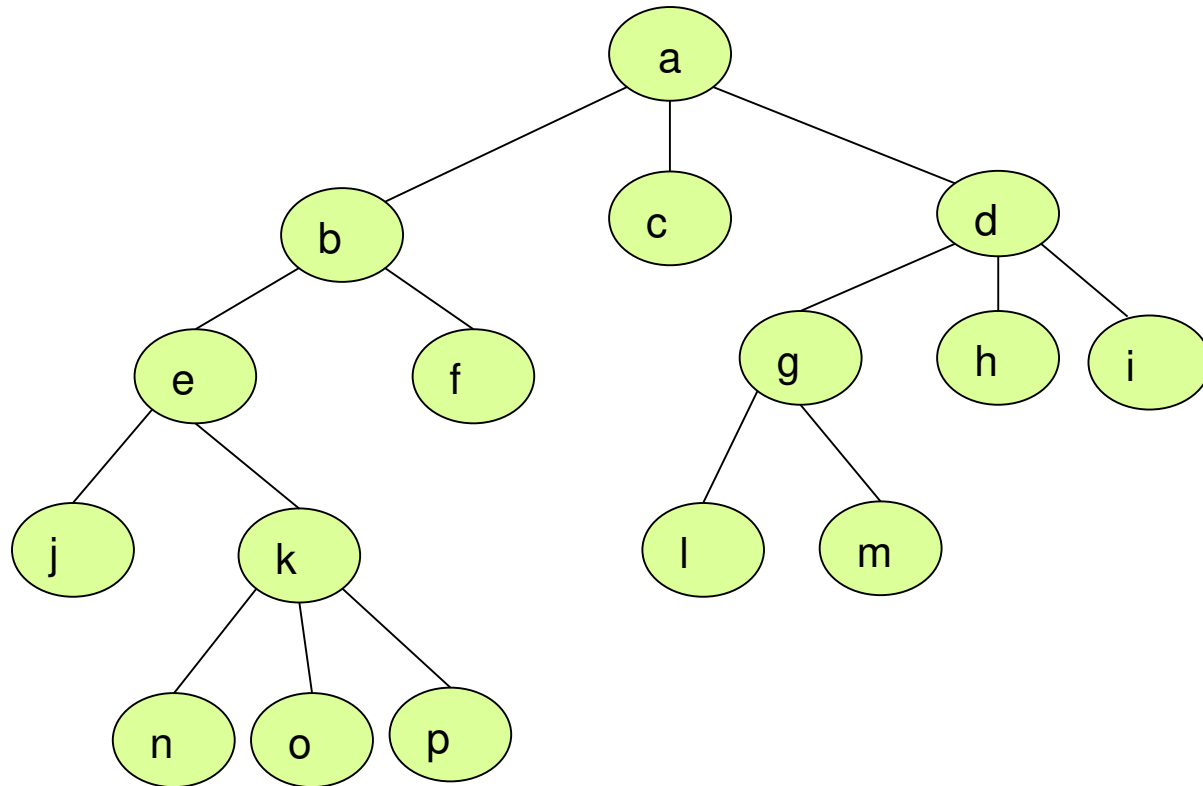
Pre-order $r T_1 T_2 \dots T_n$



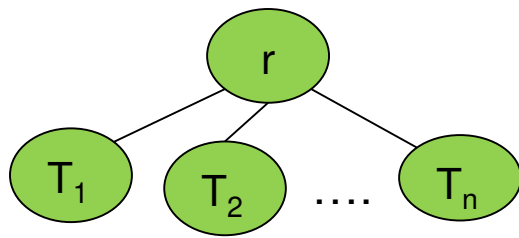
a b e j k n o p f c d g l m h i



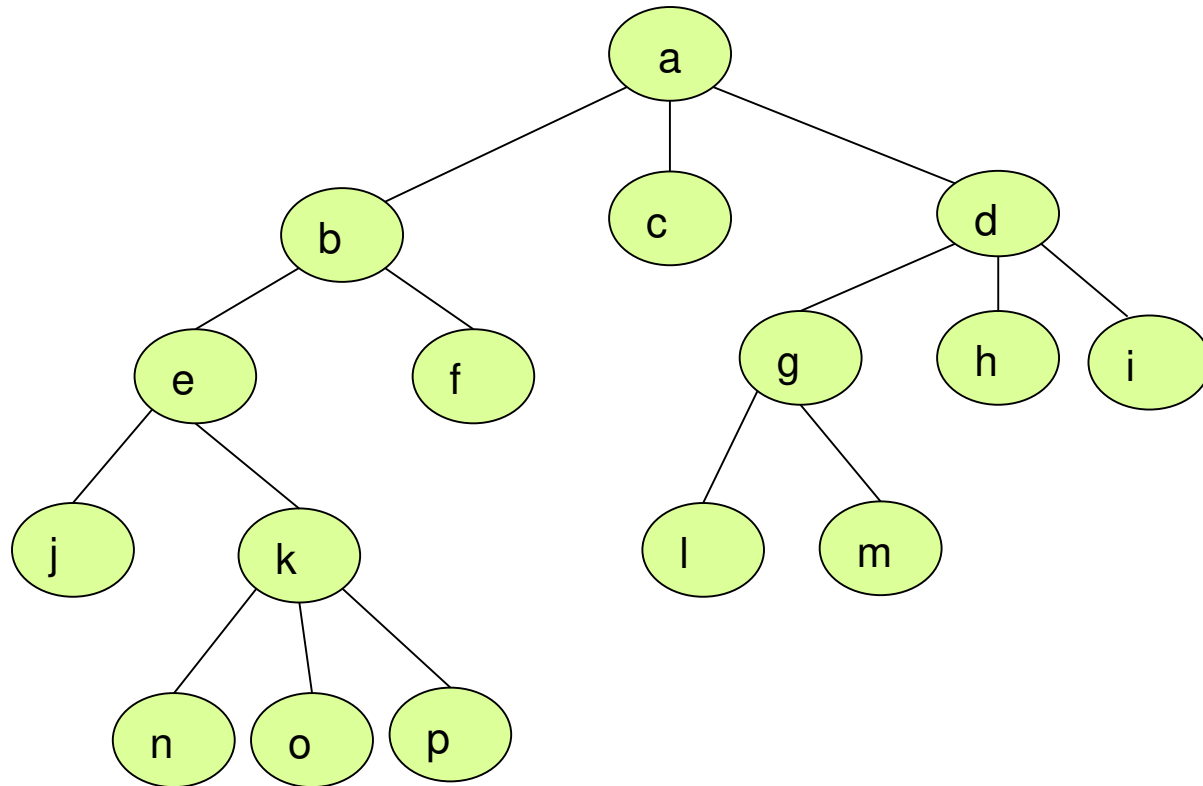
In-order $T_1 \ r \ T_2 \dots T_n$



j e n k o p b f a c l g m d h i



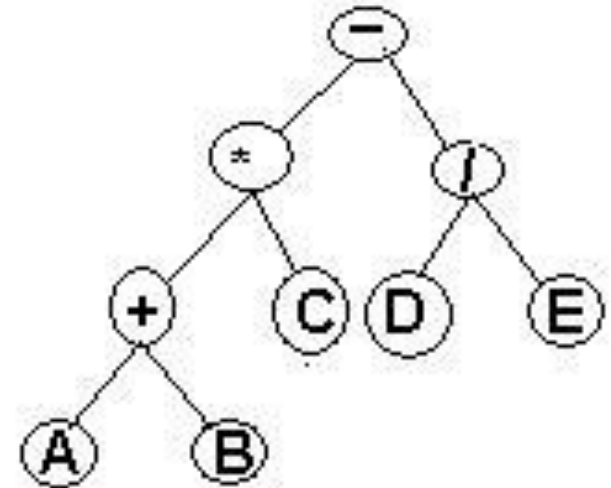
Post-order $T_1 T_2 \dots T_n r$



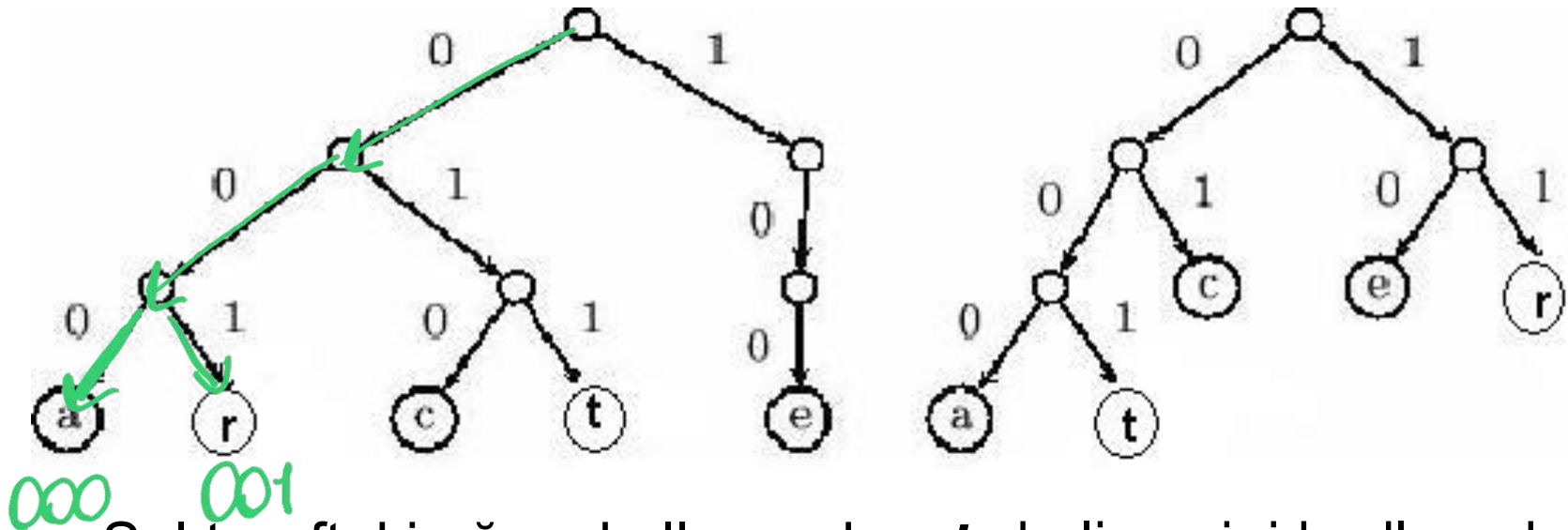
j n o p k e f b c l m g h i d a

Aritmetik İfadeler

- Standart: *infix* formu
 $(A+B) * C - D / E$
- Tüm parentezli form (in-order & parenthesis):
 $((A + B) * C) - (D / E)$
- *Postfix* formu (Polish notasyonunun tersi):
 $A B + C * D E / -$
- *Prefix* formu (Polish notasyonu):
 $- * + A B C / D E$



Huffman Kodları (Huffman Codes)



- Sol taraftaki ağacı kullanarak **rate** kelimesini kodlarsak
001 000 011 100
- Sağ taraftaki ağacı kullanarak **rate** kelimesini kodlarsak
11 000 001 10

↳ hufmon
kodlarını
freeleri kullanarak

↳ ASCII tablosundaki
butun karakterler
8 bitle kodlanıyor

HUFFMAN KODLAMA

Elimizde a, e, k, l, m ve z sembollerinden sırası ile 25, 14, 8, 7, 4 ve 2 adet olsun. Her bir sembol için Huffman kod değerini bulalım.

Adım 1: Sembollerin kullanım sıklıkları küçükten büyüğe doğru sıralanır.

2 4 7 8 14 25

Adım 2: Sıklık değerleri, soldan başlayarak sıradaki iki sıklık değerinin toplamalarının sıralamayı bozmayacak şekilde diziye yerleştirilmesi ile devam edilir.

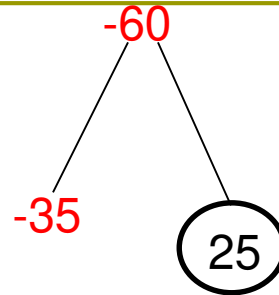
2 4 -6 7 8 -13 14 -21 25 -35 -60

Toplam değerleri kırmızı ile gösterilmiştir. Eksi değer bir ara toplam olduğunu söyler.

n tane sembolümüz varsa, genişletilmiş dizimiz **$2n-1$** adet olacaktır.

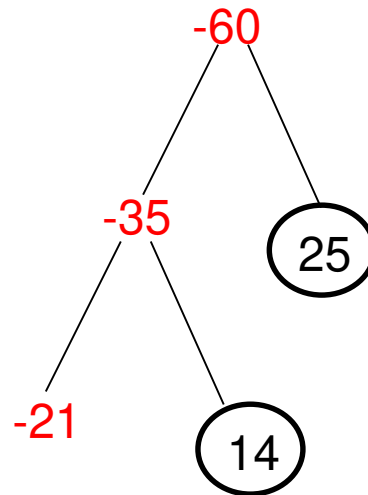
Adım 3: En sondaki eleman Huffman kodlama ağacının kök değeri olacak şekilde ikili ağaç şeklinde diğer elemanlarda ağaca belli bir kural ile yerleştirilir.

Bir sonraki $(2n-2)$ değeri ağacın sol dalına, $(2n-3)$ değeri de sağ dalına yerleşir.



25 değeri "a" sembolün sıklık değeri olup, bir yapraktır.

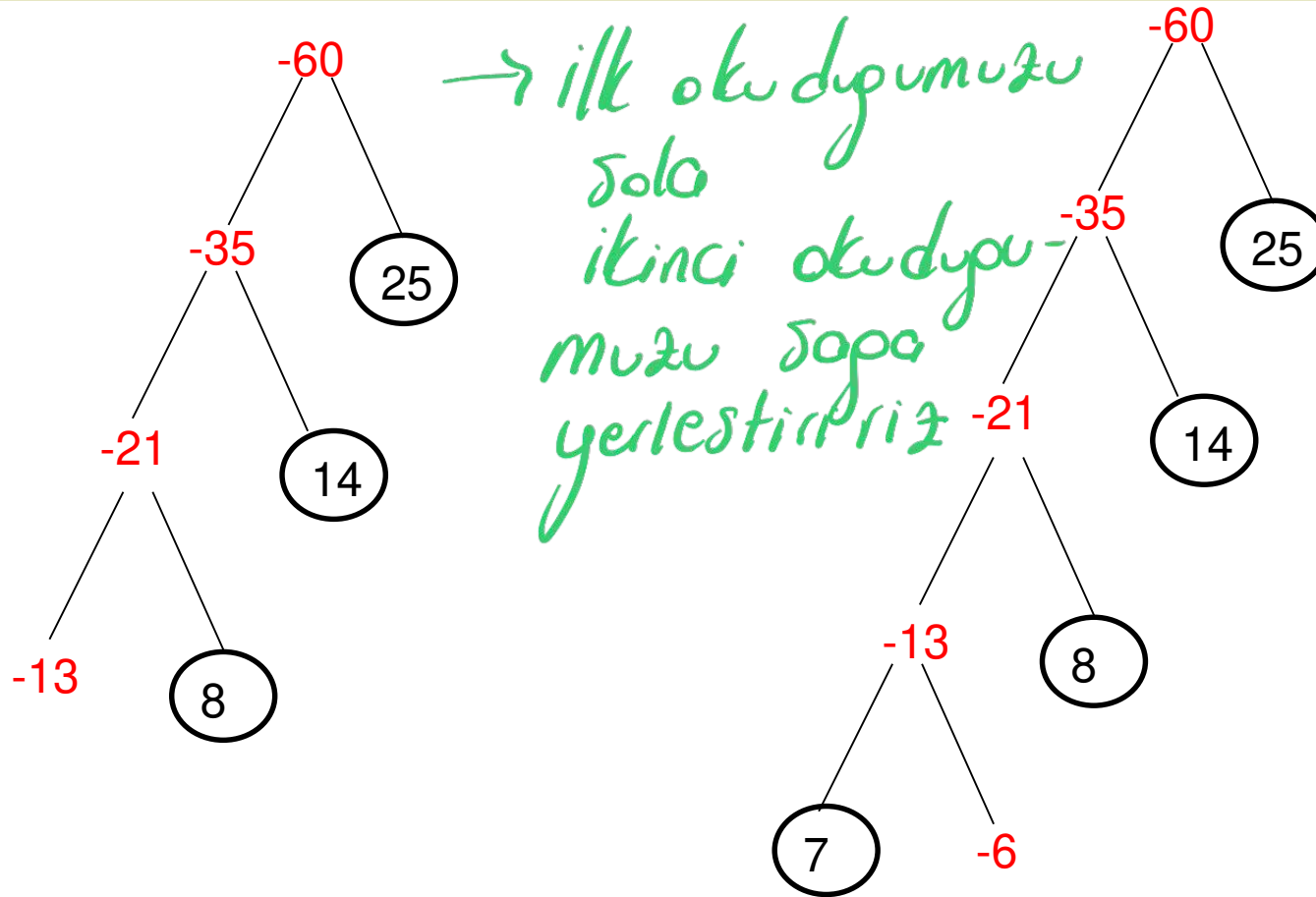
-35 ise bir ara düğümdür. Ağaç bu düğüm üzerinden oluşturulmaya devam edilir.



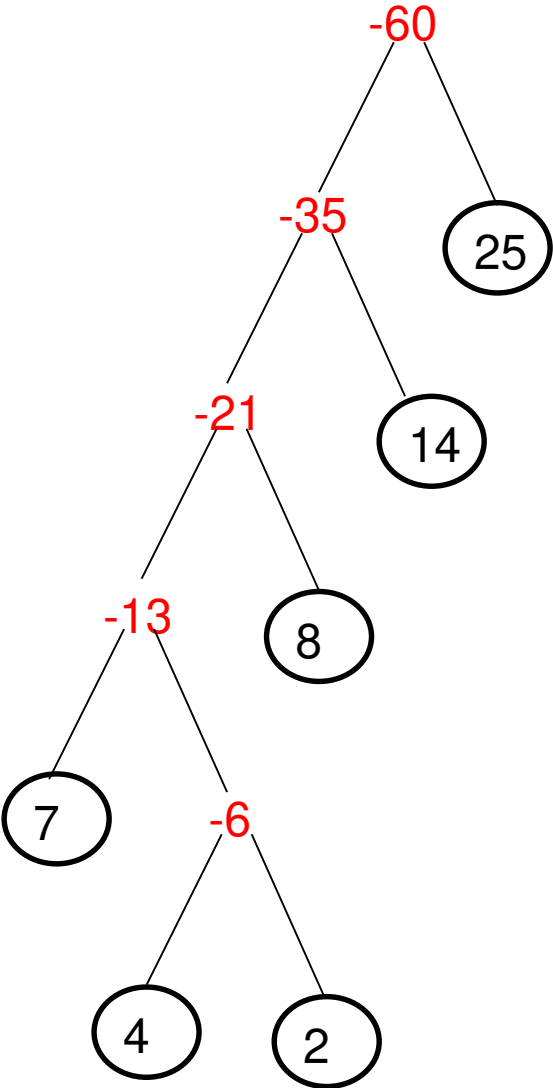
Böyle bir durumda
sola ya da sağa
gelebilir

fakat sola ya-
rarsan benzer
bir olayda
tekrar sola
yaz

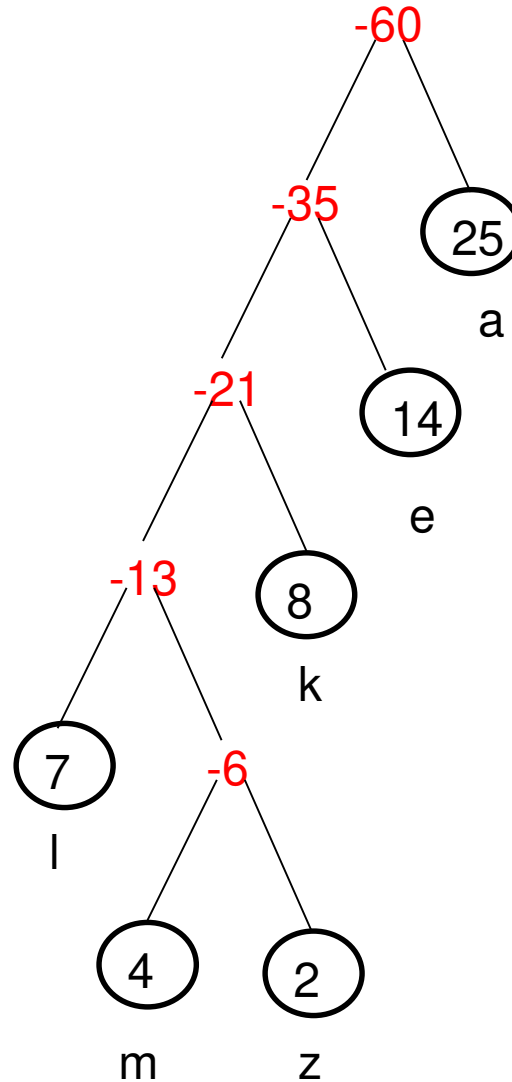
1 1 -2 2 -2 5 7 15



Huffman Kodlama Ağacının son durumu

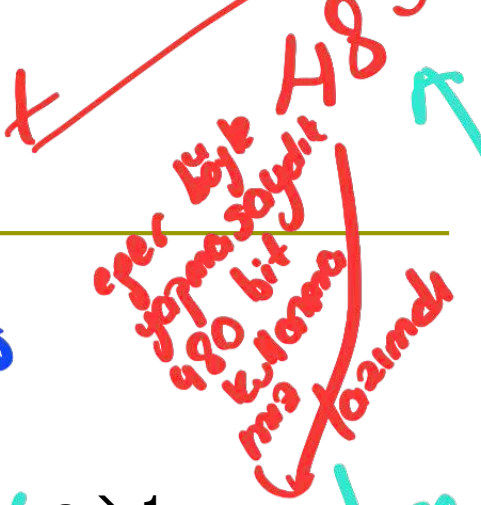


Huffman Kodlama Ağacının yapraklarına sembolleri yerleştirelim.



$25 \times 8 \rightarrow 200$
 $14 \times 8 \rightarrow 112$
 $8 \times 8 \rightarrow 64$
 $7 \times 8 \rightarrow 56$
 $4 \times 8 \rightarrow 32$
 $2 \times 8 \rightarrow 16$

~~1 ve 0 değerlerini~~



neden görüştür
Bir sembolün kodu
bir sembolün
simgesini

Bu asagıdan yukarı okunmada olur

~~Binary tree~~

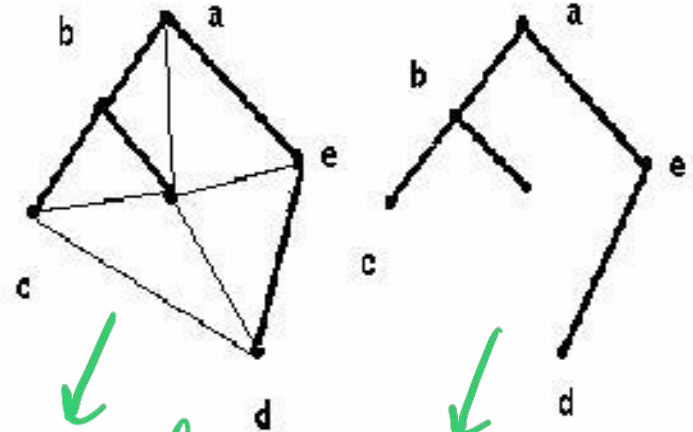
Spann

→ Bu yapıdan farklı türde treeler oluşturabiliriz

***T*,**

ve

- **G** grafının bütün düğümlerini içerir



Begrüßung

Bu prof
aynı zamanda
bir tree

→ Treeler profloin özel bir durumudur.

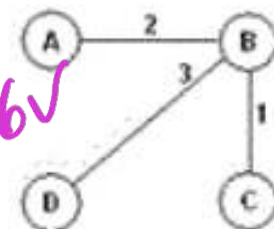
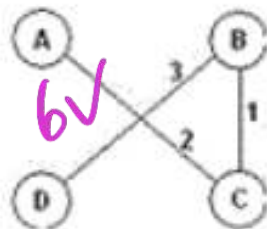
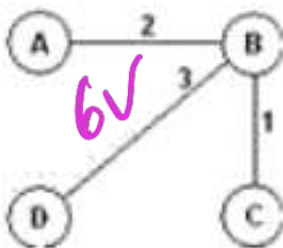
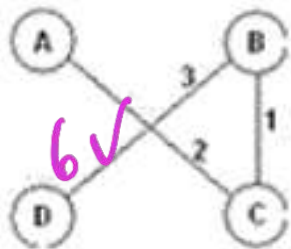
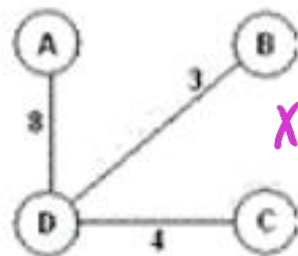
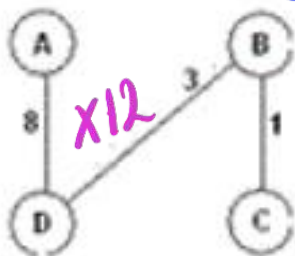
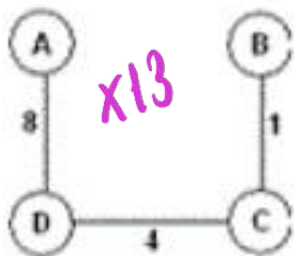
→ profloer de treelerden

C profinin
her bir düğümünü

0 farklı olarak
topoloji görüntüleri
olabilir

Farklı Spanning Trees

İçericek şekilde
Gizilen her bir
tree'ye ben
spanning tree
diyorum



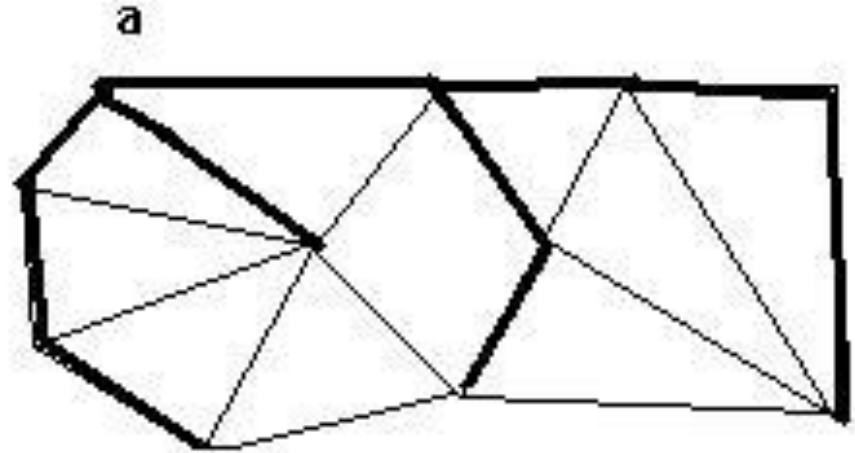
✓
Minimum
Spanning

Birde bunun
minimum spanning
var bunda ise
tree'yi gizlerken
kısıt olduğun
anlam

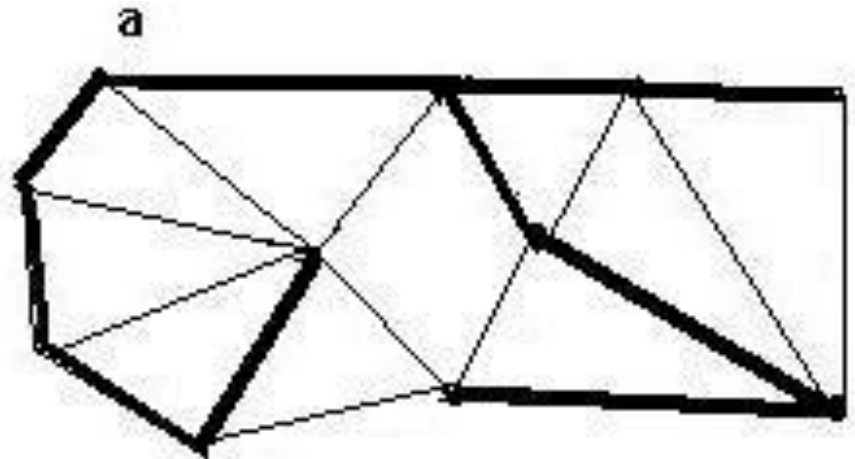
↓
Spanning
tree'leri
bu şekillerde
gizebilirsin

Spanning tree oluşturma yöntemleri

- Breadth-first search method

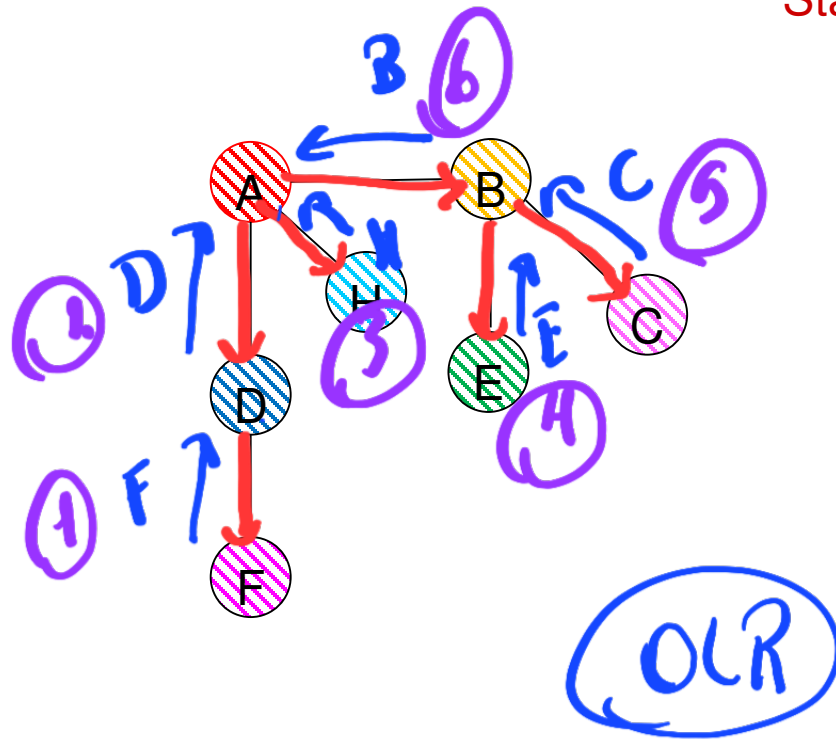


- Depth-first search method (backtracking)



DFS (Depth First Search – Derin Öncelikli Arama)

- Ziyareti kök düğümden başlatır ve gidebildiği en uzak düğüme kadar gider.
- Tüm düğümler ziyaret edilene kadar işlem tekrar edilir.
- Stack-yığın veri yapısını kullanır.

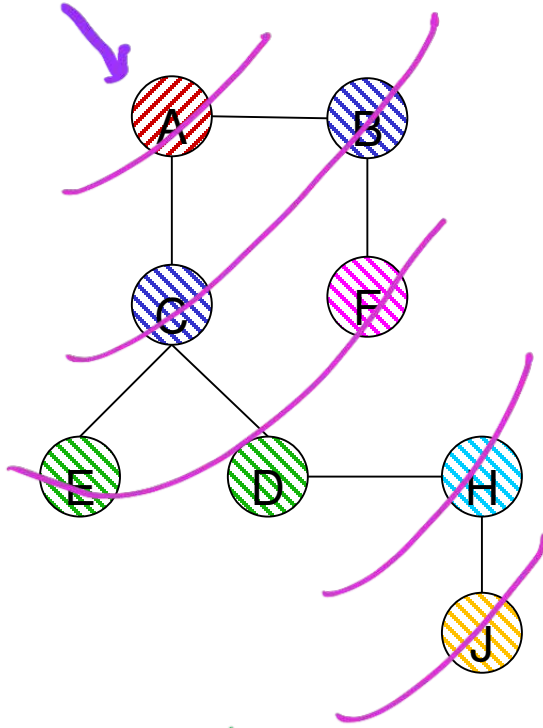


Output : A-D-F-H-B-E-C

Adım	İşlem	Stack	Çıkan	Output
1	Push(A)	A	-	A
2	Push(D)	D-A	-	D
3	Push(F)	F-D-A	-	F
4	Pop()	D-A	F	-
5	Pop()	A	D	-
6	Push(H)	H-A	-	H
7	Pop()	A	H	-
8	Push(B)	B-A	-	B
9	Push(E)	E-B-A	-	E
10	Pop()	B-A	E	-
11	Push(C)	C-B-A	-	C
12	Pop()	B-A	C	-
13	Pop()	A	B	-
14	Pop()	-	A	-

BFS (Breadth First Search – Geniş Öncelikli Arama)

- Dğümleri seviye bazında ziyaret eder.
- Kök düğümden başlar ve düğümleri düzey düzey ziyaret eder.
- Bir sonraki seviyeye geçmeden önce bulunduğu seviyedeki tüm düğümleri ziyaret eder.
- Queue-kuyruk veri yapısını kullanır.



Depth First
Start A
A
C
D
H
J
E
B
F

Adım	Eklenen	Çıkan	Kuyruk	Output
1	A	-	A	A
2	B-C	A	B-C	B-C
3	F	B	C-F	F
4	E-D	C	F-E-D	E-D
5	-	F	E-D	-
6	-	E	D	-
7	H	D	H	H
8	J	H	J	J
9	-	J	-	-

→ Kuyruksda
ilk giren ilk
çıkıyor

Output : A-B-C-F-E-D-H-J

Burda düğümler



Seviye bazında
ziyaret ediliyor

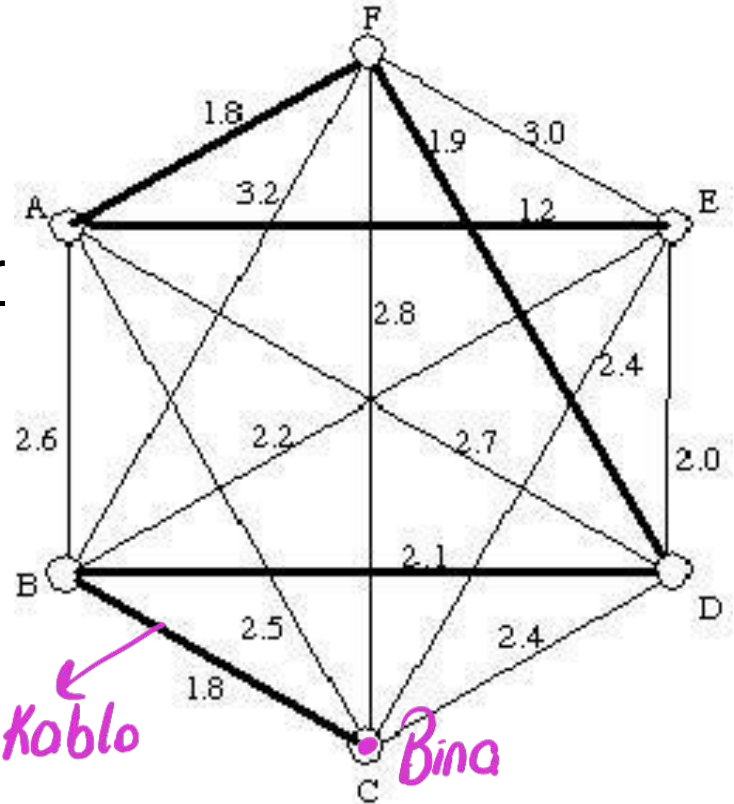
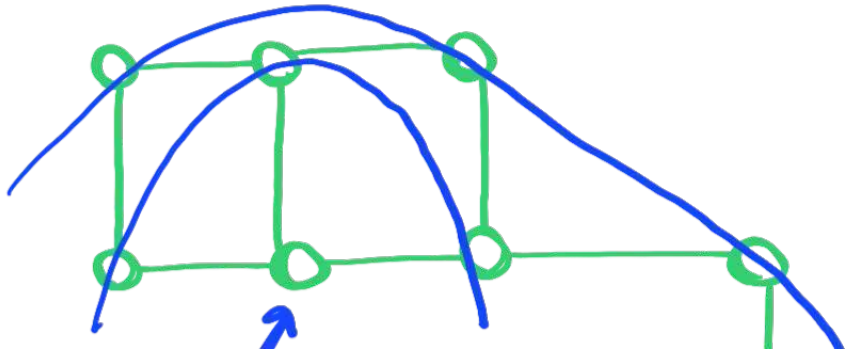
Minimal spanning trees

Kenarlar da açık değeri
varsa minimal spanning
kullanmak lazım

Verilen ağırlıklı bir **G** grafının

Minimal spanning tree'si

- **G** grafının spanning tree 'si
olup (tüm düğümlerden geçilmiş)
- Ağırlıklar toplamıda minimumdur



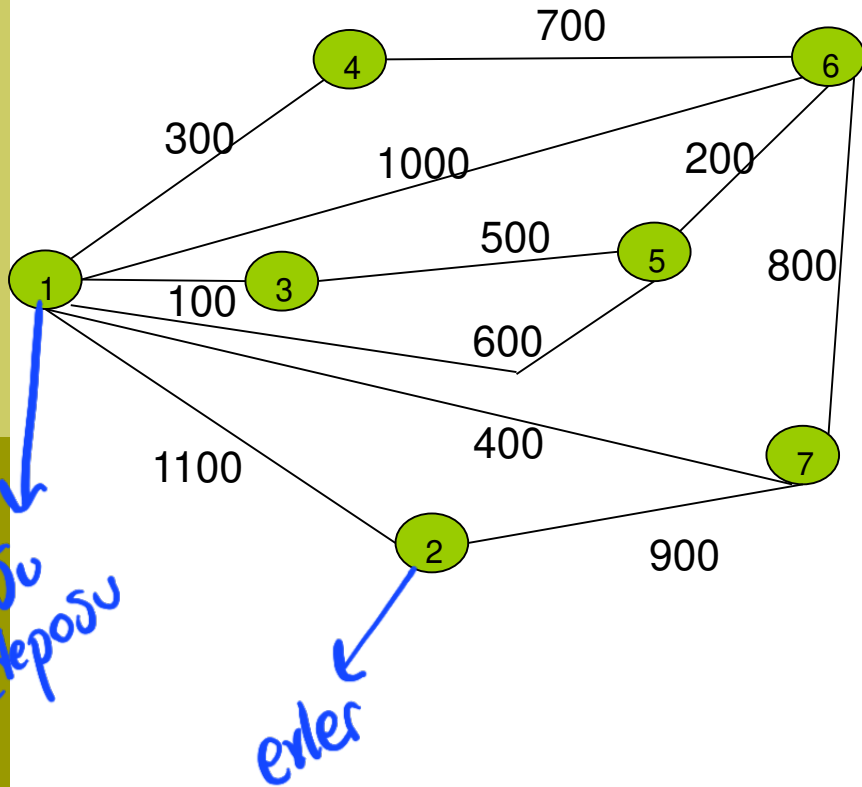
Breadth

→ Burada...

Örnek : PRIM Algoritması

→ Breadth
first
search

minimum küt-
lüğü nasıl
döşeriz
mesela



	1	2	3	4	5	6	7
1	0	1100	100	300	600	1000	400
2	1100	0	M	M	M	M	900
3	100	M	0	M	500	M	M
4	300	M	M	0	M	700	M
5	600	M	500	M	0	200	M
6	1000	M	M	700	200	0	800
7	400	900	M	M	M	800	0

Bu erleri
Bu depoya
öyle bir bağlayalım ki
minimum
olsun

	1	2	3	4	5	6	7
1	0	1100	100	300	600	1000	400
2	1100	0	M	M	M	M	900
3	100	M	0	M	500	M	M
4	300	M	M	0	M	700	M
5	600	M	500	M	0	200	M
6	1000	M	M	700	200	0	800
7	400	900	M	M	M	800	0

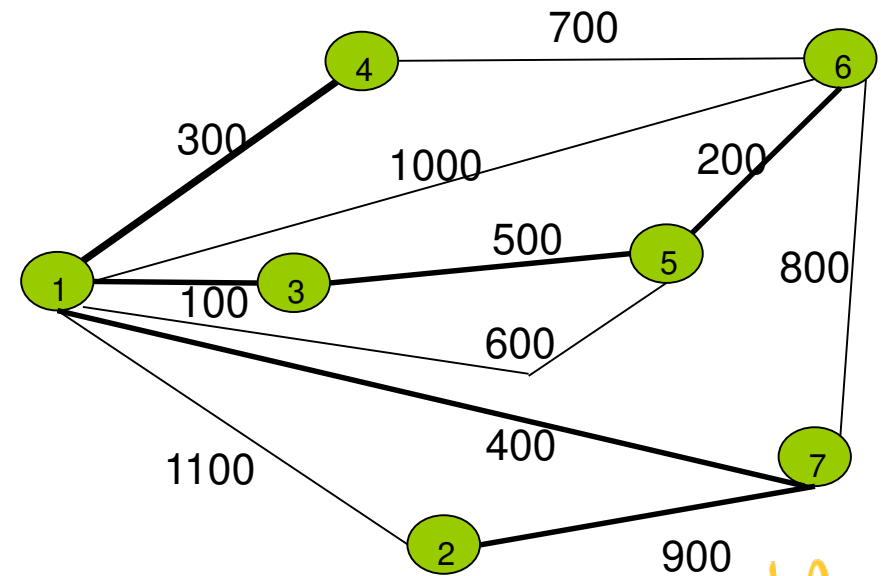
	1	2	3	4	5	6	7
1	0	1100	100	300	600	1000	400
2	1100	0	M	M	M	M	900
3	100	M	0	M	500	M	M
4	300	M	M	0	M	700	M
5	600	M	500	M	0	200	M
6	1000	M	M	700	200	0	800
7	400	900	M	M	M	800	0

	1	2	3	4	5	6	7
1	0	1100	100	300	600	1000	400
2	1100	0	M	M	M	M	900
3	100	M	0	M	500	M	M
4	300	M	M	0	M	700	M
5	600	M	500	M	0	200	M
6	1000	M	M	700	200	0	800
7	400	900	M	M	M	800	0

	1	2	3	4	5	6	7
1	0	1100	100	300	600	1000	400
2	1100	0	M	M	M	M	900
3	100	M	0	M	500	M	M
4	300	M	M	0	M	700	M
5	600	M	500	M	0	200	M
6	1000	M	M	700	200	0	800
7	400	900	M	M	M	800	0

	1	2	3	4	5	6	7
1	0	1100	100	300	600	1000	400
2	1100	0	M	M	M	M	900
3	100	M	0	M	500	M	M
4	300	M	M	0	M	700	M
5	600	M	500	M	0	200	M
6	1000	M	M	700	200	0	800
7	400	900	M	M	M	800	0

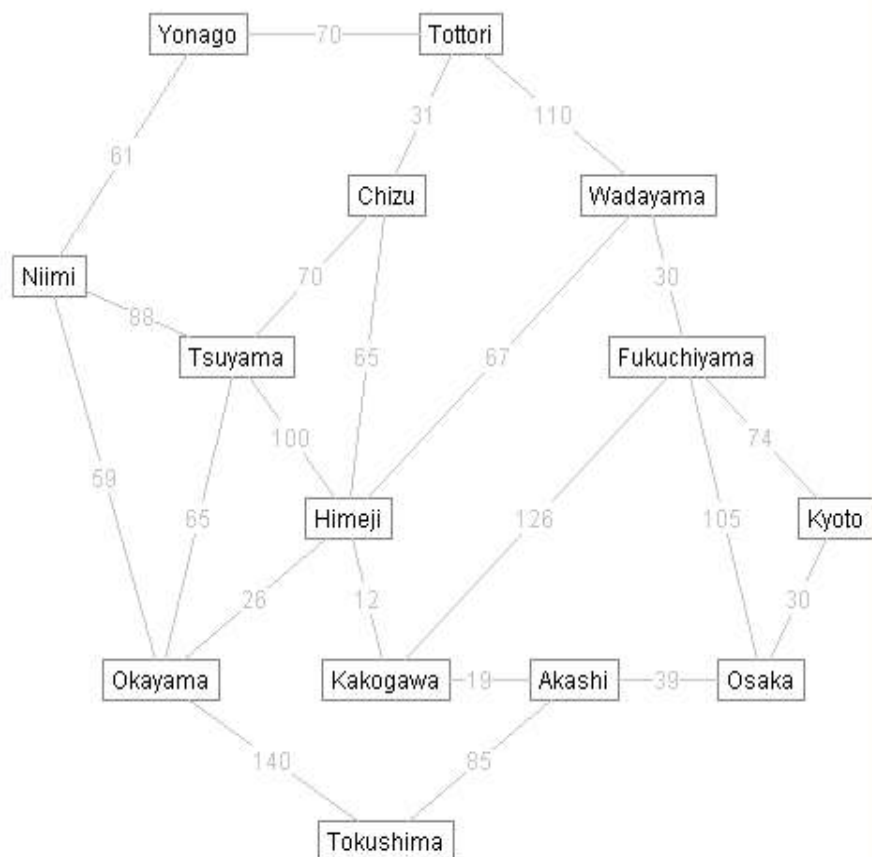
	1	2	3	4	5	6	7
1	0	1100	100	300	600	1000	400
2	1100	0	M	M	M	M	900
3	100	M	0	M	500	M	M
4	300	M	M	0	M	700	M
5	600	M	500	M	0	200	M
6	1000	M	M	700	200	0	800
7	400	900	M	M	M	800	0



1700
 900
 2600
 Kimse 2600'den
 grup küçük bir
 spanning tree

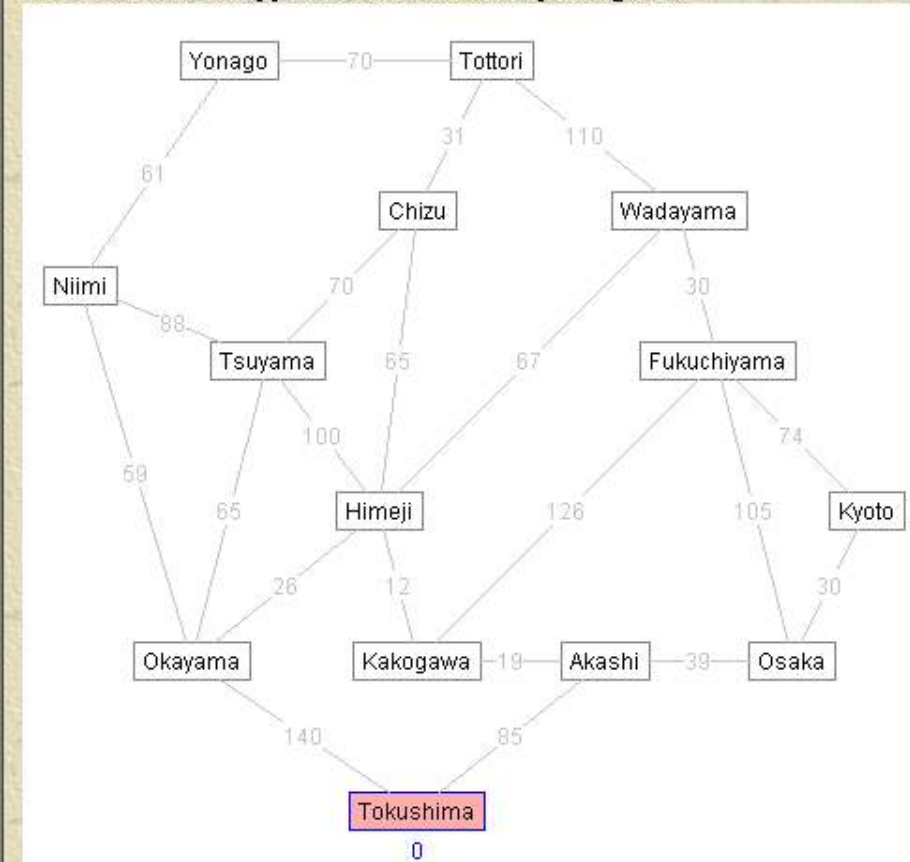
Java Applet Demo of Prim's Algorithm

Click on the below applet to find a minimum spanning tree.



Java Applet Demo of Prim's Algorithm

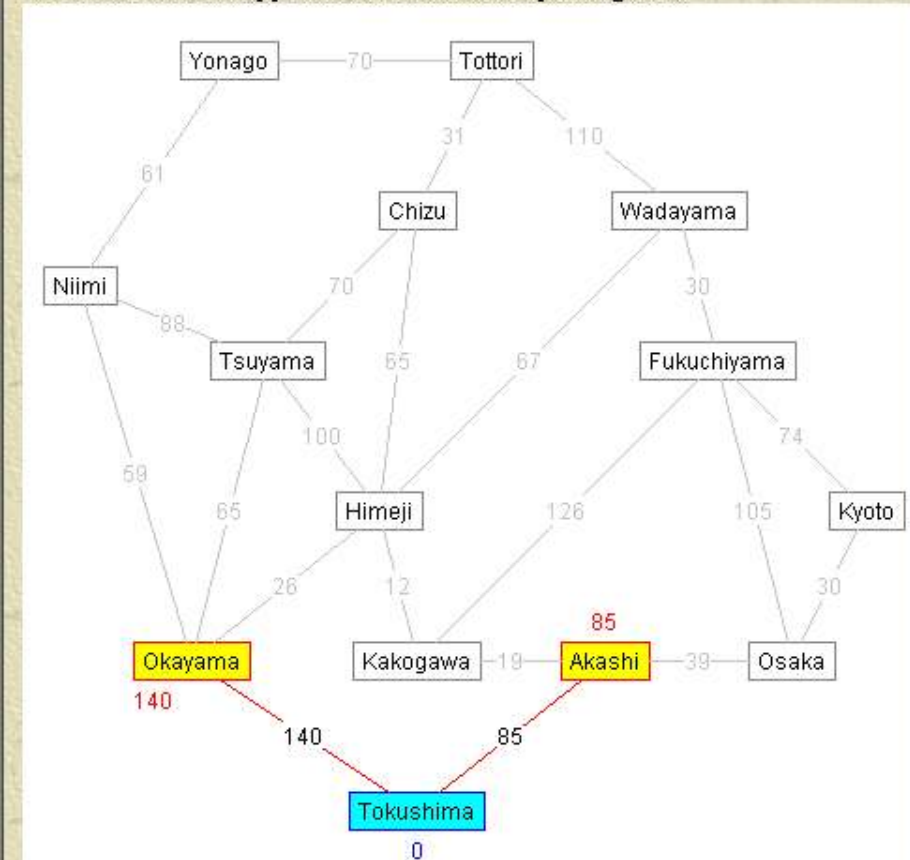
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

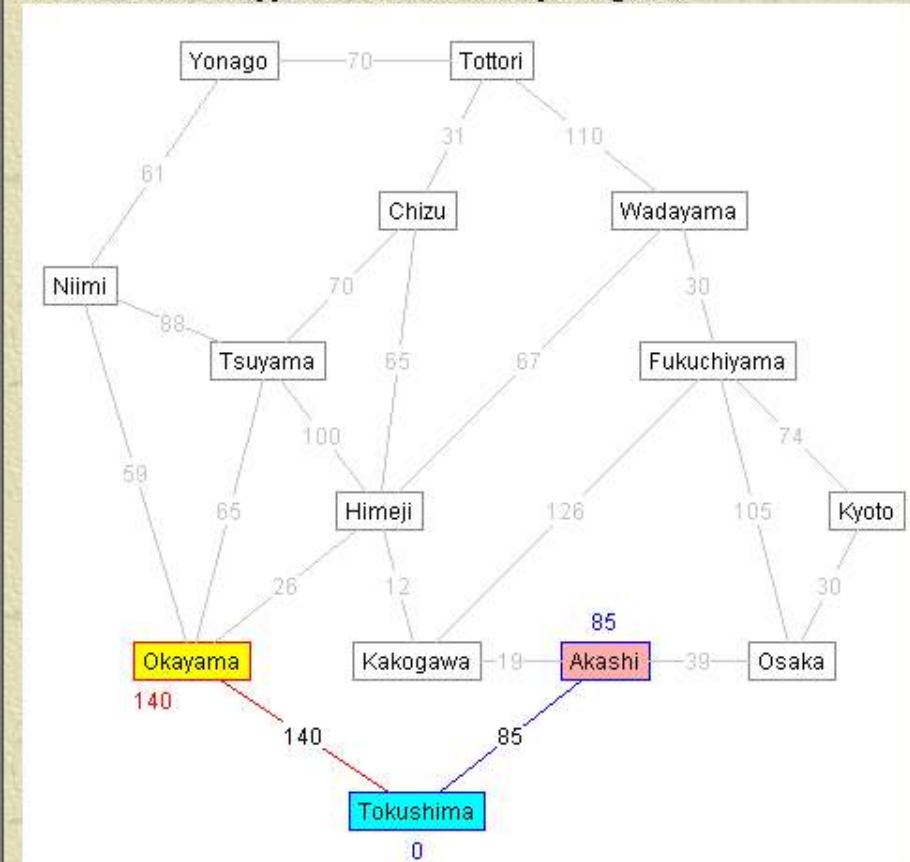
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

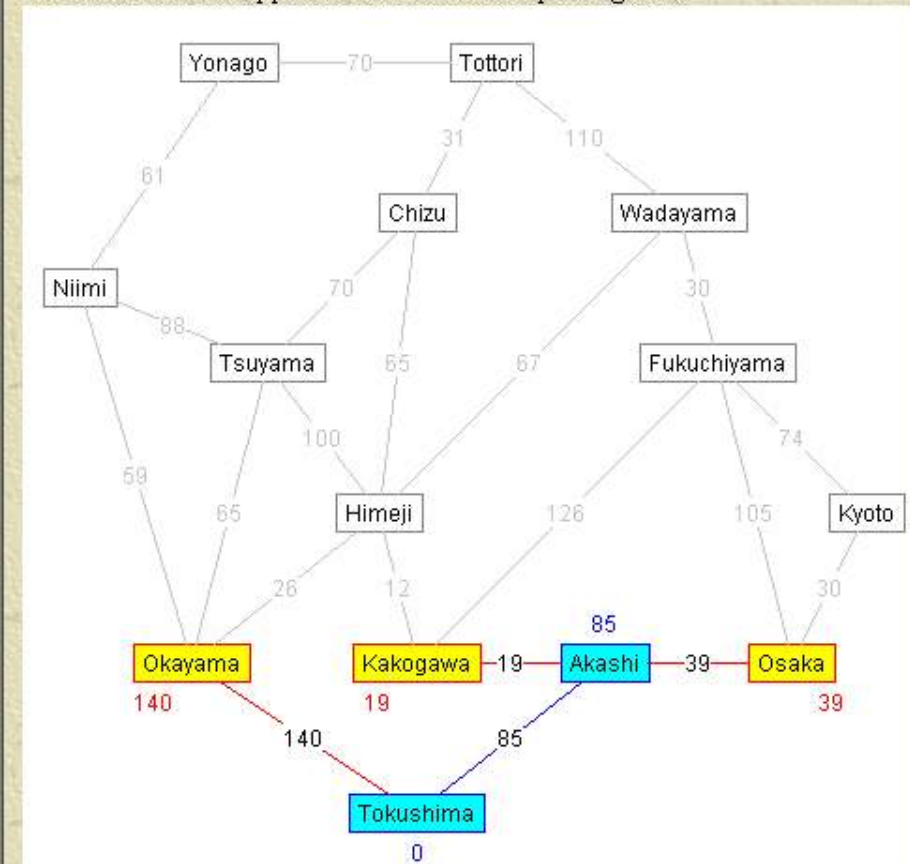
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

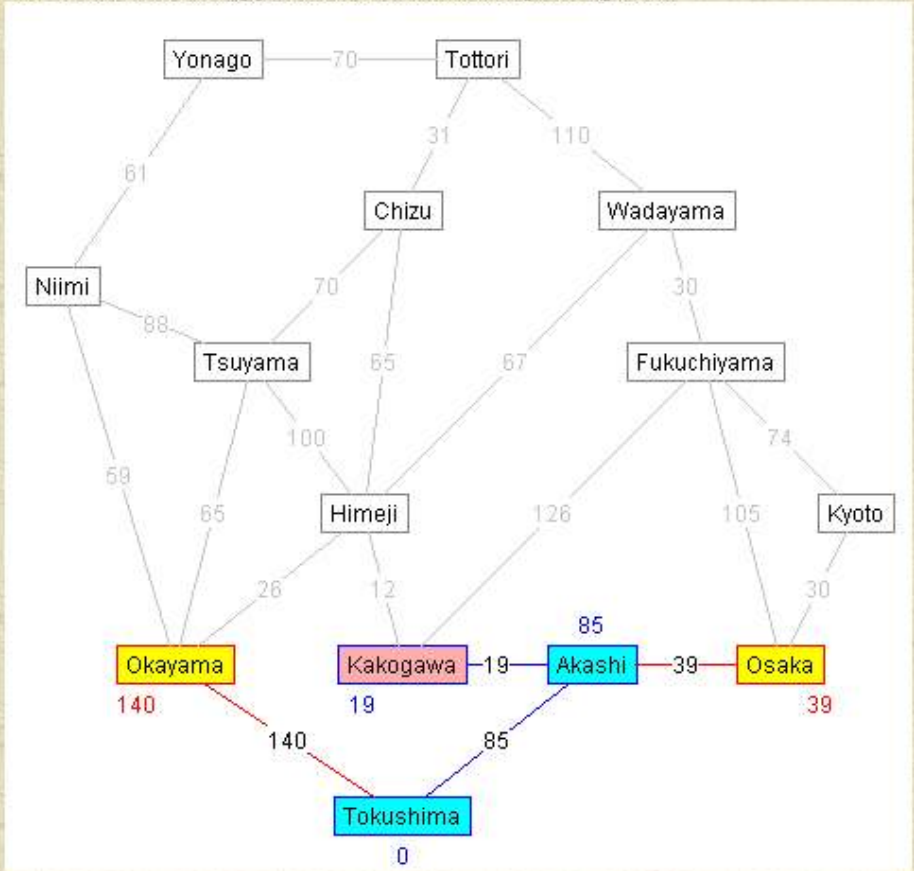
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

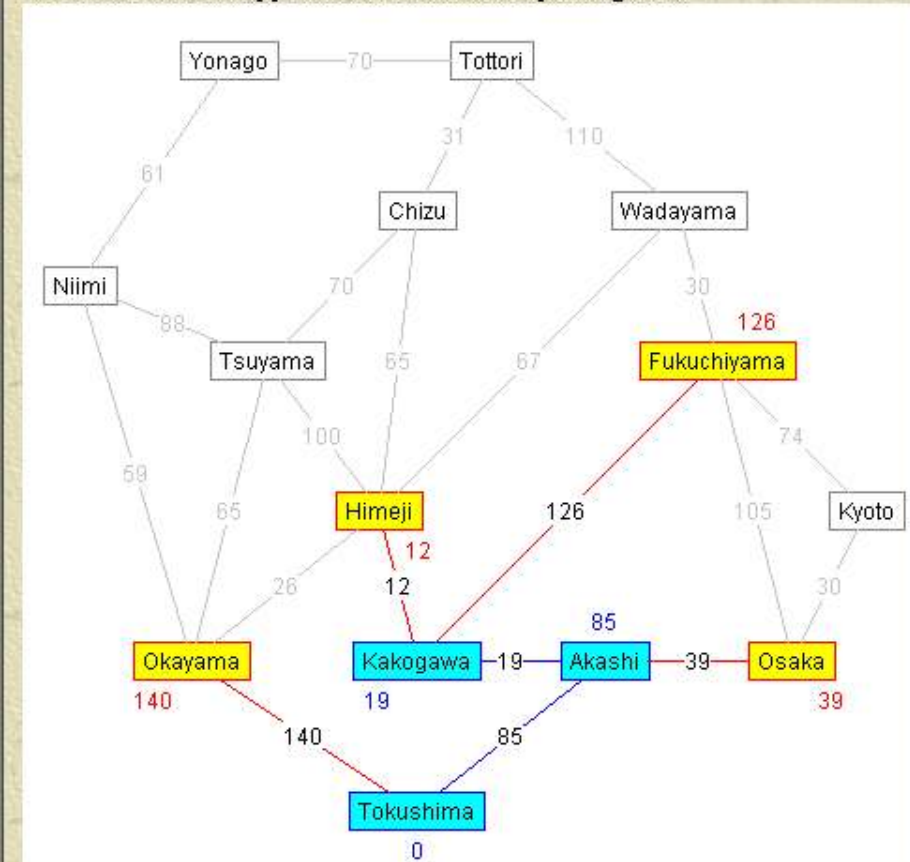
Click on the below applet to find a minimum spanning tree.



Applet written by: Kenji Ikeda

Java Applet Demo of Prim's Algorithm

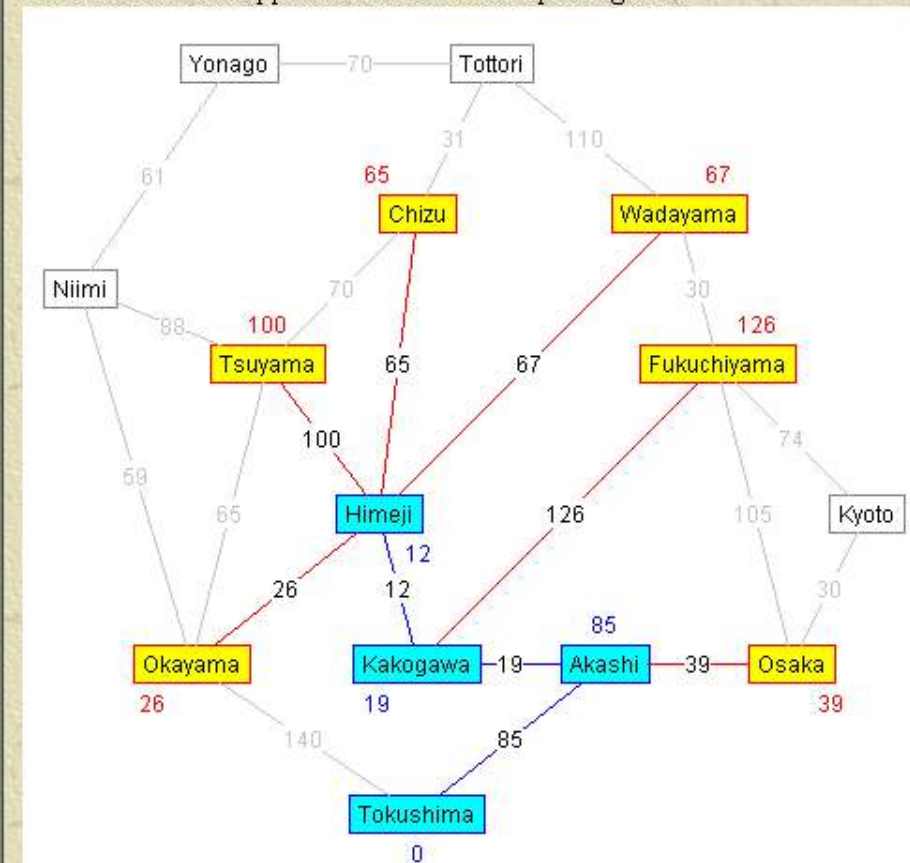
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

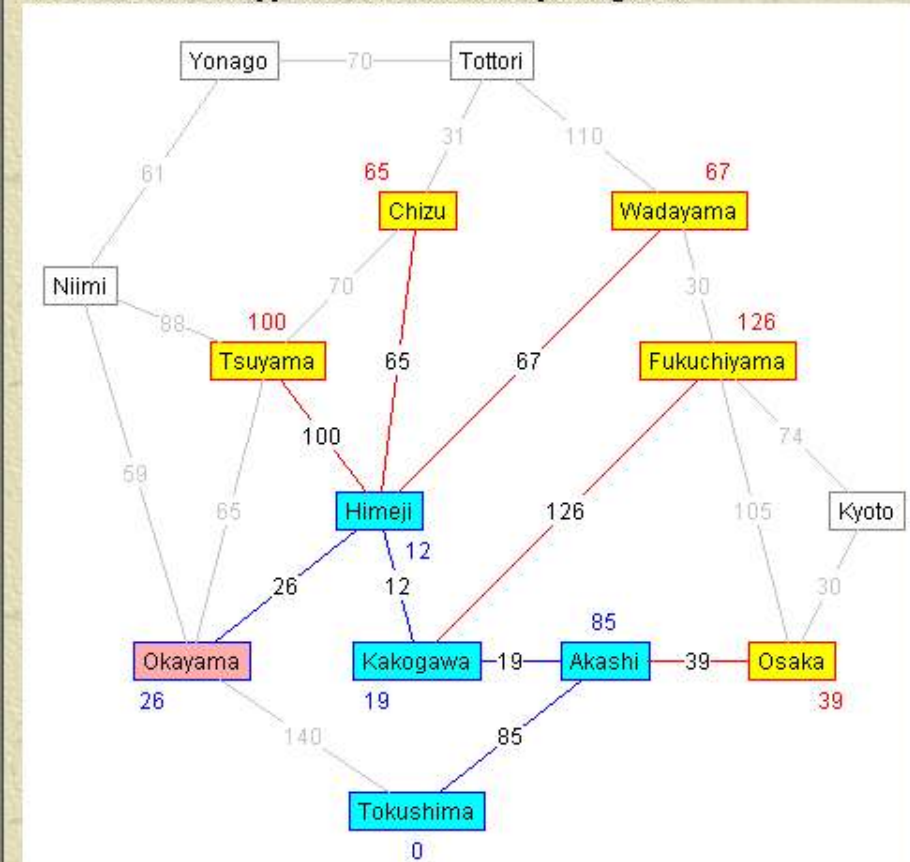
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

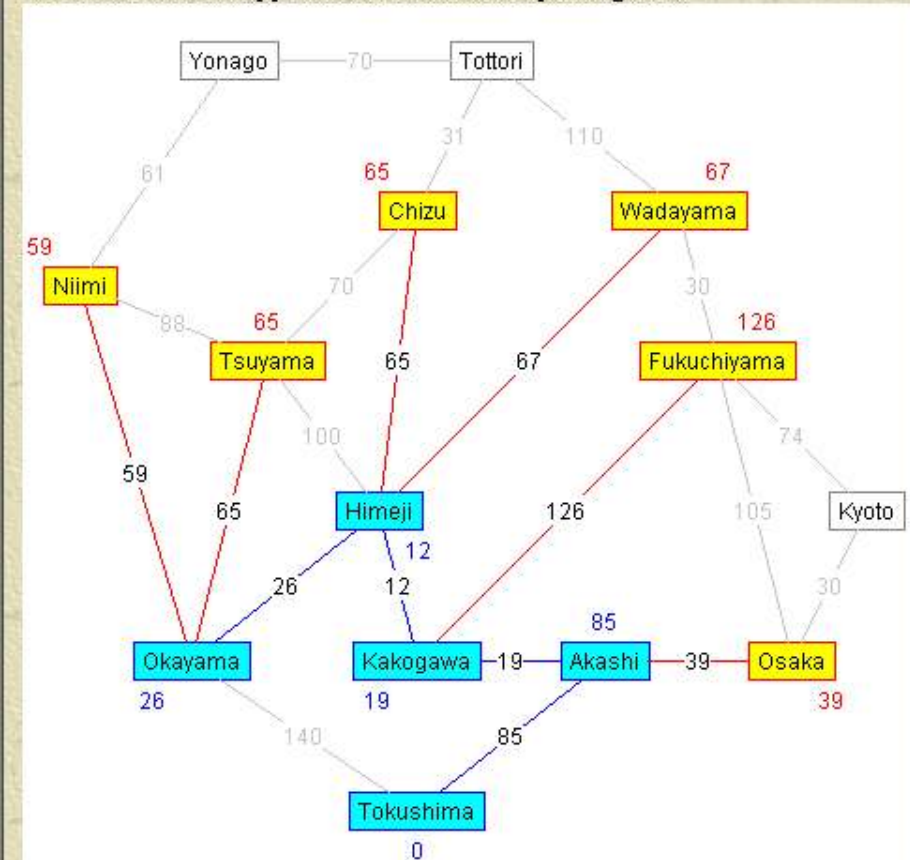
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

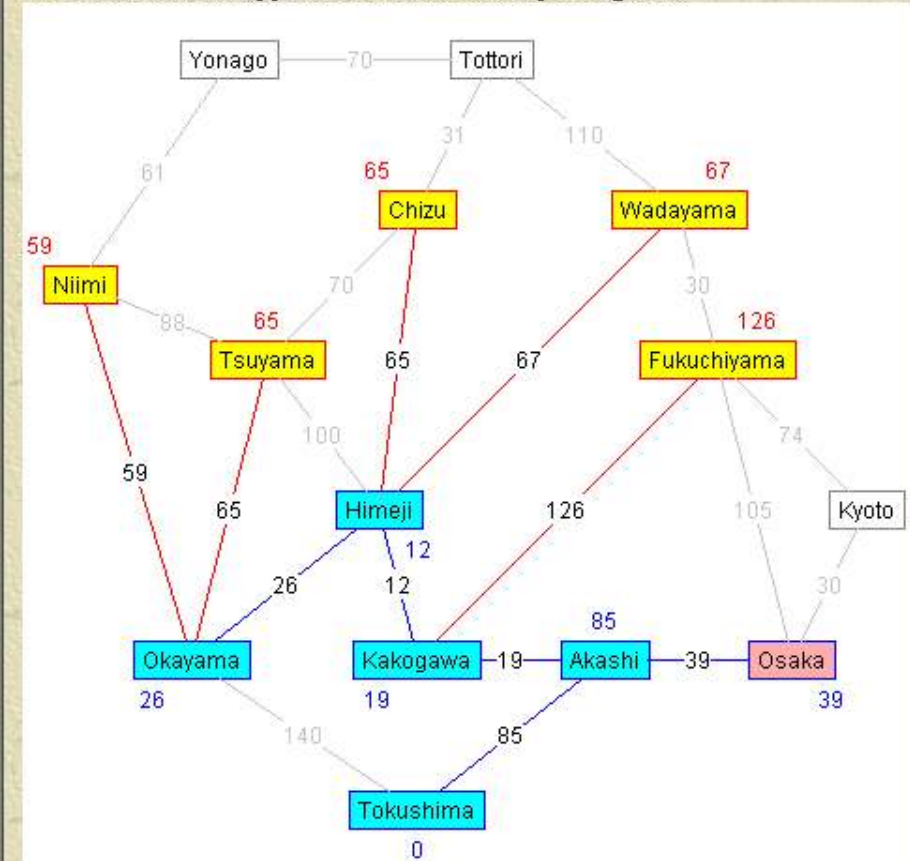
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

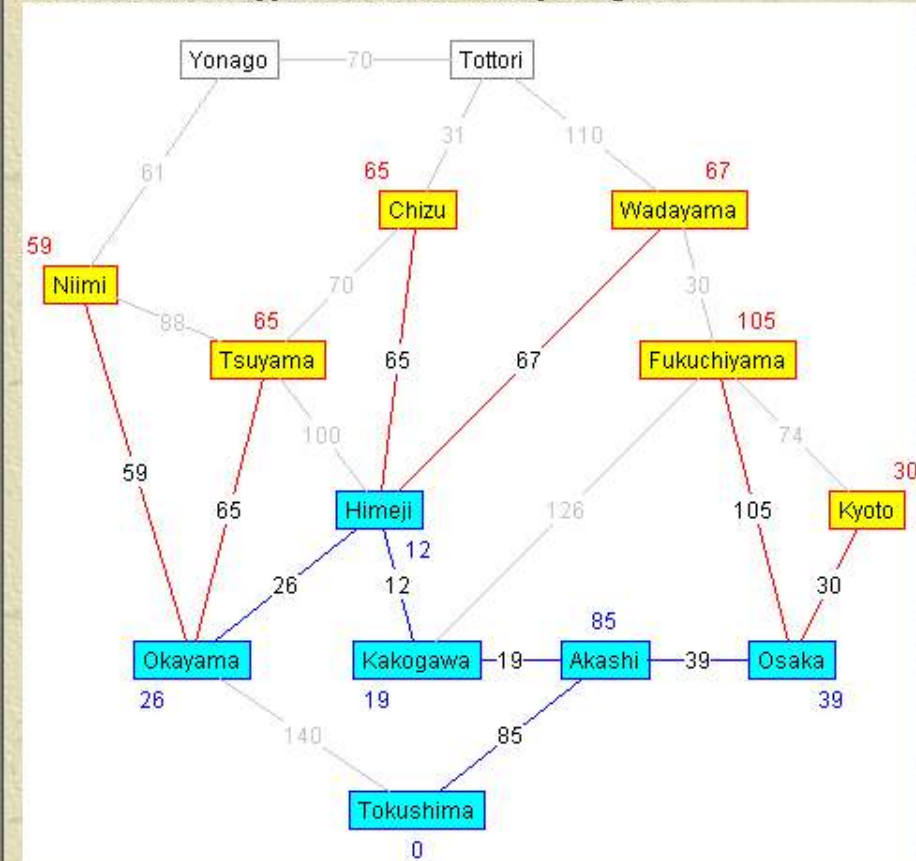
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

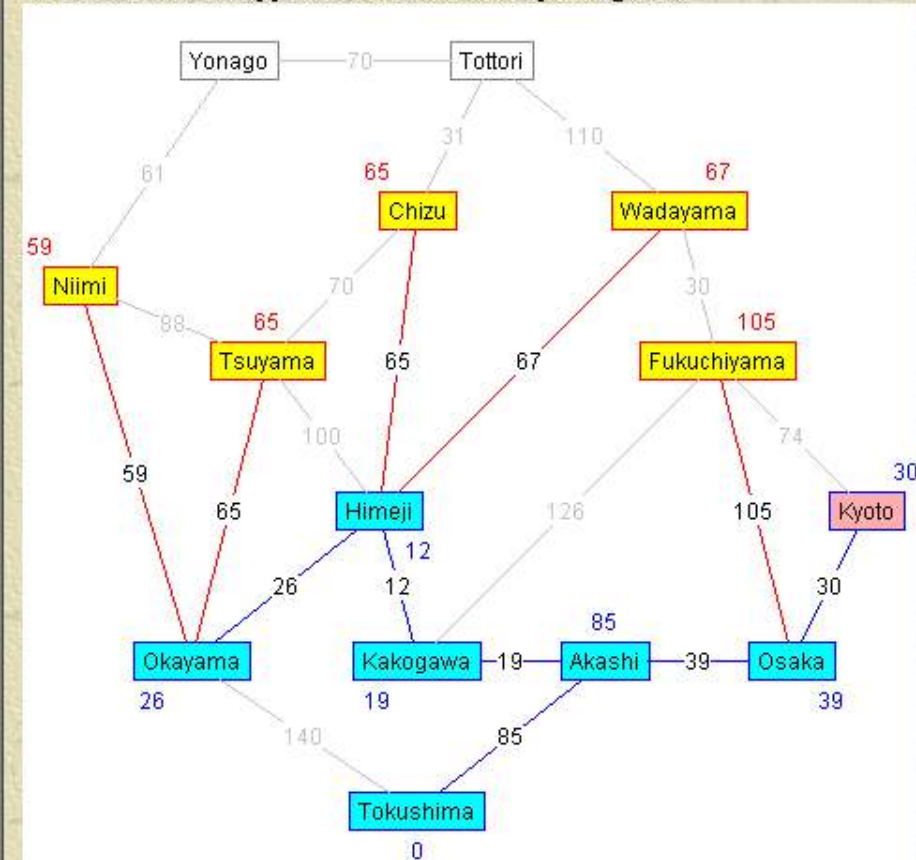
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

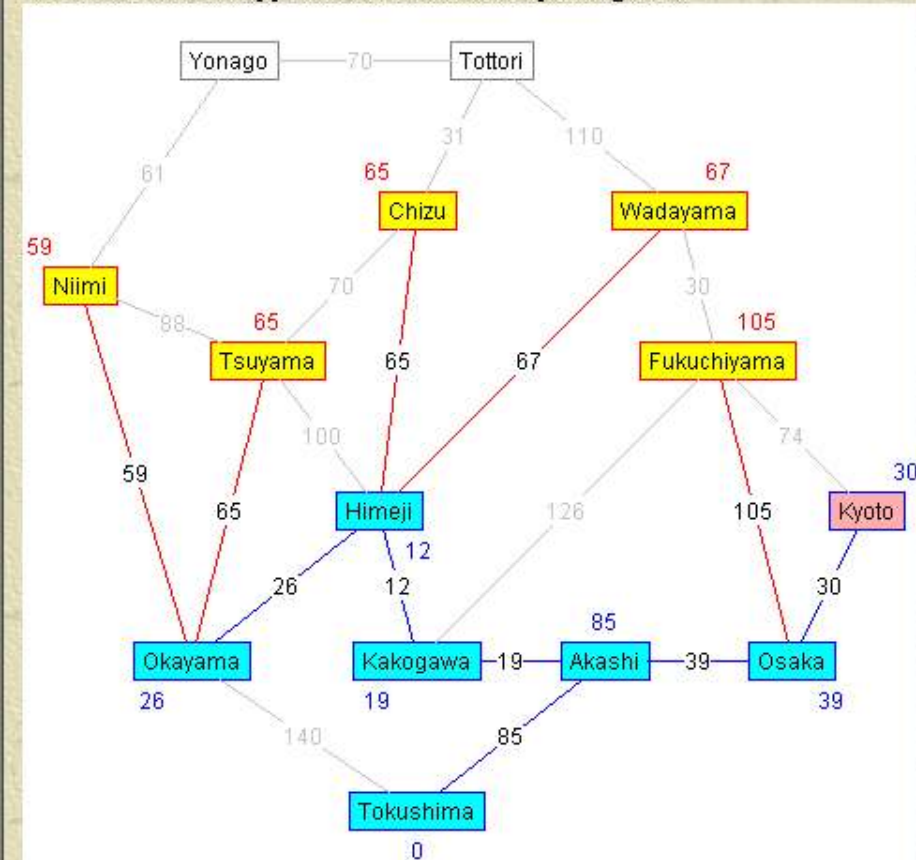
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

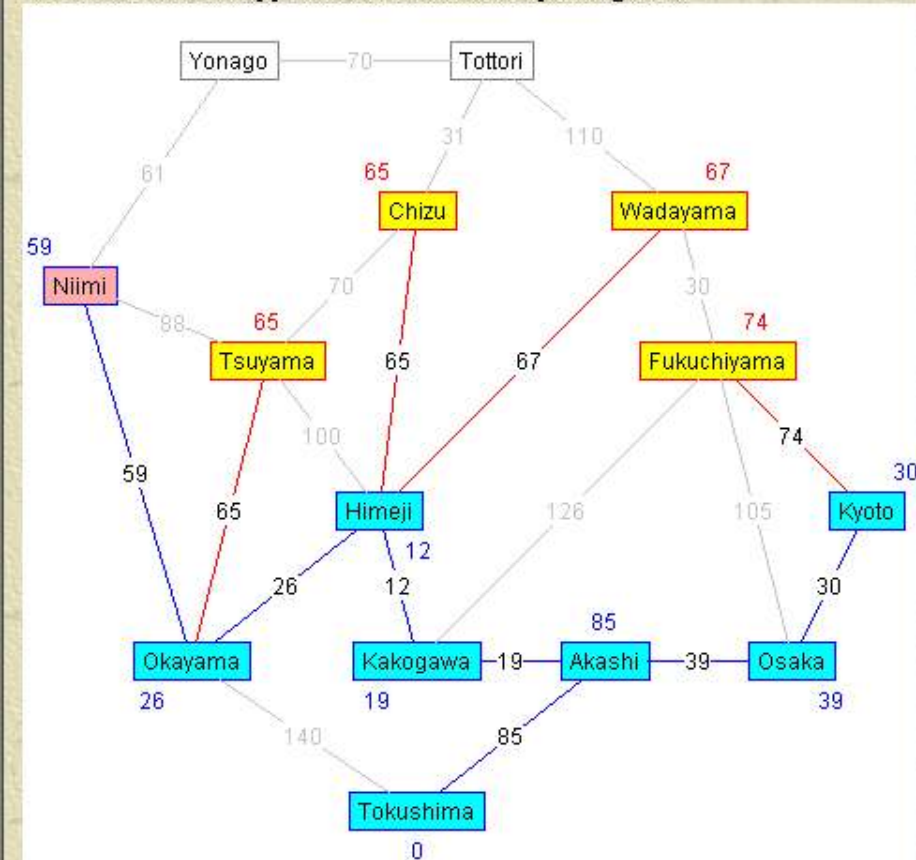
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

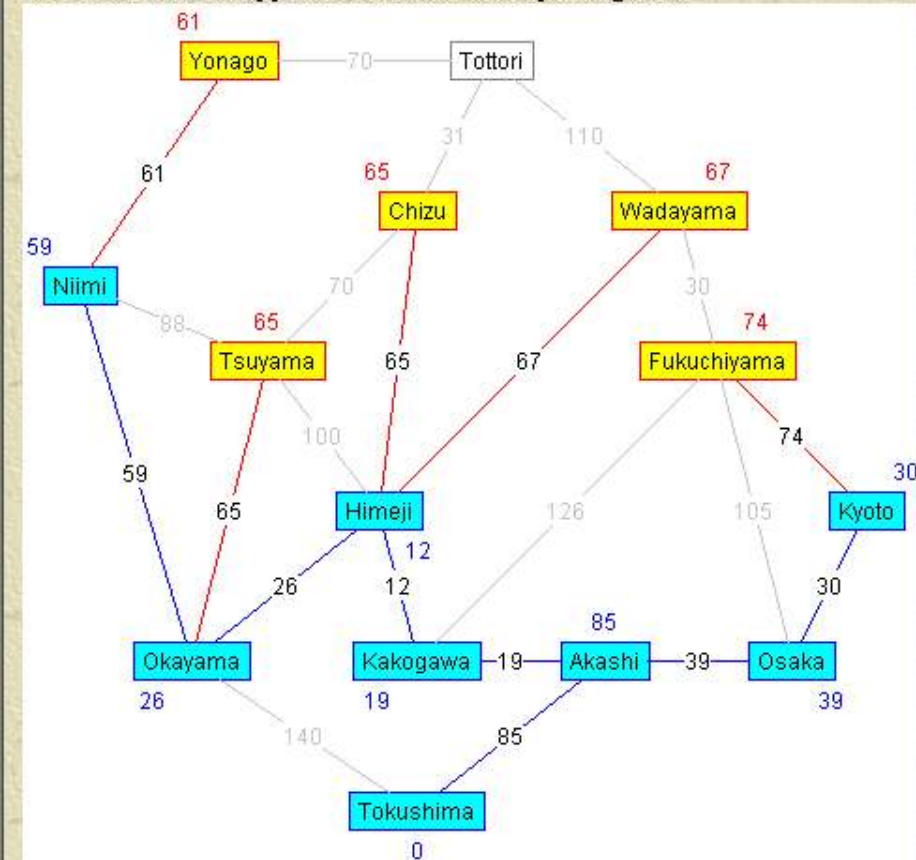
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

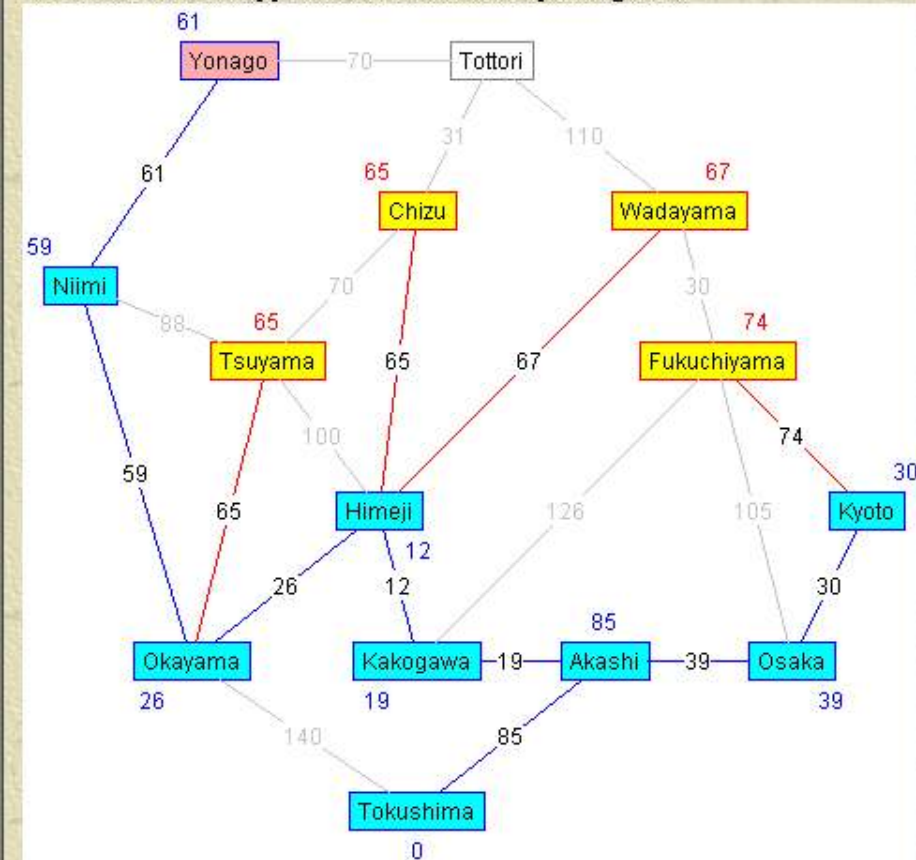
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

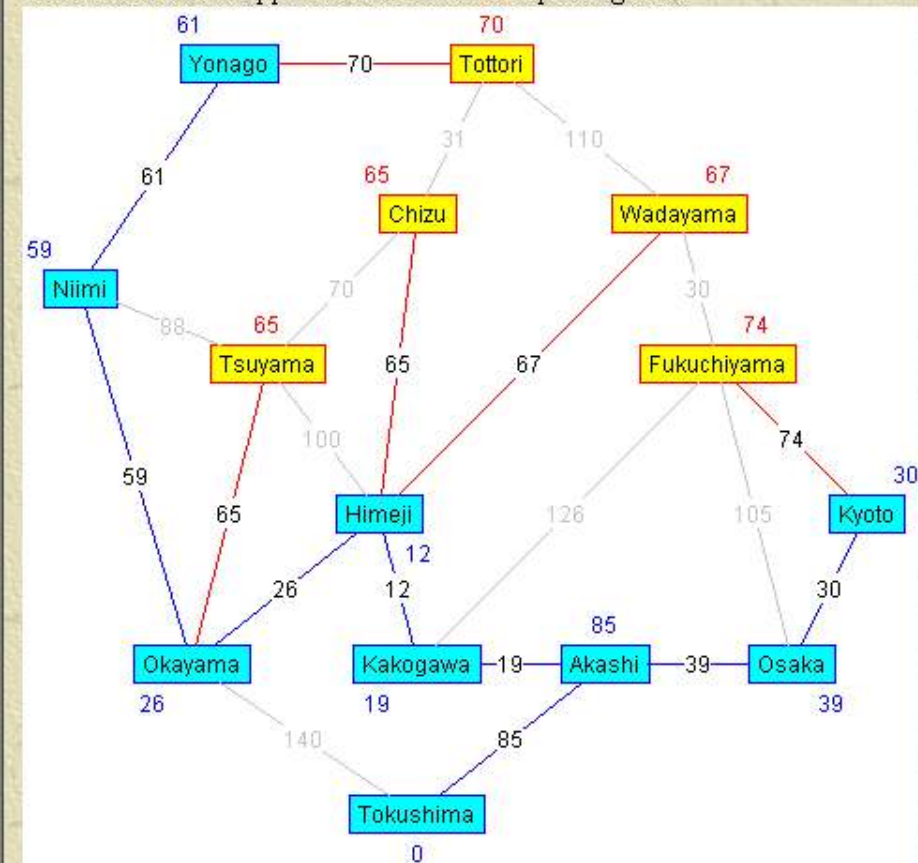
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

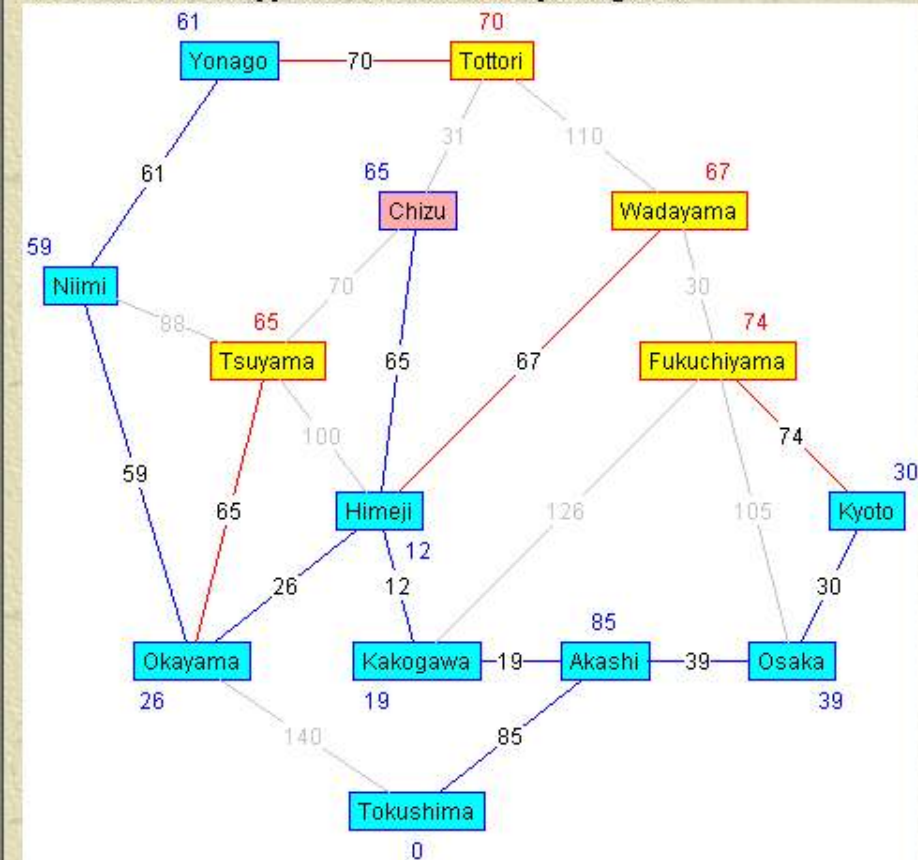
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

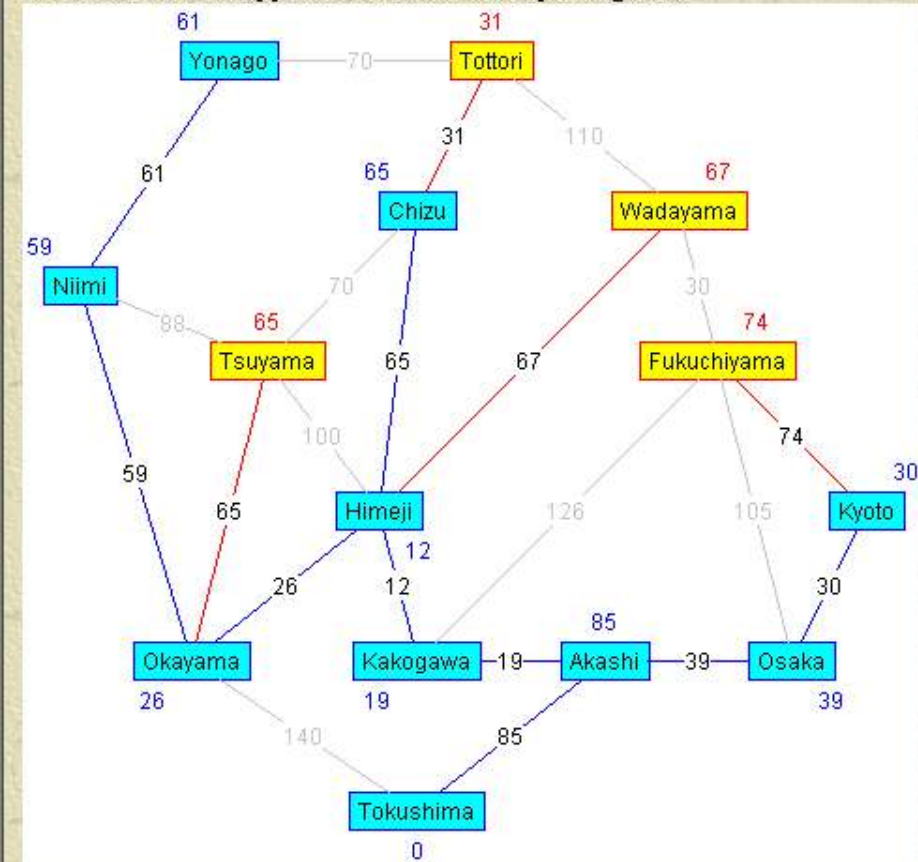
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

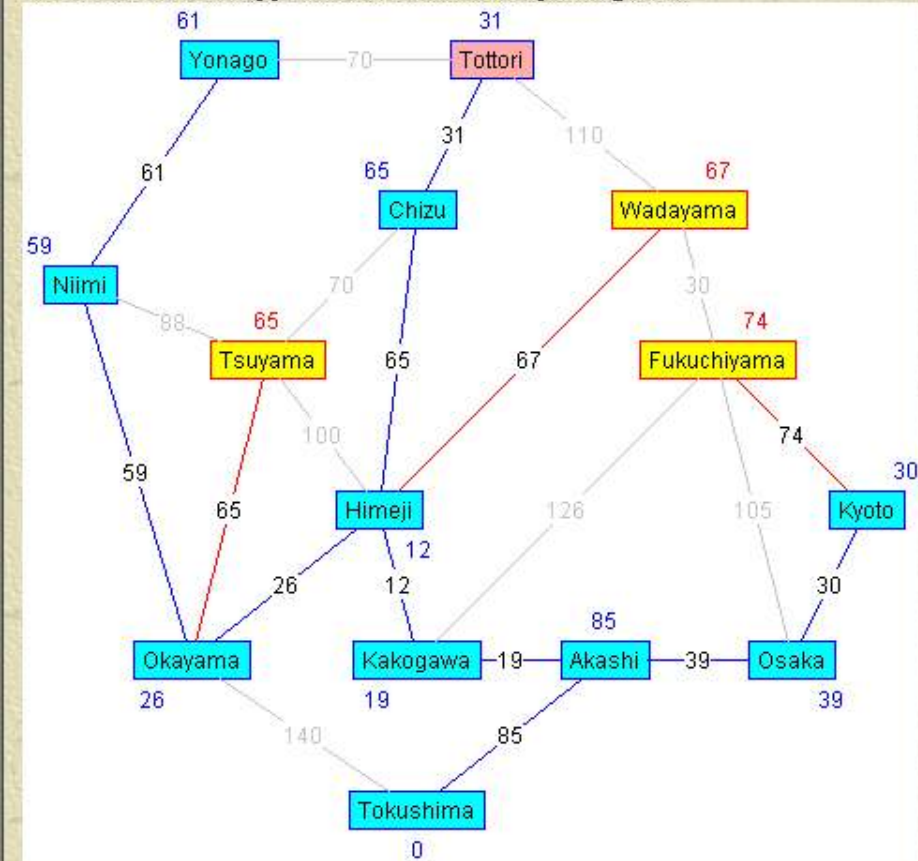
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

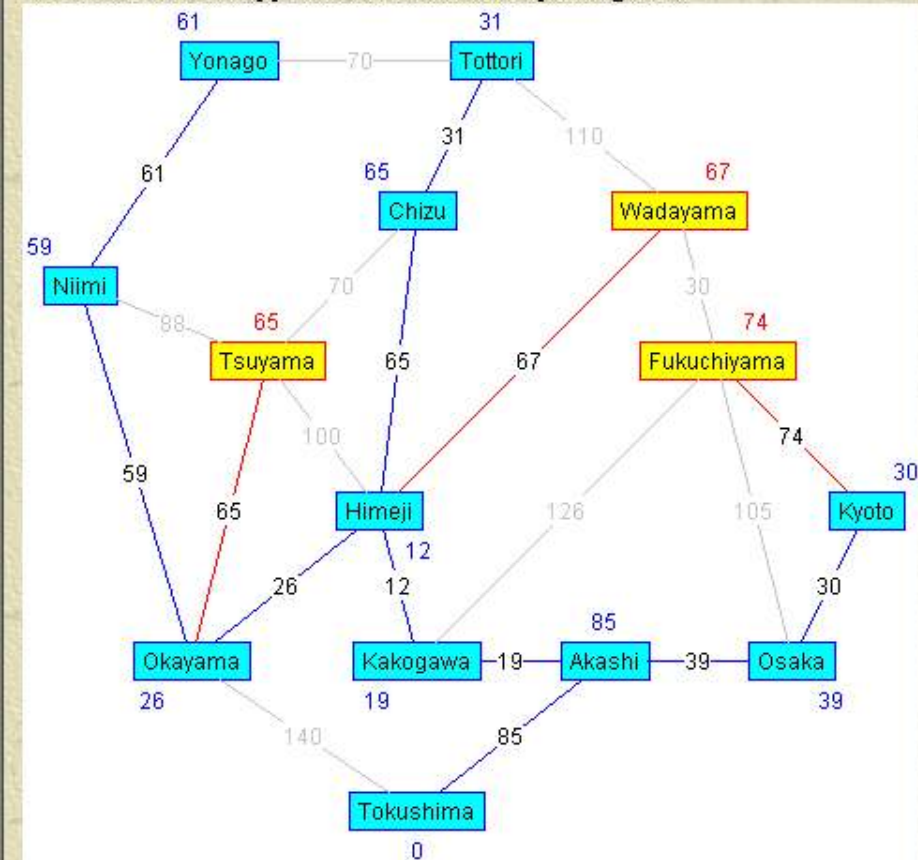
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

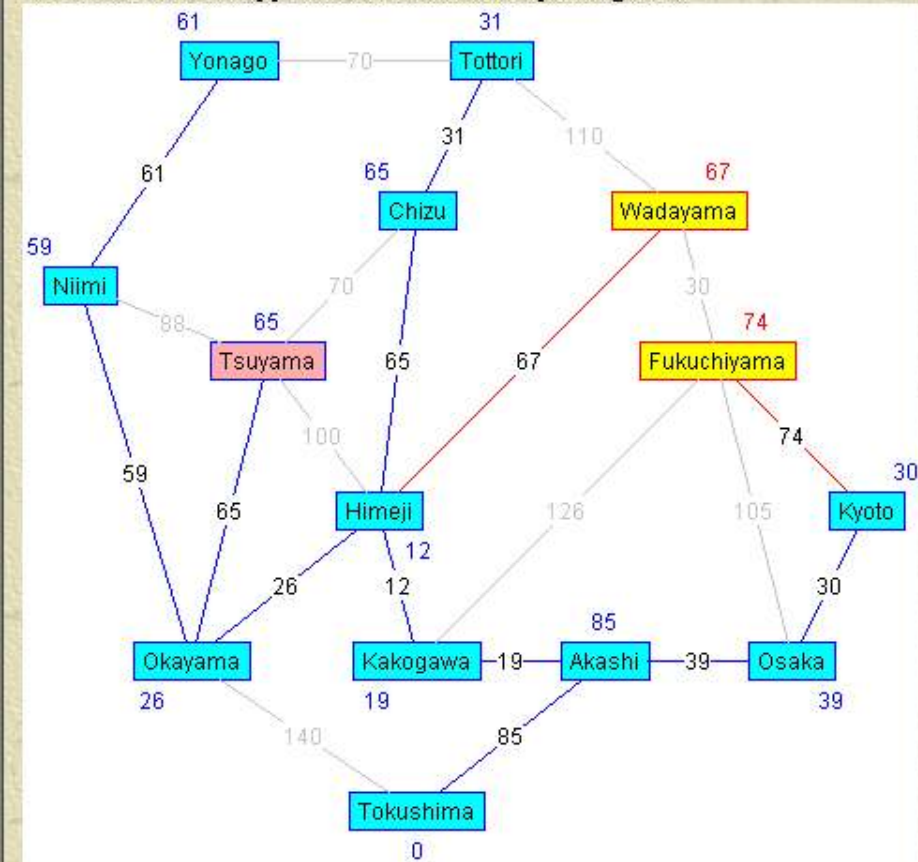
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

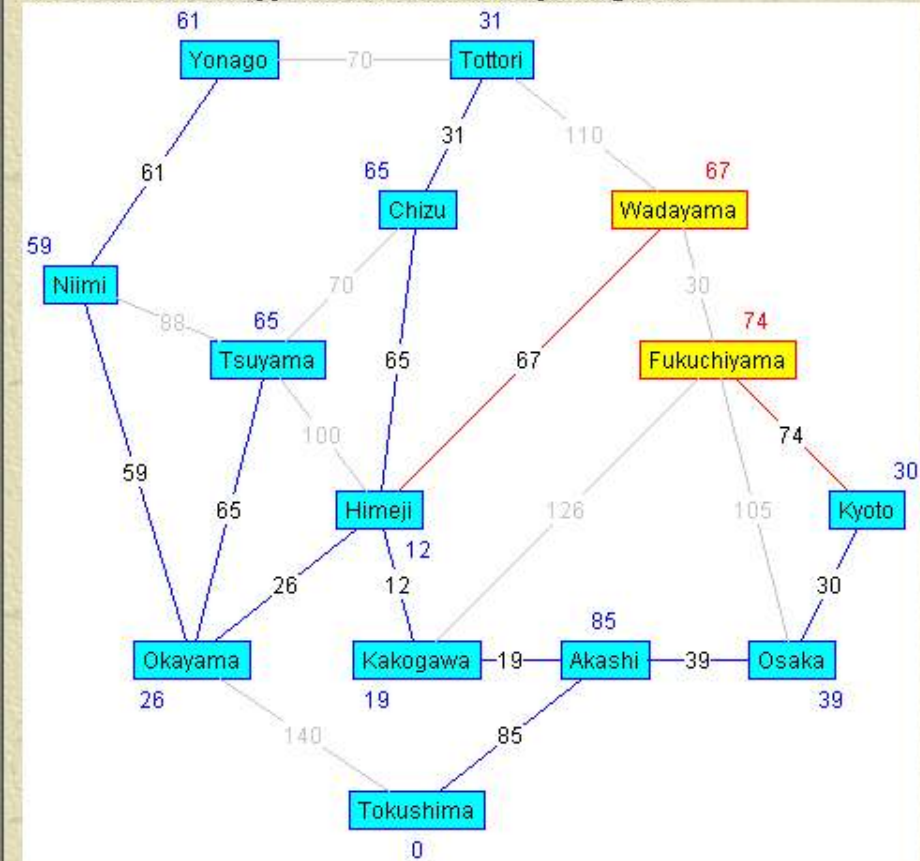
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

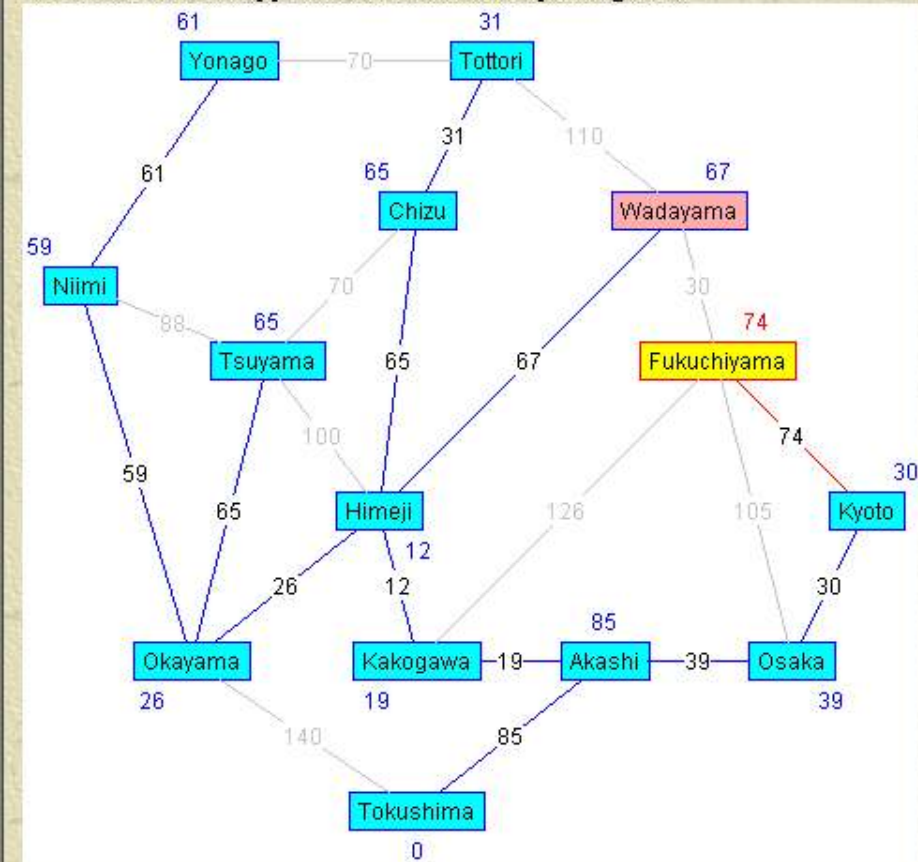
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

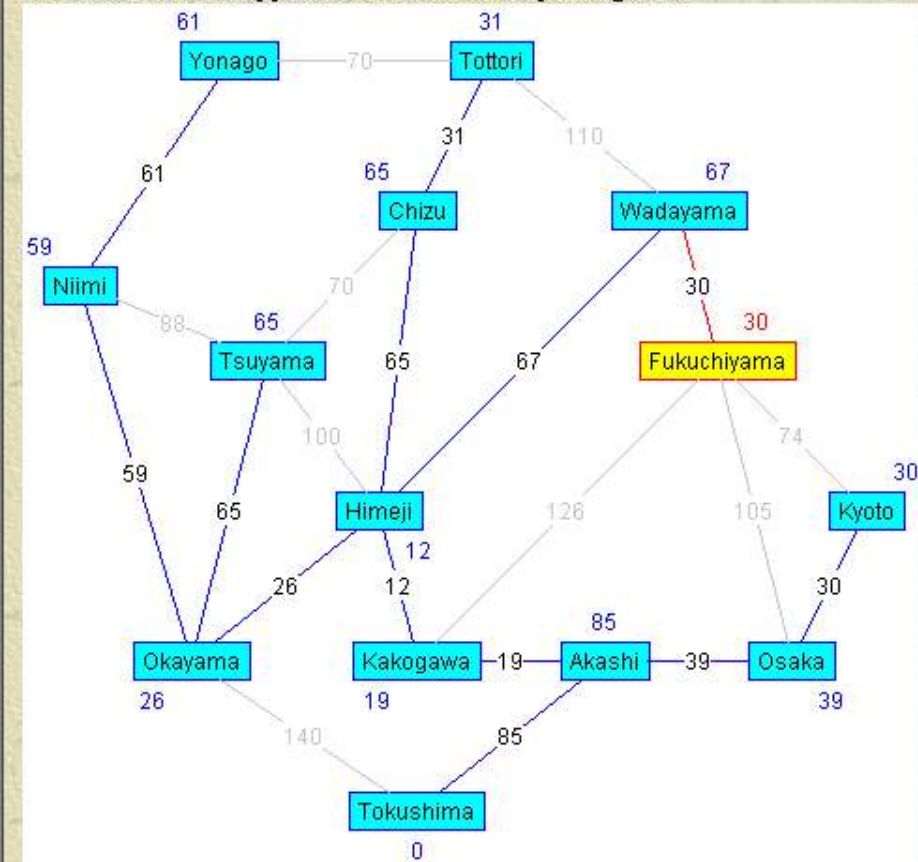
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

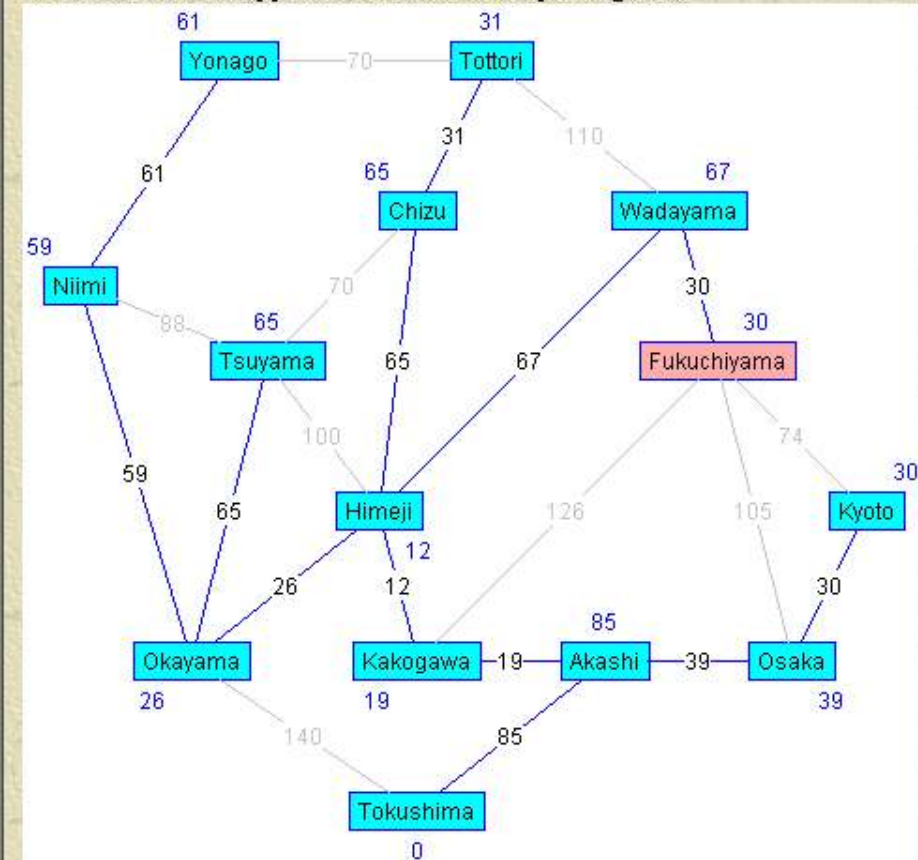
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

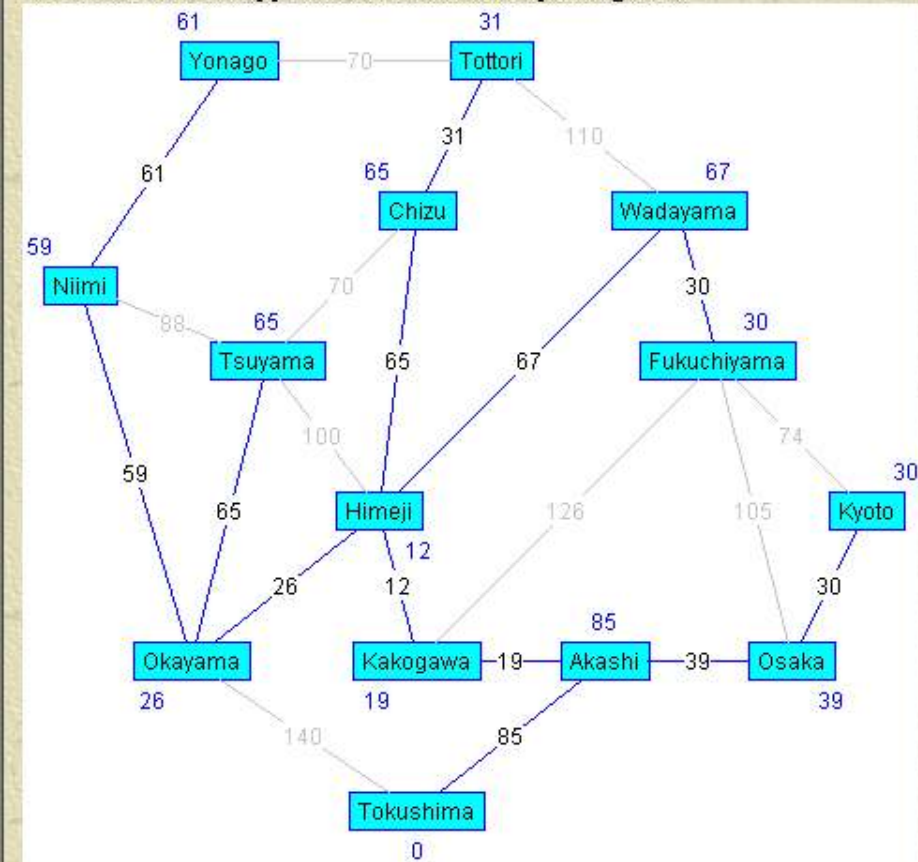
Click on the below applet to find a minimum spanning tree.



Applet written by: [Kenji Ikeda](#)

Java Applet Demo of Prim's Algorithm

Click on the below applet to find a minimum spanning tree.



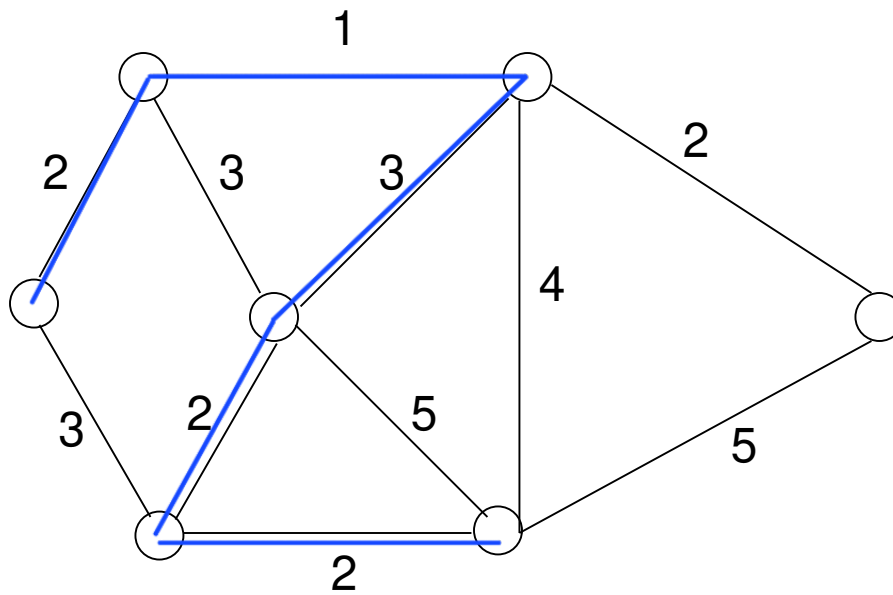
Applet written by: [Kenji Ikeda](#)

→ Bu da spanning tree'yi bulmaya geliriz

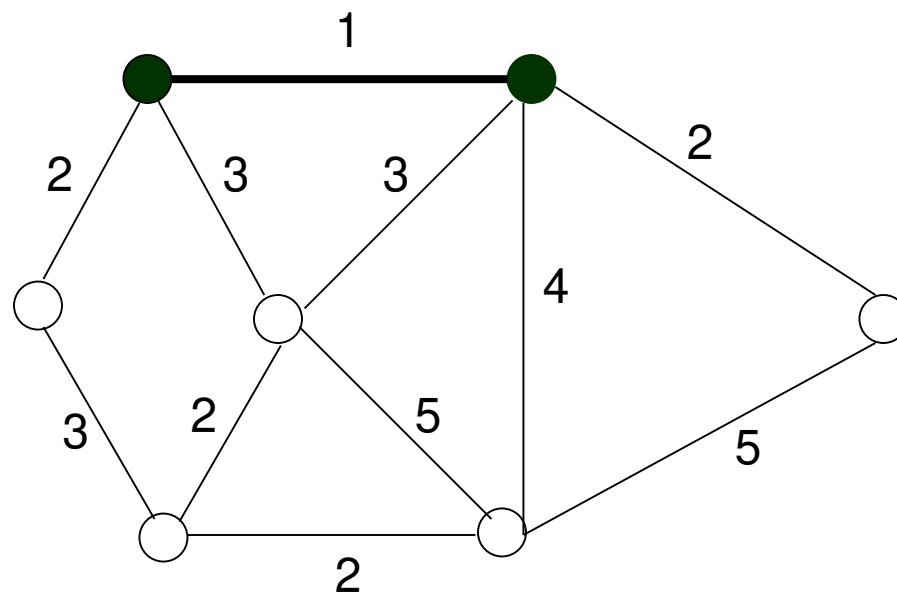
Kruskal's Algorithm

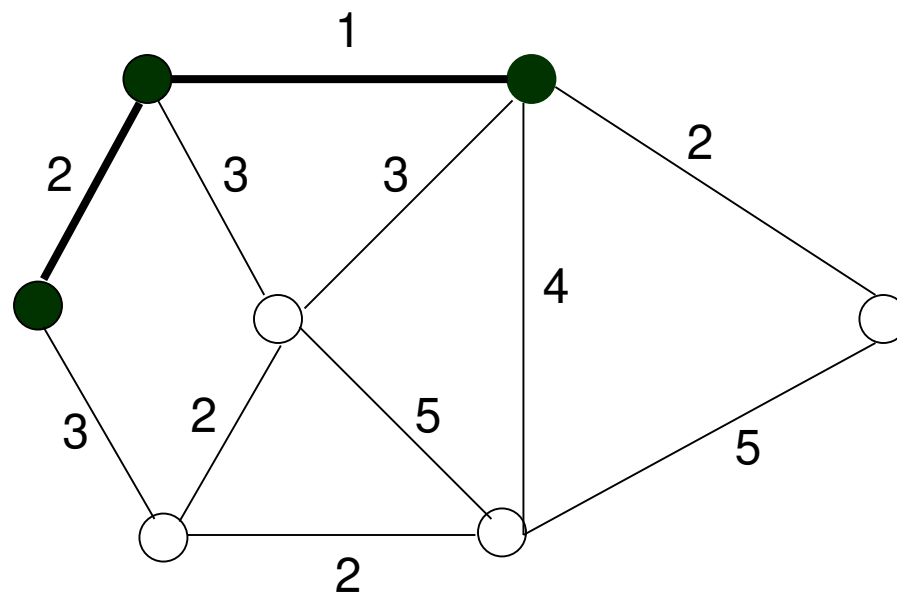
- Adım 1: Graftaki en küçük ağırlık değerine sahip kenarı bul (eğer birden fazla varsa rastgele birini seç)
 - Adım 2: En küçük değere sahip bir sonraki kenarı bul ancak kapalı bir döngü oluşturmamasın
 - Adım 3: Graftaki her bir düğümü dolaşana kadar adım-2 yi yap

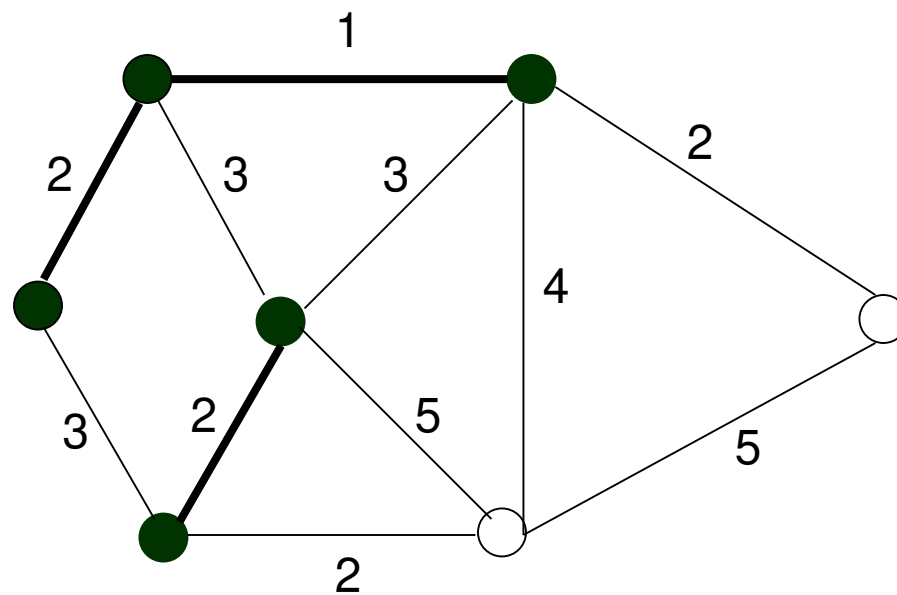
n düğüm için n-1 kenar seçilmelidir

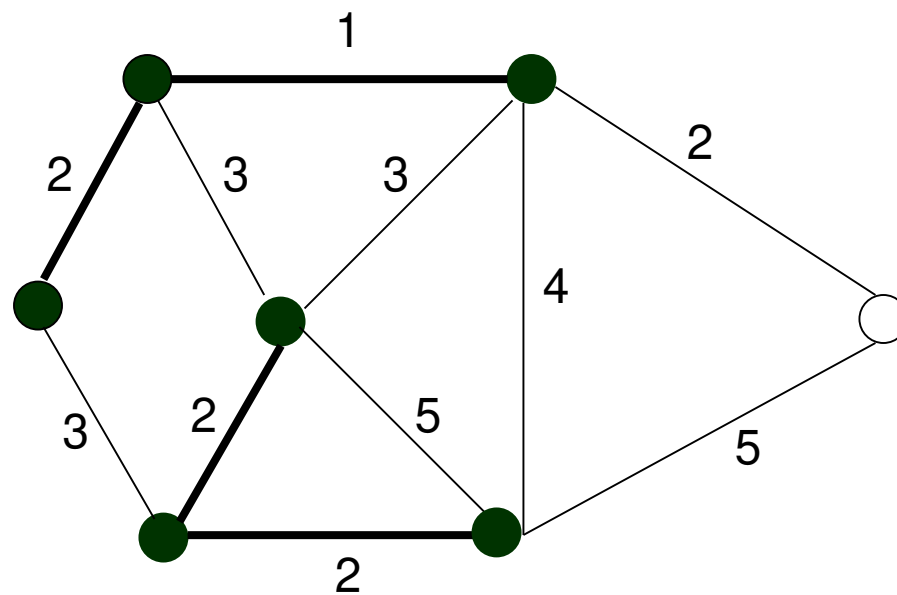


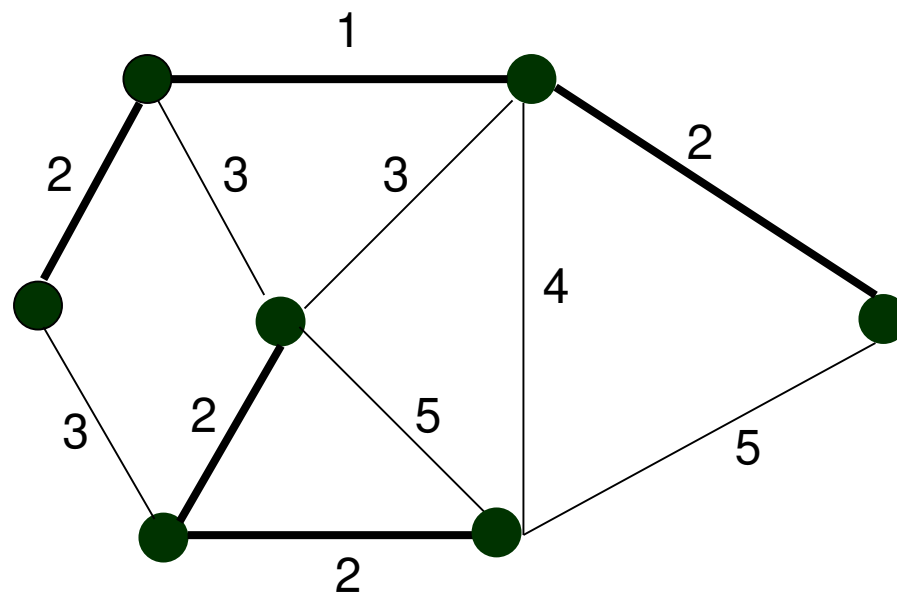
~~1~~
~~2~~
~~2~~
~~2~~
~~2~~
3
3
3
4
5
5

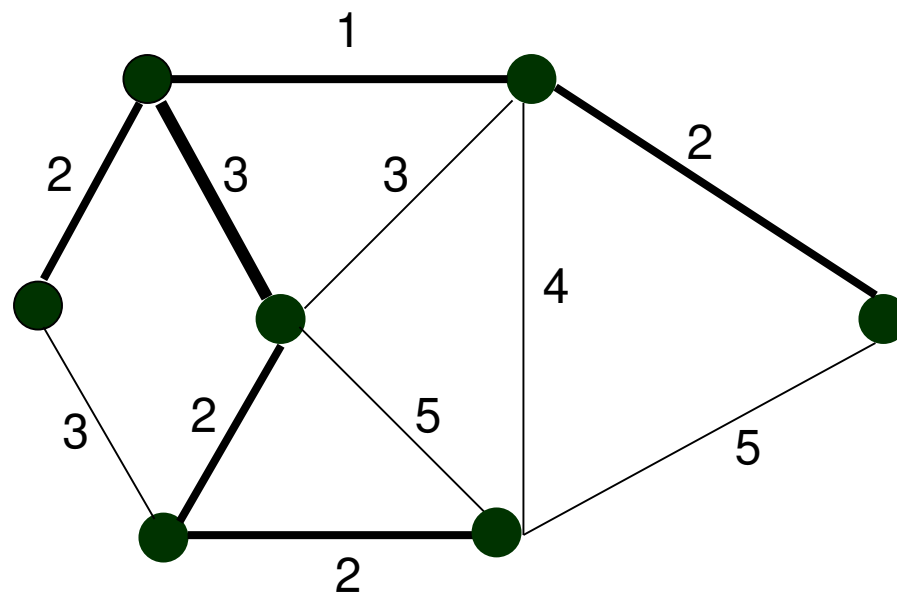




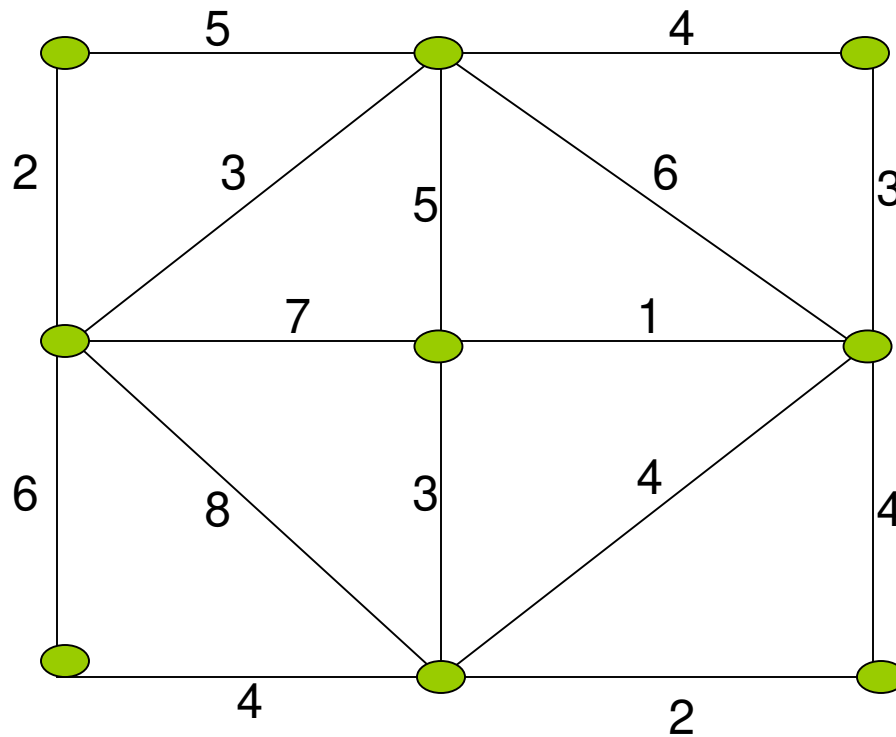








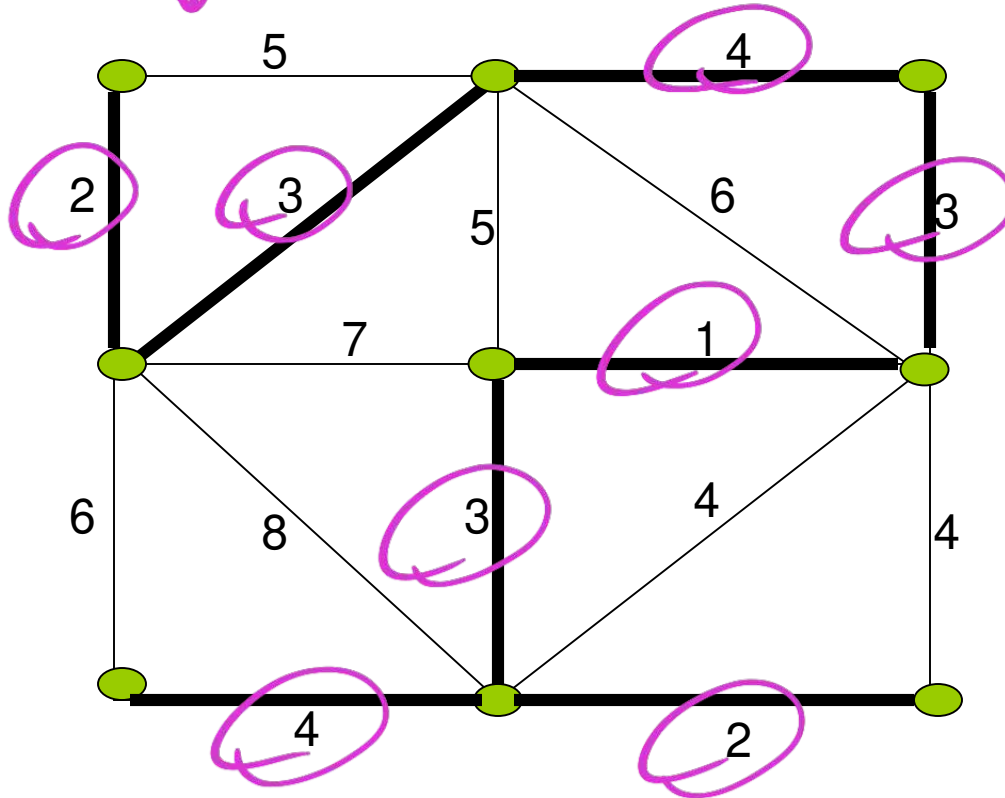
Örnek



Kısa yol problemi

Kontrol
Junla
yaparsın
çözüm

n düğümlü
 $n-1$
kenc seçilmeli

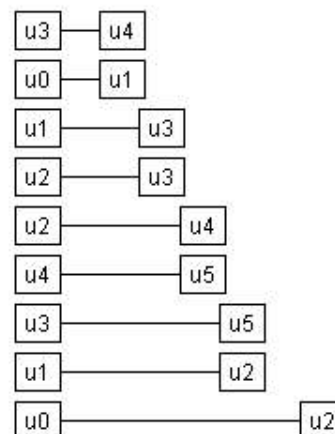
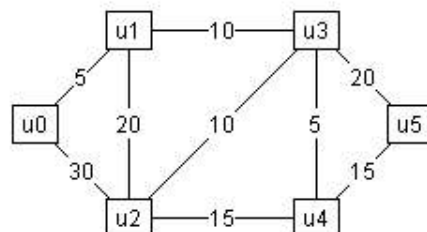


**Mathematical
Programming**[Simplex](#)[Twophase](#)[Dijkstra](#)[Prim](#)[Kruskal](#)[Ford-Fulkerson](#)Kruskal
Java applet demos:

- [demo1](#)
- [demo2](#)
- [demo3](#)
- [demo4](#)
- [demo5](#)

[FAQ](#)[Kenji Ikeda's
Home Page](#)**Java Applet Demos of Kruskal's Algorithm**

Click on the applet below to find a minimum spanning tree. (Not on the right one.)



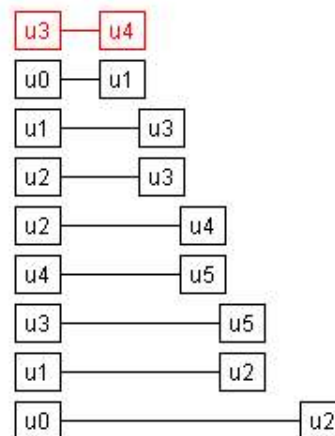
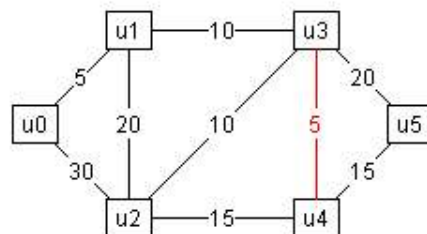
Last Modified: Friday, 24-May-2002 12:17:20 JST

**Mathematical
Programming**[Simplex](#)[Twophase](#)[Dijkstra](#)[Prim](#)[Kruskal](#)[Ford-Fulkerson](#)Kruskal
Java applet demos:

- [demo1](#)
- [demo2](#)
- [demo3](#)
- [demo4](#)
- [demo5](#)

[FAQ](#)[Kenji Ikeda's
Home Page](#)**Java Applet Demos of Kruskal's Algorithm**

Click on the applet below to find a minimum spanning tree. (Not on the right one.)



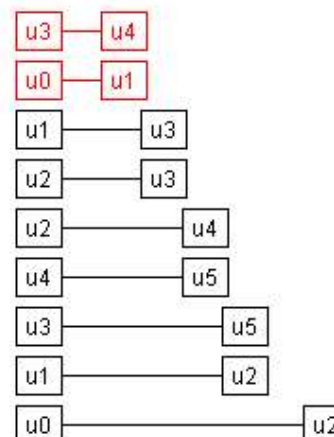
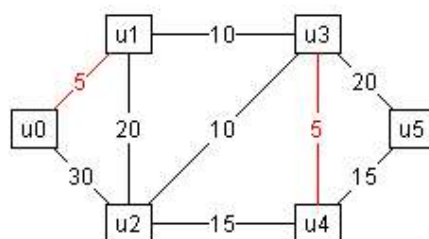
Last Modified: Friday, 24-May-2002 12:17:20 JST

Mathematical ProgrammingSimplexTwophaseDijkstraPrimKruskalFord-FulkersonKruskal
Java applet demos:

- [demo1](#)
- [demo2](#)
- [demo3](#)
- [demo4](#)
- [demo5](#)

FAQKenji Ikeda's Home Page**Java Applet Demos of Kruskal's Algorithm**

Click on the applet below to find a minimum spanning tree. (Not on the right one.)



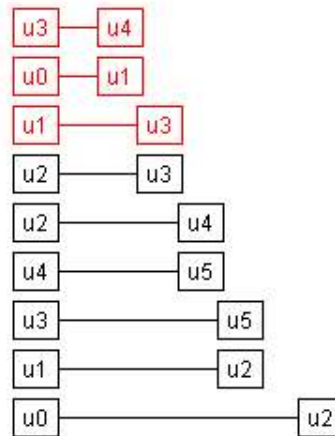
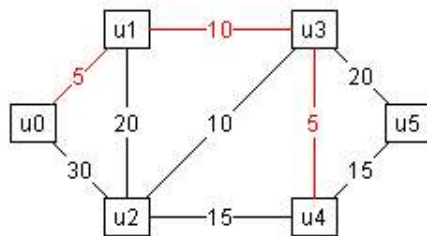
Last Modified: Friday, 24-May-2002 12:17:20 JST

**Mathematical
Programming**[Simplex](#)[Twophase](#)[Dijkstra](#)[Prim](#)[Kruskal](#)[Ford-Fulkerson](#)Kruskal
Java applet demos:

- [demo1](#)
- [demo2](#)
- [demo3](#)
- [demo4](#)
- [demo5](#)

[FAQ](#)[Kenji Ikeda's
Home Page](#)**Java Applet Demos of Kruskal's Algorithm**

Click on the applet below to find a minimum spanning tree. (Not on the right one.)



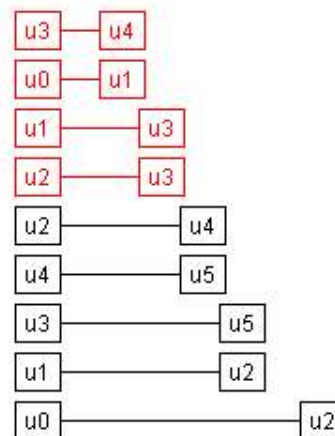
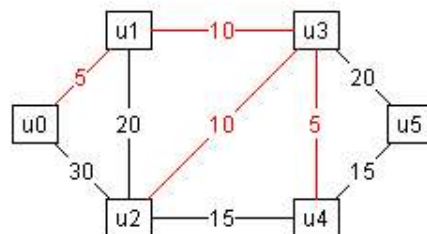
Last Modified: Friday, 24-May-2002 12:17:20 JST

**Mathematical
Programming**[Simplex](#)[Twophase](#)[Dijkstra](#)[Prim](#)[Kruskal](#)[Ford-Fulkerson](#)Kruskal
Java applet demos:

- [demo1](#)
- [demo2](#)
- [demo3](#)
- [demo4](#)
- [demo5](#)

[FAQ](#)[Kenji Ikeda's
Home Page](#)**Java Applet Demos of Kruskal's Algorithm**

Click on the applet below to find a minimum spanning tree. (Not on the right one.)



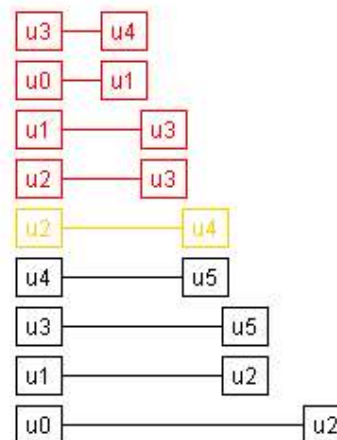
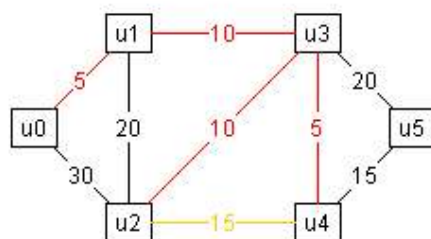
Last Modified: Friday, 24-May-2002 12:17:20 JST

Mathematical ProgrammingSimplexTwophaseDijkstraPrimKruskalFord-FulkersonKruskal
Java applet demos:

- [demo1](#)
- [demo2](#)
- [demo3](#)
- [demo4](#)
- [demo5](#)

FAQKenji Ikeda's Home Page**Java Applet Demos of Kruskal's Algorithm**

Click on the applet below to find a minimum spanning tree. (Not on the right one.)



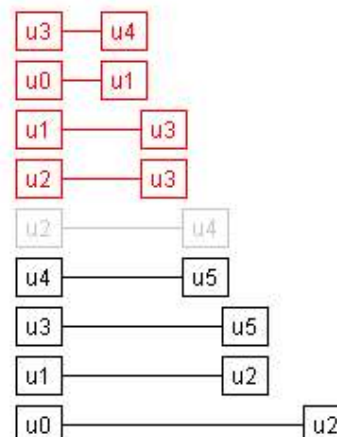
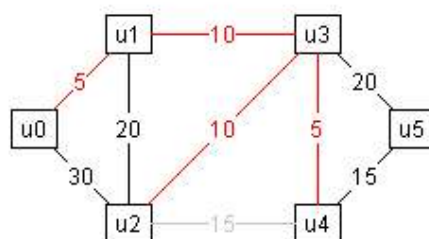
Last Modified: Friday, 24-May-2002 12:17:20 JST

Mathematical ProgrammingSimplexTwophaseDijkstraPrimKruskalFord-FulkersonKruskal
Java applet demos:

- [demo1](#)
- [demo2](#)
- [demo3](#)
- [demo4](#)
- [demo5](#)

FAQKenji Ikeda's Home Page**Java Applet Demos of Kruskal's Algorithm**

Click on the applet below to find a minimum spanning tree. (Not on the right one.)



Last Modified: Friday, 24-May-2002 12:17:20 JST

Mathematical Programming

[Simplex](#)
[Twophase](#)
[Dijkstra](#)
[Prim](#)
[Kruskal](#)
[Ford-Fulkerson](#)

Kruskal
Java applet demos:

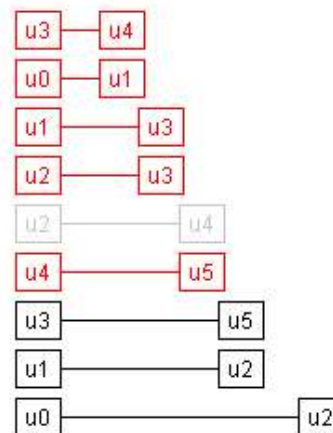
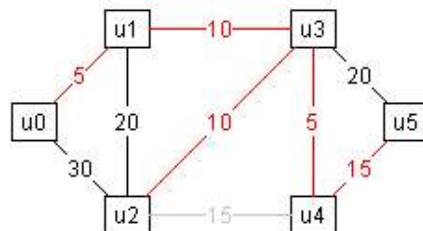
- [demo1](#)
- [demo2](#)
- [demo3](#)
- [demo4](#)
- [demo5](#)

[FAQ](#)

[Kenji Ikeda's Home Page](#)

Java Applet Demos of Kruskal's Algorithm

Click on the applet below to find a minimum spanning tree. (Not on the right one.)



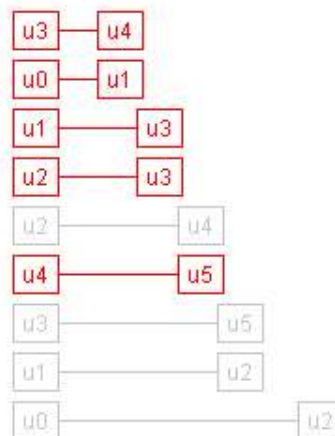
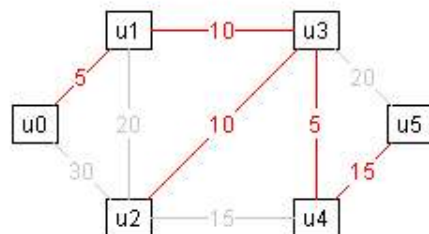
Last Modified: Friday, 24-May-2002 12:17:20 JST

**Mathematical
Programming**[Simplex](#)[Twophase](#)[Dijkstra](#)[Prim](#)[Kruskal](#)[Ford-Fulkerson](#)Kruskal
Java applet demos:

- [demo1](#)
- [demo2](#)
- [demo3](#)
- [demo4](#)
- [demo5](#)

[FAQ](#)[Kenji Ikeda's
Home Page](#)**Java Applet Demos of Kruskal's Algorithm**

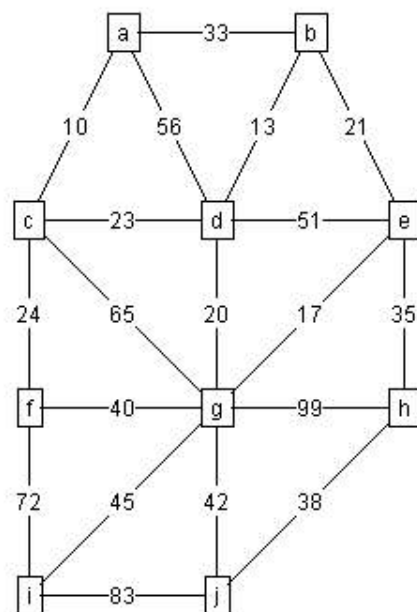
Click on the applet below to find a minimum spanning tree. (Not on the right one.)



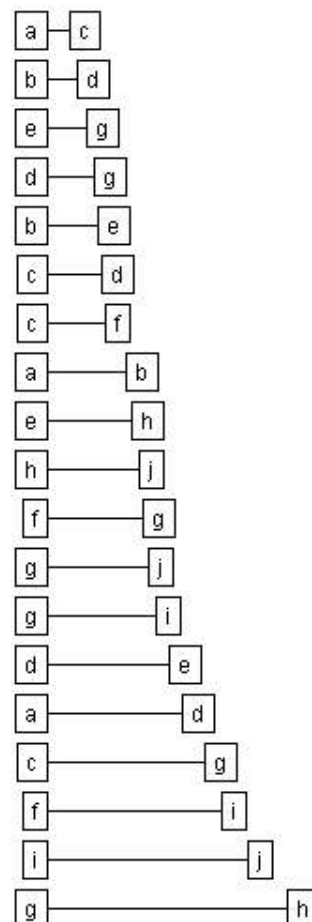
Last Modified: Friday, 24-May-2002 12:17:20 JST

**Mathematical
Programming**[Simplex](#)[Twophase](#)[Dijkstra](#)[Prim](#)[Kruskal](#)[Ford-Fulkerson](#)Kruskal
Java applet demos:

- [demo1](#)
- [demo2](#)
- [demo3](#)
- [demo4](#)
- [demo5](#)

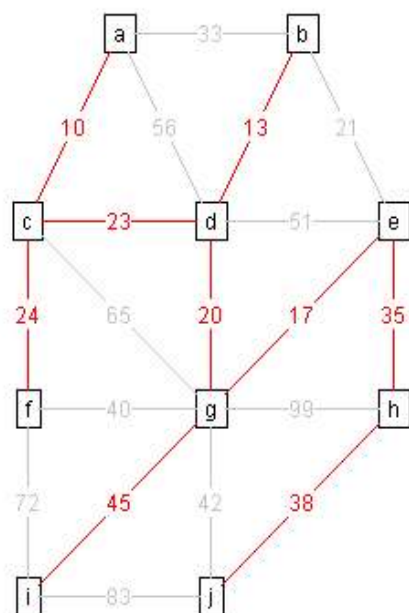
[FAQ](#)**Java Applet Demos of Kruskal's Algorithm**

Click on the applet above to find a minimum spanning tree. (Not on the right one.)

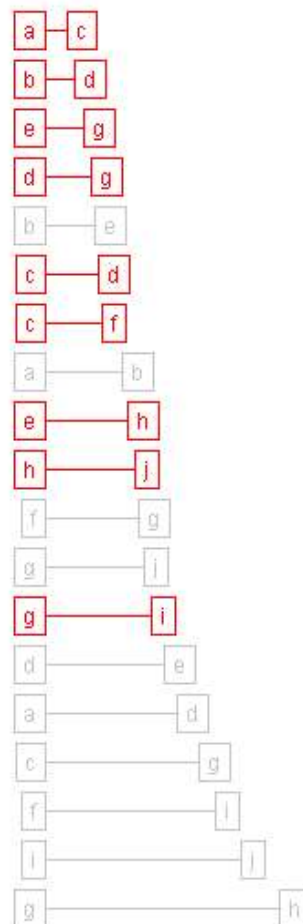


Mathematical ProgrammingSimplexTwophaseDijkstraPrimKruskalFord-FulkersonKruskal
Java applet demos:

- [demo1](#)
- [demo2](#)
- [demo3](#)
- [demo4](#)
- [demo5](#)

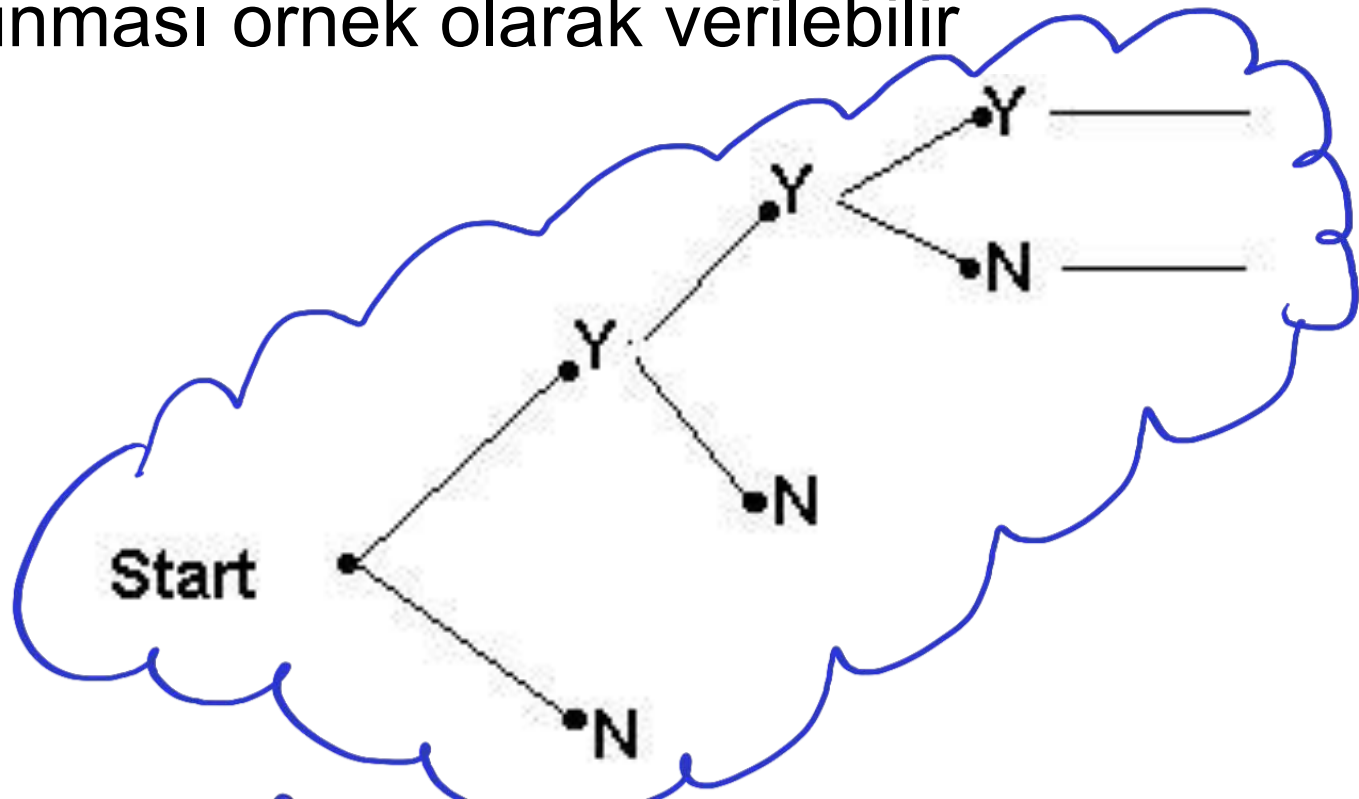
FAQ**Java Applet Demos of Kruskal's Algorithm**

Click on the applet above to find a minimum spanning tree. (Not on the right one.)



Karar Ağaçları (Decision trees)

Karar Ağaçları, bir Binary Tree olup bir olayın sonuçlandırılmasında sorunun cevabına göre hareket ederler. Binary Search, Hileli paranın bulunması örnek olarak verilebilir



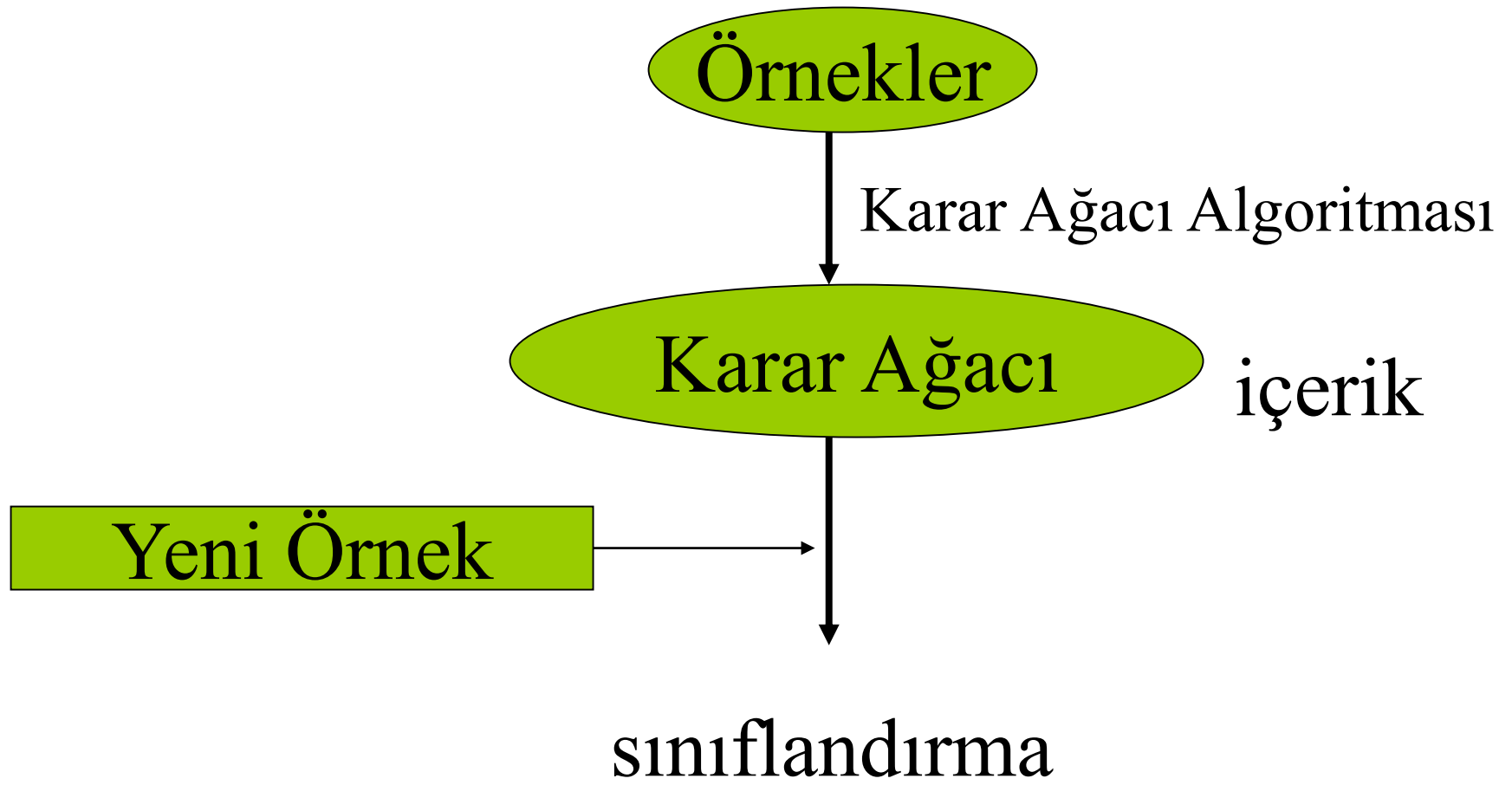
↳ türetili
bir soru sorup
sorunun cevabına
göre dallanıyor sam
ben aslında bir karar
ağacı oluşturmuyum
demektir

Tümevarımsal Sonuç Çıkarım

(inductive inference) için kullanılan

en popüler yöntemlerden birisi

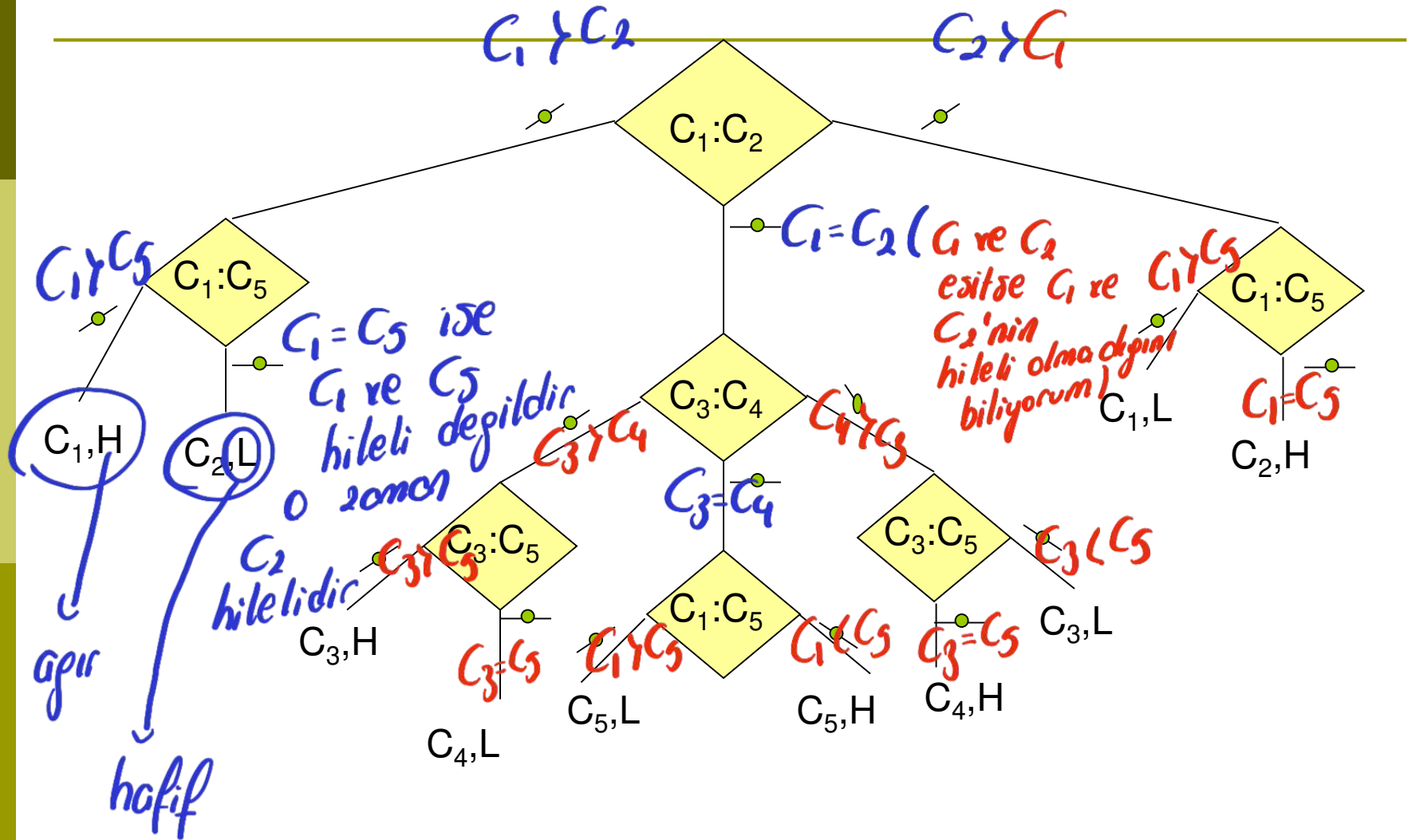
Karar Ağaç'larıdır (Decision Tree)



FIVE-COINS PUZZLE

- Görünüřleri birbirinin aynı olan 5 paradan biri sahtedir.
- Sahte olan para dięerlerinden daha ağır veya daha hafif olabilir.
- Karar ağacını kullanarak hangi paranın sahte olduğuna karar veriniz
- Sahte olan para dięerlerine göre hafif midir yoksa ağır mıdır?

K A R A R ? ? ? ?

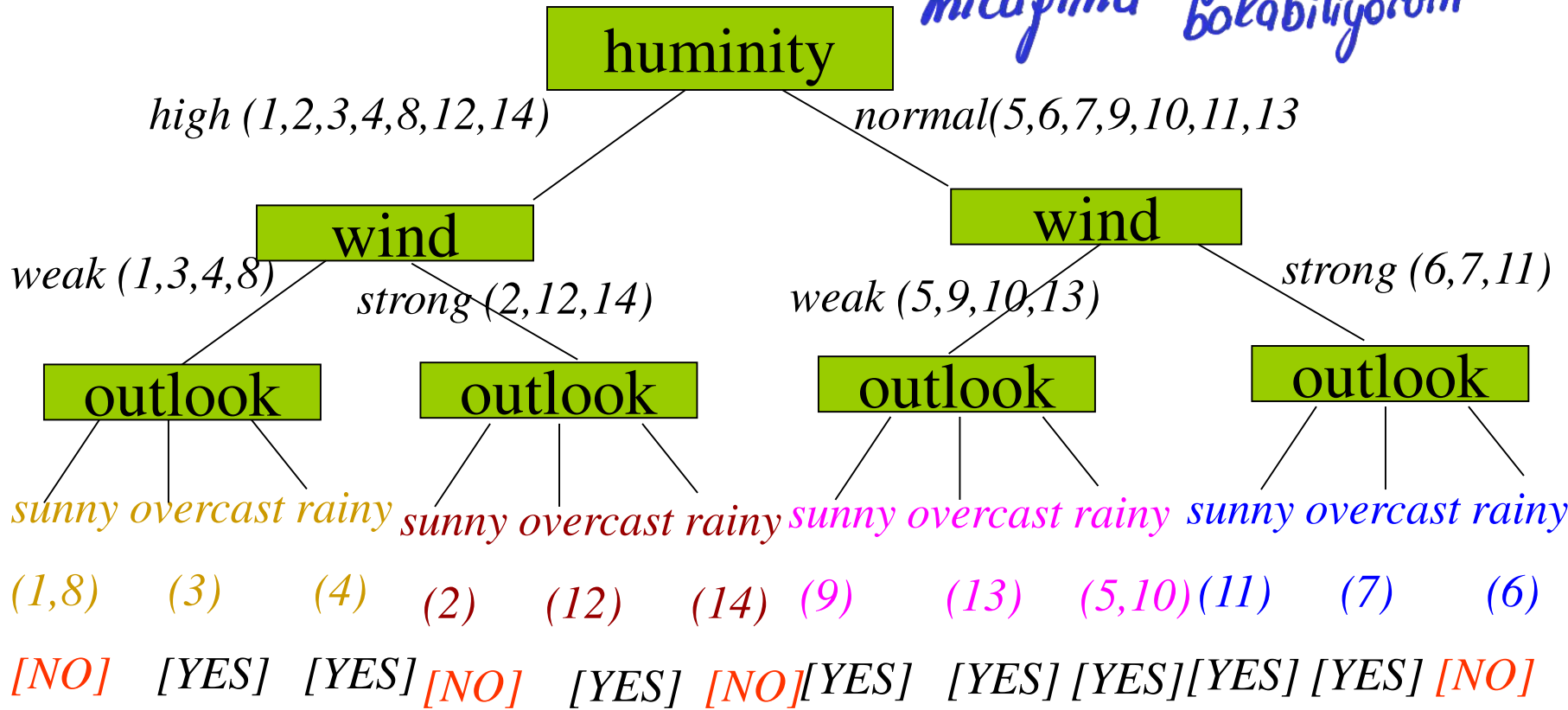


Play Tennis ?????

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
1	sunny	hot	high	weak	no
2	sunny	hot	high	strong	no
3	overcast	hot	high	weak	yes
4	rain	mild	high	weak	yes
5	rain	cool	normal	weak	yes
6	rain	cool	normal	strong	no
7	overcast	cool	normal	strong	yes
8	sunny	mild	high	weak	no
9	sunny	cool	normal	weak	yes
10	rain	mild	normal	weak	yes
11	sunny	mild	normal	strong	yes
12	overcast	mild	high	strong	yes
13	overcast	hot	normal	weak	yes
14	rain	mild	high	strong	no

Özelliklerden biri kök seçilir (outlook, temp, humidity, wind)

İki özelliğe temp konulmamış demek-
ti temp olmadan da tenis oynayıp oyna-
mıyıpma bakabiliyorum



minimum değeri
tarafından
bu ağacı oluşturmanız
gerekli

Karar Ağacı iyi bir çözümdür
ancak

optimum değildir

optimum bir karar ağacının oluşturulması
için **bir kuralın olması gerekir**

aynı şekilde

→ minimum $\rightarrow$ head of

↳ **tenis oynamak**

→ tenis
onomatopoeia

yes No



eğer örneklerin aynı
sınıftıysa entropy'ın 0 olur
yani 14 olurdu

A özelliğinin, S örneği için kazancı (information gain)

$$\text{Gain}(S, A) \equiv \text{Entropy}(S) - \sum P(v) \text{Entropy}(S(v))$$

v: Values of A

$$P(v) \equiv |S(v)| / |S|$$

Attribute
(özellik)

formüle
böyle

Entropy



T, tl, O, w

her bir özelliğin gain'ini hesapla
kuk olarak

her bir özelliğe göre
hangisi büyükse o özelliğe göre

S:[9+,5-]

E = 0.940

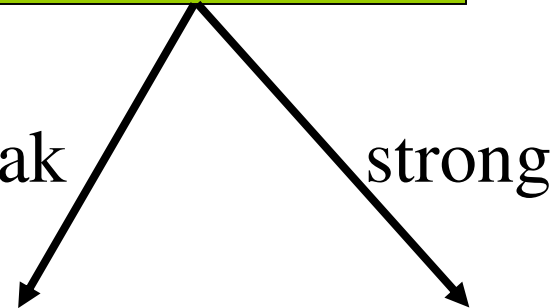
Gain(S,wind) = ?

Gain(S,humidity) = ?

Gain(S,temperature) = ?

Gain(S,outlook) = ?

Wind



[6+,2-]

[3+,3-]

E = 0.811

E = 1.0

Gain(S,wind)

$$\begin{aligned}
 & E(S) - \sum P(w) E(w) \\
 &= 0.940 - (8/14)[-(6/8)\log_2(6/8) - (2/8)\log_2(2/8)] \\
 &\quad - (6/14)[-(3/6)\log_2(3/6) - (3/6)\log_2(3/6)] \\
 &= 0.048
 \end{aligned}$$



$$\text{Gain}(S, \text{wind}) = 0.048$$

$$\text{Gain}(S, \text{humidity}) = 0.15$$

$$\text{Gain}(S, \text{temperature}) = 0.029$$


$$\text{Gain}(S, \text{outlook}) = 0.246$$


outlook

sunny

(1,2,8,9,11)
[+2,-3]

overcast

(3,7,12,13)
[+4,-0]

rainy

(4,5,6,10,14)
[+3,-2]

YES

$S_{\text{sunny}} = [+2, -3]$ sunny'nin entropisi

$E(\text{sunny}) = -(2/5)\log_2(2/5) - (3/5)\log_2(3/5) = 0.97$

$\text{Gain}(S_{\text{sunny}, \text{humidity}}) = ?$

$\text{Gain}(S_{\text{sunny}, \text{temp}}) = ?$

$\text{Gain}(S_{\text{sunny}, \text{wind}}) = ?$

bu alıktı
için demek ki

temp nem sıcaklık
?
hazır
bakıyor

?
cinin
en üs-
tüne
bu gelecek

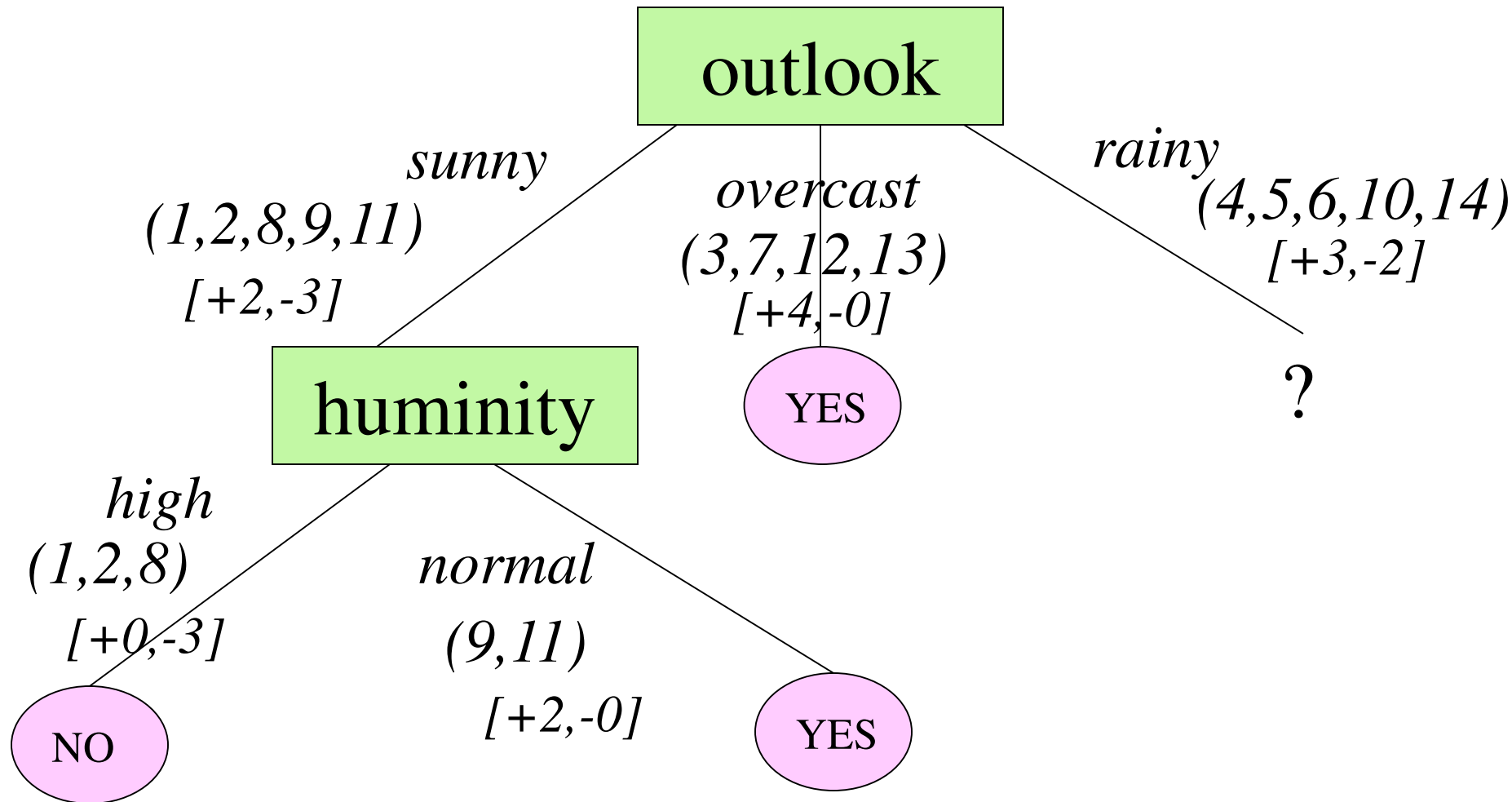
$$\text{Gain}(S_{\text{sunny,humidity}}) = 0.97 - (2/5)[-(2/2)\log_2(2/2) - (0/2)\log_2(0/2)] \\ - (3/5)[-(3/3)\log_2(3/3) - (0/3)\log_2(0/3)]$$

$$= 0.97$$

en yüksek
bu ağırlık

$$\text{Gain}(S_{\text{sunny,wind}}) = 0.019$$

$$\text{Gain}(S_{\text{sunny,temp}}) = 0.57$$



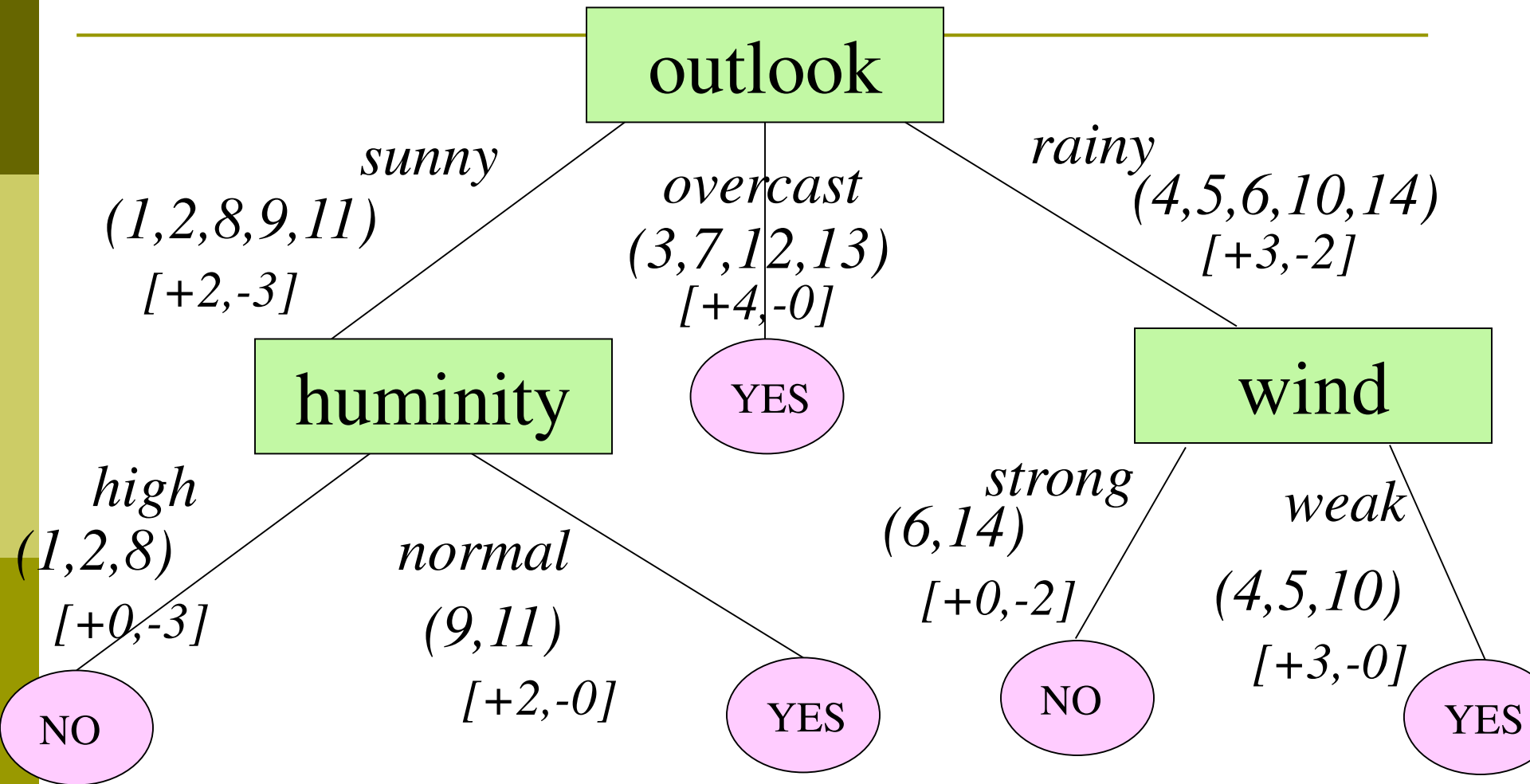
Aynı işlem

$$\text{Gain}(S_{\text{rainy,humidity}}) = ?$$

$$\text{Gain}(S_{\text{rainy,temp}}) = ?$$

$$\text{Gain}(S_{\text{rainy,wind}}) = ?$$

bulmak için yapılır.



Ağaçlarda Benzerlik (Isomorphism)

Bunlar isomorfik

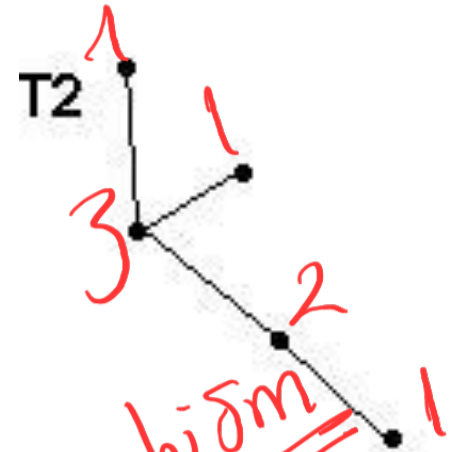
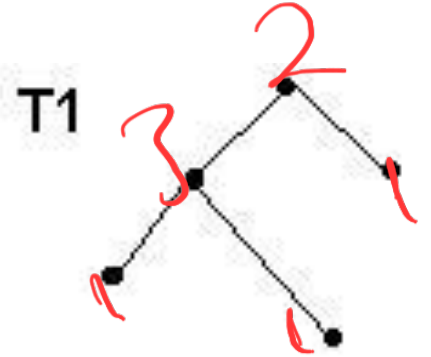
Verilen iki ağaç T_1 ve T_2 olsun

□ T_1 ve T_2 isomorphic dir

□ Bire-Bir ve örten fonksiyon
özelliklerini görmemiz gerekir

$$f : T_1 \rightarrow T_2$$

□ T_1 ve T_2 birbirlerine çok yakındır



isomorphism

düğüm sayıları
kenar sayıları
düğüm dereceleri
yakınlık matrisleri

Köklü Ağaçlarda (Isomorphism)

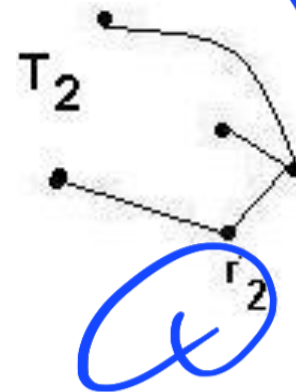
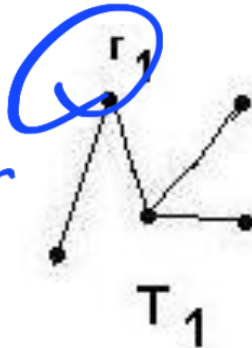
T_1 ve T_2 , r_1 ve r_2 kök değerine sahip köklü iki ağaç olsun. Eğer

■ Bire-Bir fonksiyon özelliği mevcut ise

$$f: V(T_1) \rightarrow V(T_2)$$

Örnek:

■ T_1 ve T_2 , köklü bir ağaç olarak isomorphic



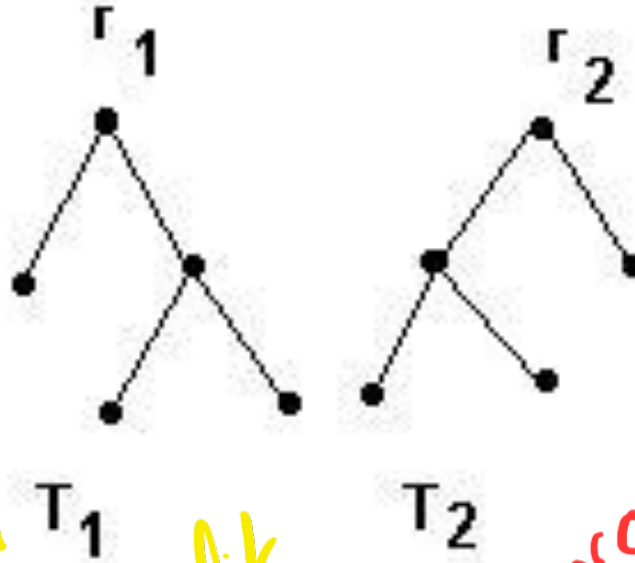
agaclarda
kök bil
gibi bana
verilmiş-
se
burası
veril-
memişse
eki

Bunları ise
söyle onlara
bunları köklerin
den tutup çözüm
gibi ver

Devam...

Örnek: aşağıdaki iki ağaç

- isomorphic köklü bir ağaç olarak
- not isomorphic binary tree olarak



Bu iki ağaç
için kök
bilgisi veril-
medüğünde
isomorfik
diyebilirim

Kök bilgisi
verilirse isomorfik
değildir

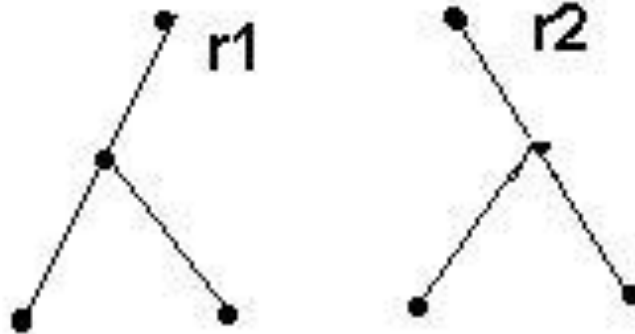
çünkü
sıra dışı
kök
bilgisi
tree'ler
midir

çünkü
slogt

Özet...

Ağaçlarda 3 çeşit isomorphism vardır

- Ağaçlarda Isomorphism
- Köklü ağaçlarda Isomorphism
- Binary Tree'de Isomorphism



Two binary trees
isomorphic as rooted trees,
not as binary trees

bunlar
eğer
binary
tree ise
isomorfik
dildir

Yeni birine
diyebiliriz
pek: 20
koku 11
12 olan
isomorfik
isomorfik
Aynı tree lerde
biri doğru
bitmiş 300
doğru bitmiş
değil bitmiş
Peki
Amorfoz oarb-
rin he? oarb-
yönde ilerle-
mesi gerekir.

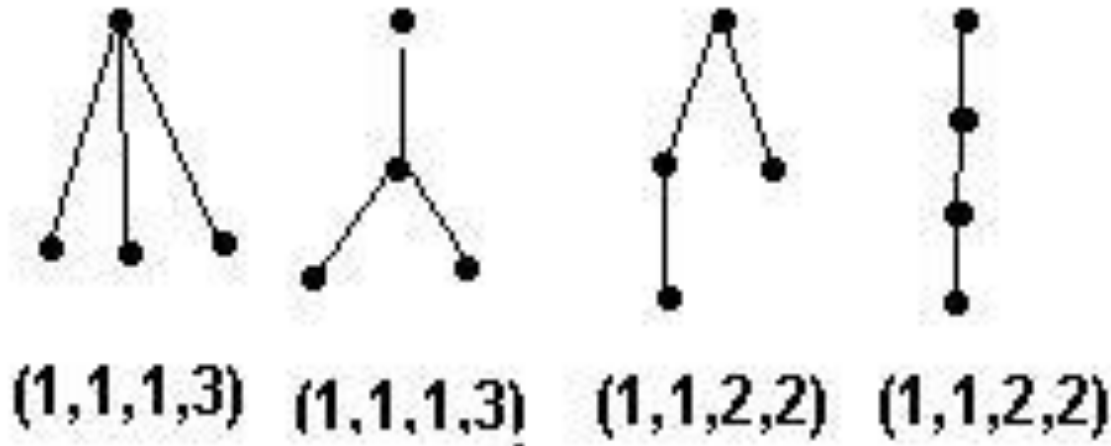
Ağaçlarda Non-isomorphism

- ❑ Çoğu zaman iki ağaçda not isomorphism olduğunu göstermek ağaçlarda isomorphism olduğunu göstermekten daha kolaydır
- ❑ İki ağaç arasında isomorphism aramak için aşağıdaki şartların oluşması beklenir

- Düğüm sayıları aynı olmalı
- Kenar sayıları aynı olmalı
- Karşılıklı düğümlerin dereceleri aynı olmalı
- Kökten köke eşleme yapılmalı (rooted tree)
- Çocukların(child-leave) pozisyonları aynı olmalı (binary tree)

Köklü Ağaçlarda Non-isomorphism

- En tepedeki düğümü kök olarak kabul edersek düğüm ve kenar sayıları eşit olsa bile bu köklü ağaçlar non-isomorphism dir
- Köklü ağaç özelliği verilmeseydi, 1 ve 2. ağaç, 3 ve 4. ağaç kendi aralarında isomorphism olacaktı



Binary Tree'lerde Non-isomorphic

Theorem

- n düğümlü, $C(2n, n) / (n+1)$ adet non-isomorphic ikili ağaç(binary tree) vardır, $n \geq 0$

n tane düğümü olan ikili ağaç binary tree yazarsanız kaç tane bilirsin

isomorfizm olmuyor farklı şekillerde gösteriliyor sak isomorfizm olmaz

bu da isomorfik değil



5 nonisomorphic binary trees with 3 vertices

$C(6, 3)$

3 farklı düğüme sahip binary tree çizebiliriz

Handwritten calculations:

$$\frac{6!}{3! 3! \cdot 4}$$

$$\frac{6 \cdot 5 \cdot 4}{3 \cdot 3 \cdot 4}$$

